

Hin zu optimierten Microservice- Choreografien

Evaluierung gängiger Java-Frameworks in Cloud-nativen verteilten Systemen

Masterarbeit

Eingereicht in teilweiser Erfüllung der Anforderungen zur Erlangung des akademischen Grades:

Master of Science in Engineering

An der Fachhochschule FH Campus Wien

Masterstudiengang: Software Design and Engineering

Vorgelegt von:

Esad Hrbatović

Personenkennzeichen

2310838033

Betreuer:

Bernhard Taufner, BSc, MSc

Eingereicht am:

15. 08. 2025

Erklärung:

Ich erkläre, dass die vorliegende Abschlussarbeit von mir selbst verfasst wurde und ich keine anderen als die angeführten Behelfe verwendet bzw. mich auch sonst keiner unerlaubten Hilfe (wie z.B. ChatGPT oder ähnlichen auf künstlicher Intelligenz basierenden Programmen) bedient habe. Ich versichere, dass diese Arbeit keine personenbezogenen Daten enthält und dass ich sämtliche urheber-, lizenz- sowie bildrechtliche Fragen im Zusammenhang mit der elektronischen Veröffentlichung dieser Arbeit geklärt habe, widrigenfalls werde ich die FH Campus Wien von Ersatzansprüchen Dritter schad- und klaglos halten. Ich versichere, dass ich diese Abschlussarbeit bisher weder im In- noch im Ausland (einer Beurteilerin/einem Beurteiler zur Begutachtung) in irgendeiner Form als Prüfungsarbeit vorgelegt habe und dass die von mir eingereichten Exemplare (ausgedruckt und elektronisch) identisch sind.

Datum: 15. 08. 2025

Unterschrift:

A handwritten signature in black ink, reading "Esad Hribatovic". The signature is written in a cursive style with a horizontal line underneath the name.

Kurzfassung

Diese Arbeit beschäftigt sich mit choreografiebasierten Microservice-Architekturen und untersucht die Leistungsfähigkeit der führenden Java-Frameworks Quarkus, Micronaut und Spring Boot in diesem Kontext. Zudem wird evaluiert inwiefern die AOT-Kompilierung mit GraalVM und Native-Images eine Optimierung von Leistung und Ressourcenverbrauch ermöglichen. Choreografiebasierte Microservice-Architekturen setzen auf lose Kopplung und dezentrale Koordination um Skalierbarkeit und Wartbarkeit zu erhöhen. GraalVM und Native-Images sind eine vergleichsweise neue Technologie, die in den letzten Jahren vermehrt zur Reduktion von Startzeiten und Ressourcenverbrauch eingesetzt wird. Daher fokussiert sich diese Arbeit bei der Untersuchung der Forschungsfrage auf einen systematischen Vergleich quantitativer Metriken wie Startzeit, Speicherverbrauch, CPU-Effizienz und Verarbeitungszeit. Zusätzlich werden qualitative Aspekte und Herausforderungen der jeweiligen Frameworks bei der Umsetzung und Bereitstellung als nativ kompilierte Anwendungen berücksichtigt und miteinander verglichen. Als Grundlage dient ein praxisnaher Prototyp eines E-Shops für digitale Güter. Dessen Microservice-Architektur basiert auf ereignisgesteuerter Kommunikation gestützt durch Apache Kafka und wendet etablierte Entwurfsmuster an. Dazu gehören das API-Gateway, Event Driven Architecture (EDA), Saga-Pattern, Database per Service und Command Query Responsibility Segregation (CQRS). Es resultieren sechs Ausprägungen des Prototyps, da die mit den drei Frameworks umgesetzten Lösungen sowohl in einer klassischen JVM-Umgebung als auch als GraalVM-Native-Images bereitgestellt werden.

Bei den anschließenden Messungen zeigt sich, dass alle drei Frameworks durch den Einsatz von nativer Kompilierung Leistungsgewinne erzielen. In Bezug auf die CPU-Zeit, erreichen die nativen Varianten von Micronaut und Spring Boot mit ~ 45 s den besten Wert während Quarkus bei ~ 56 s liegt. Beim Arbeitsspeicher zeigt Quarkus Native mit ca. 76 MB (statt 252 MB) die beste Reduktion von 70 %, gefolgt von Micronaut (106 MB statt 288 MB) und Spring Boot (129 MB statt 360 MB). Quarkus verkürzt die Startzeit von ~ 1487 ms auf ~ 54 ms, Micronaut von ~ 1495ms auf 155 ms und Spring Boot von ~ 4000 ms auf 238 ms. Bei der Verarbeitungszeit führt Micronaut ebenfalls mit 12,8 ms vor Spring Boot (13,4 ms) und Quarkus (18,7 ms). Micronaut in der nativen Ausführung liefert bei der direkten Gegenüberstellung über alle Kategorien hinweg die beste Gesamtleistung.

Abstract

This thesis deals with choreography-based microservice architectures and examines the performance of the leading Java frameworks Quarkus, Micronaut, and Spring Boot in this context. It also evaluates the extent to which AOT compilation with GraalVM and Native-Images enables optimization of performance and resource consumption. Choreography-based microservice architectures rely on loose coupling and decentralized coordination to increase scalability and maintainability. GraalVM and Native-Images are a relatively new technology that has been increasingly used in recent years to reduce startup times and resource consumption. Therefore, the work focuses on a systematic comparison of quantitative metrics such as startup time, memory consumption, CPU efficiency, and processing time when investigating the research question. In addition, qualitative aspects and challenges of the respective frameworks in the implementation and deployment as natively compiled applications are taken into account and compared with each other. A practical prototype of an e-shop for digital goods serves as the basis. Its microservice architecture is based on event-driven communication supported by Apache Kafka and applies established design patterns. These include the API gateway, event-driven architecture (EDA), saga pattern, database per service, and command query responsibility segregation (CQRS). This results in six versions of the prototype, as the solutions implemented with the three frameworks are provided both in a classic JVM environment and as GraalVM native images.

Subsequent measurements show that all three frameworks achieve performance gains through the use of native compilation. In terms of CPU time, the native versions of Micronaut and Spring Boot achieve the best value at ~ 45 CPU s, while Quarkus comes in at ~ 56 s. In terms of memory, Quarkus Native shows the best reduction of 70 % with approx. 76 MB (instead of 252 MB), followed by Micronaut (106 MB instead of 288 MB) and Spring Boot (129 MB instead of 360 MB). Quarkus reduces the start-up time from ~ 1487 ms to ~ 54 ms, Micronaut from ~ 1495 ms to 155 ms, and Spring Boot from ~ 4000 ms to 238 ms. Micronaut also leads in processing time with 12.8 ms, ahead of Spring Boot (13.4 ms) and Quarkus (18.7 ms). Micronaut in its native version delivers the best overall performance in a direct comparison across all categories.

Abkürzungsverzeichnis

2PC	Two-Phase-Commit
AES	Advanced Encryption Standard
ACID	Atomicity, Consistency, Isolation, and Durability
AOT	Ahead of Time
API	Application Programming Interface
aud	Audience
AWS	Amazon Web Services
BCrypt	Blowfish Crypt
CBC	Cipher Block Chaining
CDI	Contexts and Dependency Injection
CI/CD	Continuous-Integration- und Continuous-Deployment
CLR	Common Language Runtime
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
CQRS	Command-Query Responsibility Segregation
DDD	Domain Driven Design
DNS	Domain Name System
EDA	Event Driven Architecture
ELB	Elastic Load Balancing
ESS	Event Sourced Systems
exp	Expired
GCM	Galois/Counter Mode
GCP	Google Cloud Platform
HMAC	Hash-based Message Authentication Code
IR	Intermediate Representation
IoT	Internet of Things
iss	Issuer
IV	Initialisierungsvektor
JAR	Java Archive
JIT	Just-in-Time
JVM	Java Virtual Machine
JVMCI	JVM Compiler Interface
JWT	JSON Web Token
JWE	JSON Web Encryption
JWS	JSON Web Signature

jti	JWT ID
NATS	Neural Autonomic Transport System
OIDC	OpenID Connect
ORM	Object Relational Mapping
REST	Representational State Transfer
SLOs	Service Level Objectives
sub	Subject
SVM	Substrate VM
VM	Virtual Machine

Schlüsselbegriffe

Choreografie

CPU-Effizienz

GraalVM

Microservices

Micronaut

Quarkus

Speicherverbrauch

Spring Boot

Startzeit

Inhaltsverzeichnis

1. EINLEITUNG.....	1
1.1. Forschungsziele und Forschungsfragen	1
1.2. Methodik.....	2
2. RELATED WORK	3
3. GRUNDLAGEN UND THEORETISCHER HINTERGRUND	5
3.1. Monolith und Microservice-Architekturen.....	5
3.1.1. Monolith	5
3.1.2. Microservice Ansatz.....	6
3.2. Java Frameworks.....	8
3.2.1. Quarkus	9
3.2.2. Spring Boot	9
3.2.3. Micronaut	9
3.3. Java Laufzeitumgebungen.....	10
3.3.1. JVM.....	10
3.3.2. GraalVM und Native Image.....	10
3.4. Cloud-native und Serverless	11
4. ORCHESTRIERUNG UND CHOREOGRAFIE	13
4.1. Von zentralen zu verteilten Transaktionen.....	13
4.1.1. Verteilte Transaktionen - Two-Phase-Commit	13
4.1.2. Saga Pattern	14
4.2. Orchestrierung vs. Choreografie.....	16
4.2.1. Orchestrierung – Zentrale Steuerung durch einen Orchestrator	16
4.2.2. Choreografie – Koordination durch Events	17
4.3. Vergleich anhand Skalierbarkeit, Flexibilität und Ausfallsicherheit ..	18
4.3.1. Skalierbarkeit	18
4.3.2. Flexibilität	18
4.3.3. Ausfallsicherheit.....	19
4.3.4. Fazit	19
5. ENTWURFSMUSTER UND BEST PRACTICES	20
5.1. Event Driven Architecture.....	20
5.2. Idempotent Consumer.....	21
5.3. Database per Service	21
5.4. Event Sourcing	22

5.5.	CQRS	22
5.6.	Strangler Fig	23
5.7.	API-Gateway.....	24
5.8.	Domain Driven Design	25
5.9.	Service Discovery	25
5.10.	Circuit Breaker	26
5.11.	Token based authentication	26
5.12.	Observability	28
5.12.1.	Log-Aggregation.....	28
5.12.2.	Sidecar Pattern.....	28
5.12.3.	Distributed Tracing	29
5.13.	Zusammenfassung	30
6.	PROTOTYP IMPLEMENTIERUNG	31
6.1.	Lösungsdesign	31
6.1.1.	Überblick über funktionale Anforderungen	31
6.1.2.	High Level Architecture	31
6.2.	Einsatz von Design Patterns.....	32
6.2.1.	Event-Messaging und Choreografie.....	32
6.2.2.	Saga Pattern	33
6.2.3.	Database per Service – CQRS	34
6.2.4.	API-Gateway.....	35
6.3.	Cross-Cutting Concerns	35
6.3.1.	JWT-Token	35
6.3.2.	Hashing.....	35
6.3.3.	Monitoring	35
6.3.4.	API-First und Code Generation.....	35
6.4.	Einschränkungen.....	36
7.	VERGLEICH DER IMPLEMENTIERUNGEN.....	37
7.1.	API.....	37
7.2.	Persistenz - Datenbankzugriff	38
7.3.	JWT und rollenbasierte Zugriffskontrolle.....	40
7.4.	Messaging und Kafka Integration	43
7.5.	Native-Build und GraalVM-Kompatibilität.....	45
7.6.	Hashing	46
7.7.	Vergleichsfazit und Learnings.....	47

8. BENCHMARKING DER PROTOTYPEN	48
8.1. Testumgebung	48
8.2. Verwendete Tools	48
8.3. Durchführung der Messungen.....	48
8.3.1. CPU-Effizienz und RAM-Verbrauch	48
8.3.2. Startzeit.....	50
8.3.3. Verarbeitungszeit	50
8.4. Ergebnisse	51
8.4.1. CPU-Effizienz und RAM-Verbrauch	51
8.4.2. Startzeit.....	54
8.4.3. Verarbeitungszeit	55
8.4.4. Vergleich	58
9. DISKUSSION UND AUSBLICK.....	59
9.1. Conclusio	59
9.2. Diskussion	60
9.3. Future Work	62
LITERATURVERZEICHNIS	63
TABELLENVERZEICHNIS	72
CODEAUSSCHNITTVERZEICHNIS	73
ANHANG	74

1. Einleitung

Microservice-Architekturen ermöglichen es Softwaresysteme in kleine, unabhängig einsetzbare Module zu unterteilen. Diese sollen in Koordination miteinander ein fachliches Problem lösen. Die Integration lässt sich in zwei große Varianten unterteilen: orchestrations- und choreografiebasierte Ansätze. Choreografiebasierte Architekturen fügen sich gut in moderne Cloud Umgebungen ein und bieten mehrere Vorteile gegenüber monolithischen Systemen. Es wird auf Eventkommunikation gesetzt, wobei der Prozessfluss nicht durch eine zentrale Instanz gesteuert wird. Das fördert unabhängige Deploybarkeit, lose Kopplung und erhöhte Skalierbarkeit. Solche Systeme sind in der Lage, Lastspitzen besser zu bewältigen und bei Ausfällen eines Services, können andere weiterhin funktionieren. Choreografie unterstützt zudem die agile Entwicklung, da Teams gleichzeitig und unabhängig voneinander an verschiedenen Services arbeiten können. Im Optimalfall entsteht ein Produkt, welches flexibel anpassbar ist und keinen Single-Point-of-Failure aufweist. [1], [2] Java gilt heute noch als einer der am weitesten verbreiteten Programmiersprachen [3] und ist daher auch für Microservice-Architekturen von hoher Relevanz. Die Beliebtheit und vielseitige Tool-Unterstützung machen es zur bevorzugten Wahl vieler Unternehmen. Neuerdings haben sich neben klassischen JVM-basierten Implementierungen alternative Java-Frameworks und Laufzeiten entwickelt. Diese sind speziell auf die Anforderungen von Cloud-basierten Microservices ausgerichtet. Dazu gehören Quarkus, Micronaut und Spring Boot, jedes Framework mit eigenen Ansätzen zur Performanceoptimierung, Ressourcennutzung und Startgeschwindigkeit. [4] Interessant ist hier der Vergleich zwischen JVM-basierten und nativen Implementierungen von Java-Services mit GraalVM. Diese Technologie ermöglicht die AOT (Ahead-Of-Time) Kompilierung von Java-Anwendungen, was zu einer schnelleren Startzeit und einem geringeren Speicherverbrauch führen kann [5]. Dies sind wesentliche Vorteile für verteilte und Cloud-native Architekturen.

1.1. Forschungsziele und Forschungsfragen

Ziel dieser Arbeit ist es, zu untersuchen, welches der drei Frameworks, Quarkus, Micronaut und Spring Boot, in einer choreografiebasierten Microservice-Architektur unter realistischen Bedingungen die besten Ergebnisse hinsichtlich Startzeit, Performance und Ressourcennutzung erzielt. Hierbei sollen insbesondere notwendige Anpassungen und bestehende Einschränkungen bei der Nutzung von GraalVM und AOT-Kompilierung betrachtet werden. Als praxisnahes Beispiel dient eine E-Shop-Anwendung für digitale Güter, anhand welcher die Unterschiede zwischen den Frameworks herausgearbeitet werden. Die Ergebnisse sollen Entwickler:innen und Architekt:innen dabei helfen, fundierte Entscheidungen für die Umsetzung kosteneffizienter und leistungsstarker Cloud- sowie serverloser Lösungen zu treffen. Die Hauptforschungsfrage ist hierbei:

Wie unterscheiden sich Startzeit, Speicherverbrauch, CPU-Effizienz und Verarbeitungszeiten von Quarkus, Micronaut und Spring Boot in einer auf Microservice-Choreografie basierenden E-Shop-Anwendung, sowohl in JVM-Umgebungen als auch als GraalVM-Native-Images?

Dabei werden bei der Erarbeitung der Hauptforschungsfrage folgende Teilfragen untersucht:

1. Welche Vorteile und Herausforderungen bieten choreografiebasierte Microservice-Architekturen im Vergleich zu orchestrierten Architekturen in Bezug auf Skalierbarkeit, Flexibilität und Ausfallsicherheit?
2. Welche Entwurfsmuster, Technologien und Best Practices sind für die Umsetzung choreografiebasierter Microservices in E-Shop-Anwendungen erforderlich und empfehlenswert?
3. Wie lässt sich ein E-Shop-Prototyp für digitale Güter mit choreografiebasierten Microservices in Quarkus, Micronaut und Spring Boot umsetzen, und welche Unterschiede, Einschränkungen und Anpassungen ergeben sich sowohl zwischen den Frameworks selbst als auch bei der nativen Kompilierung mit GraalVM?

1.2. Methodik

Zur Bearbeitung der Forschungsfragen verfolgt diese Arbeit einen mehrstufigen Ansatz. Um Teilfrage 1 zu beantworten wird mit Literaturrecherchen gearbeitet. Zunächst werden theoretische Grundlagen sowie der aktuelle Stand der Technik in Java und relevante Konzepte erörtert. Danach wird ein Vergleich von choreografiebasierten und orchestrierten Architekturen diskutiert. Zur Beantwortung von Teilfrage 2 werden anhand relevanter Literatur wichtige Entwurfsmuster sowie technologische Best Practices für choreografiebasierte Microservice-Architekturen identifiziert.

Anschließend wird ein E-Shop-Prototyp für digitale Güter entwickelt, der als praktische Anwendungsgrundlage dient. Der Prototyp wird in drei getrennten Implementierungen umgesetzt, jeweils eine Variante pro Framework Quarkus, Micronaut und Spring Boot. Diese werden sowohl in der klassischen JVM-Umgebung als auch als GraalVM-native Images bereitgestellt. Der Lösungsentwurf, die Architektur sowie die wichtigsten Erkenntnisse und technischen Besonderheiten aus der Entwicklung werden dargestellt, miteinander verglichen und diskutiert. Dabei soll Teilfrage 3 behandelt werden. Daraufhin erfolgen Benchmarking- und Performance-Messungen der verschiedenen Prototypen. Hierfür wird eine Testumgebung eingerichtet und anschließend mittels geeigneter Monitoring- und Benchmarking-Tools Messungen von Startzeiten, Speicherverbrauch, CPU-Effizienz sowie Verarbeitungszeiten durchgeführt. Die erhobenen Daten werden dann ausgewertet und analysiert. Die Ergebnisse werden dargestellt, um Stärken und Schwächen der einzelnen Frameworks hervorzuheben. Schließlich werden alle Ergebnisse herangezogen um die Hauptforschungsfrage zu beantworten.

2. Related Work

In der Studie von Sihombing und Zarlis (2025) [6] wurden die Performance-Unterschiede zwischen GraalVM und der JVM anhand von Spring-Boot-Microservices untersucht. Der Fokus lag auf mobilen Bankenanwendungen, wobei Metriken wie Startzeit, CPU-Nutzung und Speicherverbrauch unter verschiedenen Lastbedingungen (100, 300 und 600 Nutzer:innen) gemessen wurden. Die Ergebnisse zeigten, dass GraalVM die Startzeiten der Applikationen um durchschnittlich 25-27 s verbessert. Bei der CPU-Auslastung ergaben sich unterschiedliche Resultate. GraalVM zeigte bei geringer Last Vorteile, war jedoch mit der JVM bei mittlerer bis hoher Auslastung effizienter, vor allem bei Services mit CPU-intensiven Aufgaben. Ein Nachteil von GraalVM bestand in einem höheren Speicherverbrauch im Vergleich zur JVM, besonders bei hoher Last, wo GraalVM bis zu dreimal mehr Speicher benötigte. [6]

Im Gegensatz zur Arbeit [6], welche sich ausschließlich auf Spring Boot konzentriert und Performance-Unterschiede zwischen JVM und GraalVM untersucht, erweitert die vorliegende Masterarbeit die Betrachtung auf insgesamt drei unterschiedliche Frameworks. Außerdem berücksichtigt sie neben technischen Aspekten auch die Herausforderungen und notwendigen Anpassungen bei der praktischen Umsetzung einer E-Shop-Anwendung in einer Microservice-Choreografie.

In einer Studie von Wyciślik et al. (2023) [4] wurden die drei Frameworks Spring Boot, Quarkus und Micronaut im Kontext von Microservices untersucht. Die Autoren entwickelten hierfür ein eigenes Testsystem welches vier Einsatzbereiche abdeckt. Einfache REST-Aufrufe, CPU-intensive Sortieralgorithmen, externe HTTP-Kommunikation sowie Datenbankzugriffe mittels CRUD-Operationen (Create, Read, Update, Delete). Getestet wurden Antwortzeiten, Ressourcenverbrauch (CPU/RAM), Start- und Kompilierzeiten sowie Containergrößen. Dies jeweils in klassischer JAR-Form und als GraalVM-Native-Image. Die Ergebnisse zeigen, dass Quarkus in vielen Kategorien, besonders bei Startzeit und Ressourceneffizienz, am besten abgeschnitten hat. Spring Boot war insgesamt stabiler unter hoher Last, zeigte jedoch höhere Startzeiten und größeren Ressourcenbedarf. Micronaut lieferte gute Werte bei Startzeit und Docker-Image-Größe, hatte aber unter hoher Last Stabilitätsprobleme durch fehlgeschlagene Requests. Insgesamt boten Native-Images Vorteile bei Startzeit und Containergröße, benötigten aber mehr Zeit zum Kompilieren und waren weniger fehlertolerant. [4]

Im Gegensatz dazu fokussiert sich die vorliegende Masterarbeit nicht nur auf künstliche Benchmarks, sondern analysiert die Leistungsfähigkeit und technische Umsetzbarkeit einer möglichst realitätsnahen Microservice-Applikation. Der Fokus liegt dabei zusätzlich auf der Betrachtung von Spring Boot, Quarkus und Micronaut im Kontext von Event-getriebener Kommunikation und Choreografie in einem produktionsnahen Szenario.

Šipek et al. (2020) [5] untersuchten die möglichen Leistungsgewinne von Cloud-basierten Java-Anwendungen, ebenfalls durch die Verwendung von GraalVM und Quarkus. Sie verfolgen dabei zwei Ansätze. Einerseits wurden Benchmarks mit der „Renaissance-Suite“ durchgeführt und andererseits evaluierten sie die Performance realer RESTful (Representational State Transfer) APIs (Application Programming Interface) mit CRUD-Operationen. Diese wurden in Spring Boot, Quarkus und Micronaut implementiert, jeweils

unter Verwendung einer MySQL-Datenbank. Die Ergebnisse zeigen, dass Native-Images, die mit GraalVM erzeugt wurden, sowohl beim Speicherverbrauch als auch bei der Reaktionszeit deutlich bessere Werte liefern als die JVM-Varianten. Quarkus als native Anwendung benötigte rund 82 % weniger Speicher als Spring Boot und liefert 81,74 % schnellere Antwortzeiten. Micronaut zeigte sich hierbei ebenfalls effizient. Bei der Ausführung auf OpenJDK 13 benötigte es 22,9 % weniger Speicher als Quarkus im selben Setup und lieferte etwas schnellere Antwortzeiten. Im Vergleich zur nativen Quarkus-Variante schnitt Micronaut schlechter ab. Wenn Native-Images mit GraalVM eingesetzt werden, lieferte Quarkus die besten Gesamtergebnisse und die Studie zeigt damit das Potenzial von GraalVM und Quarkus für CPU- und speichereffiziente Cloud-basierte Microservice-Lösungen. [5]

Im Unterschied zur Arbeit von Šipek et al., die auf Benchmarks mit der „Renaissance-Suite“ und Einzelvergleiche von CRUD-basierten REST-Services setzt, basiert diese Arbeit auf einer vollständigen choreografiebasierten Microservice-Architektur. Statt isolierter Performancetests einzelner Frameworks werden die Technologien im Zusammenspiel mit Apache Kafka evaluiert. Weiters liegt der Schwerpunkt neben nativer Kompilierung mit GraalVM und damit verbundenen Herausforderungen auch auf dem Verhalten und der Interaktion der Frameworks in einem asynchronen, eventgetriebenen Gesamtsystem.

In bestehenden Arbeiten geht es vor allem um einzelne Leistungskennzahlen wie Startzeit oder Ressourcenverbrauch unter sehr kontrollierten Bedingungen. Weniger beleuchtet wird wie die Frameworks Quarkus, Micronaut und Spring Boot in realitätsnahen, ereignisbasierten Geschäftsprozessen abschneiden, vor allem im Zusammenspiel mit Messaging-Plattformen wie Kafka. Die vorliegende Arbeit nähert sich daher diesem Themengebiet mit einem praxisorientierten Prototyp. Sie untersucht dabei sowohl Leistungsmetriken und potenzielle Performancegewinne durch den Einsatz von GraalVM als auch architekturelle Patterns und Best Practices in einem konkreten Anwendungsszenario.

3. Grundlagen und theoretischer Hintergrund

Das nachfolgende Kapitel stellt die theoretischen und technologischen Grundlagen dar. Es umfasst eine Einführung in Monolith- und Microservice-Architekturen, eine Vorstellung der Java-Frameworks Quarkus, Micronaut und Spring Boot sowie eine Betrachtung der Java-Laufzeitumgebungen JVM, GraalVM und Native-Image. Weiterhin wird der Begriff Cloud-native Architektur und dessen Relevanz im gegebenen Kontext beleuchtet.

3.1. Monolith und Microservice-Architekturen

Die Art und Weise, wie Software entwickelt, bereitgestellt und betrieben wird, hat sich in den letzten Jahren immer wieder grundlegend verändert. Während früher oft große, monolithische Anwendungen üblich waren, wird neuerdings mehr auf Microservices und Cloud-native Ansätze gesetzt. [7], [8] Sie helfen dabei, Anwendungen schneller weiterzuentwickeln, Ausfälle besser abzufangen und auf wechselnde Anforderungen zu reagieren. Gleichzeitig bringen diese Ansätze auch technische Herausforderungen mit sich, auch für bewährte Technologien wie Java. Moderne Frameworks wie Spring Boot, Quarkus oder Micronaut und leistungsfähige Laufzeitumgebungen wie GraalVM zeigen, wie Java für die Cloud fähig gemacht, containerisiert und serverless betrieben werden kann. [5], [9], [10] Im Folgenden wird gezeigt warum diese Konzepte im Kontext der Arbeit wichtig sind und wie sie helfen, moderne Software zukunftssicher und leistungsfähig zu machen.

3.1.1. Monolith

Ein Monolith beschreibt eine einzelne, einheitliche Bereitstellungseinheit, die sämtliche Anwendungslogik enthält. Darunter fällt die Benutzeroberfläche, Geschäftsprozesse, der Datenbankzugriff und vieles mehr. Zur internen Strukturierung von Monolithen werden verschiedene Muster angewendet. Dabei hat sich das Schichtenmodell, auch bekannt als N-Tier Architektur, stark etabliert. Abbildung 1 zeigt die Trennung von Komponenten in horizontale Schichten wie Präsentation, Geschäftslogik und Persistenz bzw. Datenbank. Diese Unterteilung kann auch die Strukturen der umsetzenden Organisation oder Teams widerspiegeln. Dieses Phänomen wird von Conways Gesetz näher beschrieben. [7]

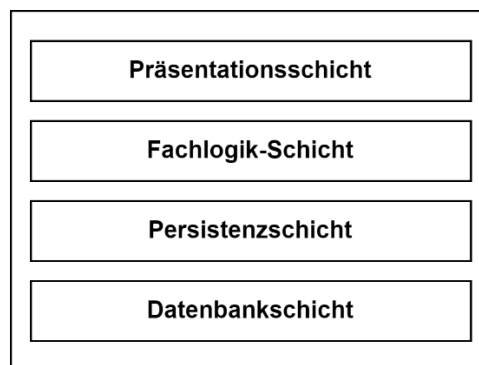


Abb. 1: Schichtenmodell im Monolith [11]

Schichten können geschlossen sein, was bedeutet, dass eine Anfrage immer die vorgesehene Reihenfolge durchläuft. Zum Beispiel kann das von der Benutzeroberfläche über die Geschäftslogik bis zur Datenbank erfolgen. Das stellt sicher, dass jede Schicht klar abgegrenzt ist und Änderungen darin nur geringe Seiteneffekte auf andere haben. Offene

Schichten erlauben jedoch Abkürzungen, wenn bestimmte Komponenten oder Daten in verschiedenen Teilen der Architektur benötigt werden. In solchen Fällen schafft eine zusätzliche Serviceschicht wie in Abbildung 2 eine saubere gemeinsame Komponente, um wiederholte Zugriffe zu sammeln und gleichzeitig die Gesamtlösung übersichtlich zu halten. [11]

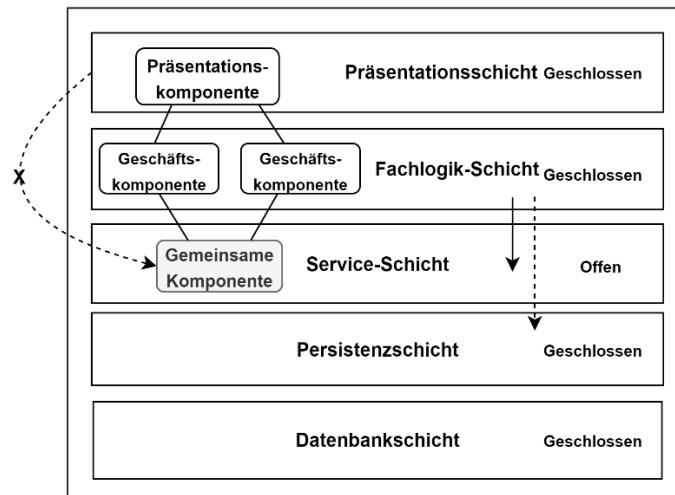


Abb. 2: Offene gemeinsame Komponente im Schichtenmodell [11]

Typischerweise besteht ein Monolith aus einem einzigen Prozess, der bei Bedarf mehrfach instanziiert werden kann. Ein weiterer Ansatz ist der modulare Monolith, welcher die Architektur nach Domänen strukturiert. In einer E-Shop-Anwendung wären das beispielsweise die Bestellung oder das Inventar. Das erlaubt eine parallele Entwicklung und hält Möglichkeiten für spätere Aufteilungen in kleinere Teile offen. Herausfordernd ist dabei oft die zentrale Datenbank, welche selten die Modulstruktur des Monolithen abbildet. [7], [11] Insgesamt sind Monolithen gerade zu Beginn leichter zu entwickeln, zu testen und bereitzustellen. ACID-Transaktionen (Atomicity, Consistency, Isolation, and Durability) sind leichter umsetzbar, da meist eine einzelne Datenbank genutzt wird. Mit wachsender Größe eines Monolithen nehmen jedoch interne Abhängigkeiten zu. Des Weiteren kann eine große Codebasis unübersichtlich werden, was Änderungen und Wartung erschwert. Jede Anpassung benötigt oft ein vollständiges neues Deployment des Monolithen. Dadurch wird Skalierung ebenfalls problematisch, da die gesamte Anwendung mitwachsen muss. Hierbei lässt sich die Leistung nicht gezielt nur an der Stelle verbessern, wo es nötig wäre. Ein weiterer Nachteil ist, dass die anfängliche Technologieauswahl spätere Innovationen behindern kann. Letztendlich leidet die Ausfallsicherheit, denn wenn eine Komponente versagt, ist der gesamte Monolith beeinträchtigt. [7], [11], [12]

3.1.2. Microservice Ansatz

Microservices sind ein Architekturkonzept, bei dem eine Anwendung in viele kleine, selbstständige Services unterteilt wird, die über definierte Schnittstellen miteinander kommunizieren. Jeder dieser Services ist so konzipiert, dass er nur eine abgegrenzte fachliche Aufgabe erfüllt. [12] Diese Abgrenzung erfolgt dabei häufig entlang fachlicher Domänen, was eng an Ansätze wie dem Domain-Driven-Design angelehnt ist. [7], [13]

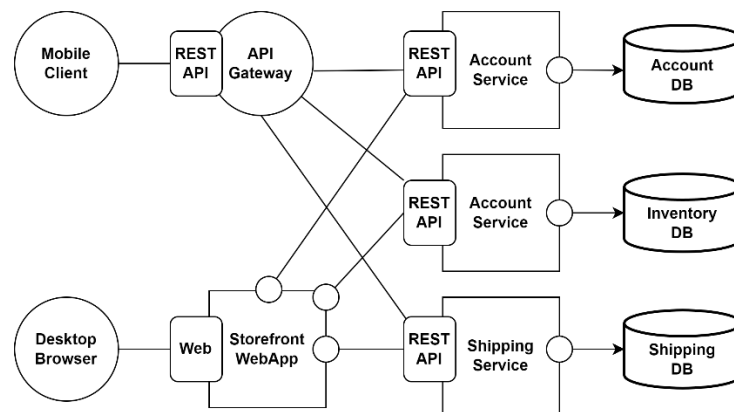


Abb. 3: Microservice-Architektur Beispiel [14]

Ein einfaches Beispiel aus [14] in Abbildung 3, wäre eine fiktive E-Commerce-Anwendung, welche folgendermaßen unterteilt ist: Ein „Account“-Service prüft Kundendaten, ein „Inventory“-Service verwaltet den Lagerbestand, und ein „Shipping“-Service organisiert den Versand. Jeder Service hat seine eigene Datenbank und ist über ein API-Gateway erreichbar. So können einzelne Teile unabhängig entwickelt, betrieben und skaliert werden. [14]

Microservices entstanden als Antwort auf typische Schwächen monolithischer Systeme. [15] Wesentlicher Faktor sind dabei die starken internen Abhängigkeiten, z.B. wenn verschiedene Programmteile direkt miteinander verknüpft sind und Änderungen oft unerwartete Folgen in anderen Teilen haben. Microservices versuchen Defizite des Monolithen zu adressieren, indem sie beispielweise jeden Service als eigenständigen Prozess mit eigener Datenhaltung umsetzen. [7], [15] Ein wichtiger Punkt ist dabei die Unabhängigkeit der Deployments. Änderungen an einem Service lassen sich einsetzen, ohne andere zu unterbrechen. [7] Diese Eigenschaft wird durch Container-Technologien wie Docker und Kubernetes zusätzlich unterstützt, welche eine isolierte und standardisierte Bereitstellung ermöglichen. [13]

Weitere Vorteile von Microservices finden sich in mehreren Bereichen. Sie erlauben zum Beispiel technologische Heterogenität. Das bedeutet, dass in einem Softwareunternehmen jedes Team für seinen Dienst die am besten geeignete Programmiersprache oder Datenbanktechnologie wählen kann. Zudem lassen sich einzelne Services gezielt skalieren, zum Beispiel bei höherem Ressourcenbedarf durch vermehrte Anfragen [12], [15]. Sie können auch einen organisatorischen Einfluss haben, indem sie kleine, funktionsübergreifende Teams fördern in denen Entwickler:innen, Tester:innen und Betriebsexpert:innen zusammenarbeiten. Damit tragen sie die volle Verantwortung für sowohl Entwicklung als auch Betrieb eines einzelnen Services nach dem Motto „You build it, you run it“ [7], [15].

In Cloud-Umgebungen lassen sich diese Vorteile ebenfalls gut nutzen. Da Microservices häufig containerisiert bereitgestellt werden, lassen sie sich flexibel über mehrere Server hinweg verteilen. So können bei Bedarf schnell zusätzliche Instanzen gestartet werden, ohne den Rest des Systems zu beeinträchtigen. [13]

Neben den Vorteilen bringen Microservices aber auch Schwierigkeiten mit sich. Durch ihre verteilte Natur, entstehen zusätzliche Verzögerungen. Eine schnelle und zuverlässige Netzwerkkommunikation sowie eine stabile Infrastruktur sind erforderlich [12], [15]. Es spielen auch Sicherheitsmaßnahmen eine wichtige Rolle, denn jede Anfrage geht über das Netzwerk und dieses sollte abgesichert und überwacht werden [15]. Außerdem kann es problematisch werden, wenn Services zu klein geschnitten sind und die Gesamtlösung aus zu vielen kleinen Teilen besteht. Es erhöht den Aufwand, alle Services effektiv miteinander zu integrieren und zuverlässig zu steuern. Das kann die Übersichtlichkeit beeinträchtigen und im schlimmsten Fall die Stabilität des gesamten Systems negativ beeinflussen. [15]

Beim Zusammenspiel der Services gibt es verschiedene Ansätze. Während Orchestrierung bedeutet, dass eine zentrale Instanz den Ablauf steuert und festlegt, wann welcher Service aktiv wird, setzen Microservice-Architekturen manchmal auch auf eine dezentrale, ereignisgesteuerte Choreografie. Dabei reagieren die Services selbstständig auf Ereignisse, tauschen Informationen direkt aus und bleiben trotzdem unabhängig voneinander. [15] Beim Übergang von Monolithen zu Microservices sind bewährte Methoden wie das Strangler-Fig-Pattern eine wichtige Stütze. Neue Funktionen werden dabei Schritt für Schritt als eigene Microservices umgesetzt, während der alte Monolith nach und nach zurückgebaut wird, ohne dass alles auf einmal ersetzt werden muss. [7]

Zusammenfassend stellen Microservices eine Fortführung klassischer Architekturansätze dar, mit Anforderungen an Organisation, Technologien und Betrieb. Auf weiterführende Details und die Evaluierung von Konzepten wie Choreografie, Orchestrierung oder wichtige Patterns wird in späteren Abschnitten dieser Arbeit vertiefend eingegangen.

3.2. Java Frameworks

Im Java-Umfeld werden spezielle Frameworks eingesetzt, welche Entwickler:innen dabei unterstützen, verteilte Systeme sauber und effizient umzusetzen. In diesem Abschnitt werden mit Spring Boot, Quarkus und Micronaut drei wichtige Java-Frameworks untersucht, welche auf Anforderungen moderner verteilter Systeme ausgerichtet sind. Ein Software-Framework stellt ein wiederverwendbares Grundgerüst für Anwendungen dar, welches Entwickler:innen bei Bedarf anpassen können. Es ermöglicht, sich auf Geschäftslogik zu konzentrieren und kann dabei die Produktivität, Qualität und Leistung verbessern. Allerdings bringen Frameworks auch eine Lernkurve mit sich und können ggf. den Software-Footprint vergrößern. [16]

Das Java-Ökosystem, als eine der beliebtesten und etabliertesten Plattformen für Unternehmenssoftware, bietet verschiedene Frameworks für gängige Anforderungen wie Datenzugriff, Sicherheit und Kommunikation. Für skalierbare Microservice-Anwendungen haben Frameworks wie Spring Boot bereits früh eine wesentliche Rolle gespielt. Neuere Frameworks wie Micronaut und Quarkus wurden gezielt entwickelt, um Java moderner und ressourcenschonender zu machen. [4]

Die Popularität einer Technologie kann man sowohl an der Community-Unterstützung als auch an ihren Weiterentwicklungschancen erkennen. Ein Indikator sind GitHub-Sterne: Spring Boot liegt mit etwa 77.800 [17] deutlich vorne, gefolgt von Quarkus mit rund 14.700 [18] und Micronaut mit 6.300 [19]. Eine Umfrage des JAXenter-Portals aus 2020 welche in [20] behandelt wird, zeigte ein ähnliches Bild: Spring Boot belegte den Spitzenplatz,

Quarkus erreichte Platz Drei. 22 % der Befragten bewerteten Quarkus als sehr interessant, weitere 25 % als interessant. Micronaut lag dabei im Mittelfeld. [21]

3.2.1. Quarkus

Quarkus, entwickelt von Red Hat und erstmals 2018 veröffentlicht, wurde speziell für die Integration mit GraalVM konzipiert. Es läuft zudem auch auf der Standard-JVM und unterstützt neben Java auch die Programmiersprache Kotlin. Das Framework folgt einem Container-First-Ansatz und versucht Speichernutzung, Startzeit und Docker-Image-Größe zu optimieren. Es soll damit für Serverless-, Cloud- und Kubernetes-Umgebungen attraktiver gemacht werden. [21]

Um native und containerisierte Umgebungen effizient nutzen zu können, stellt Quarkus für die Cloud optimierte Java-Bibliotheken bereit und vereinfacht die Erstellung nativer Images mit GraalVM. Der Entwicklungsmodus bietet automatisches Live-Reload und Änderungserkennung, was den Workflow insgesamt verbessert. Quarkus unterstützt asynchrone Programmierung und bietet fortgeschrittene Sicherheitskonzepte durch eine große Auswahl an Plug-Ins. Authentifizierung und Benutzerverwaltung lassen sich auch an externe Anbieter delegieren. Es ist mit bewährten Technologien wie Hibernate ORM (Object Relational Mapping), Eclipse sowie Vert.x kompatibel und hat Netty mit RestEasy im Einsatz. Außerdem bietet es ein standardisiertes Konfigurationssystem. Insgesamt gilt Quarkus als leistungsfähige Plattform für Cloud-native und serverlose Java-Anwendungen. [5]

3.2.2. Spring Boot

Spring Boot ist ein Framework, welches den Entwicklungsprozess vereinfacht, indem es den Konfigurationsaufwand und Boilerplate-Code reduziert. Es basiert auf dem Spring Framework und bietet automatische Konfigurationen, eingebettete Web-Server und eine Reihe von „Starter“-Paketen. Diese ermöglichen eine schnelle Inbetriebnahme, ohne dass eine komplexe Infrastruktur, wie bei herkömmlichen Java EE-Servern, manuell eingerichtet werden muss. Spring Boot integriert sich mit anderen Spring-Projekten wie Spring Data, Spring Security, Spring Cloud, verschiedenen Datenquellen und Messaging-Systemen. [22] Eingebettete Server wie Tomcat und Jetty erreichen zudem eine schnelle und einfache Bereitstellung, vor allem in Container-Umgebungen. Insgesamt vereinfacht Spring Boot die Umsetzung skalierbarer, Cloud-nativer Microservices-Architekturen. Dessen Ansätze machen es zu einer beliebten Wahl für die Entwicklung robuster, leicht zu wartender und produktionsreifer Software. Das gilt insbesondere in Umgebungen, die hohe Skalierbarkeit und schnelle Bereitstellungszyklen erfordern. [22], [23], [24], [25]

3.2.3. Micronaut

Micronaut ist ein JVM-basiertes Framework, welches 2018 von den Entwickler:innen von Grails vorgestellt wurde. Es wurde für modulare, leicht testbare Microservices mit Java, Kotlin oder Groovy konzipiert. Im Gegensatz zu Spring Boot nutzt Micronaut Annotationsverarbeitung zur Kompilierzeit statt auf Laufzeitreflexion und Classpath-Scanning zu setzen. Dieser Ansatz ermöglicht im Vergleich deutlich schnellere Startzeiten. [21]

Micronaut ist ebenfalls speziell für Cloud- und serverlose Umgebungen optimiert und bietet Integrationen für Plattformen wie GCP (Google Cloud Platform), AWS (Amazon Web Services) und Kubernetes. Dank der Dependency-Injection zur Kompilierzeit eignet es sich auch gut für Native-Images mit GraalVM, da Reflection weitestgehend vermieden wird. Dadurch können Startzeiten und Speicherbedarf weiter reduziert werden. Es unterstützt standardmäßig reaktive Programmierung und ermöglicht die Erstellung kompakter JARs (Java Archive) und kleiner Container-Images ohne eingebettete JVM. [26]

Micronaut ist in gut abgegrenzte Module gegliedert, wobei jedes davon bei Bedarf zur Buildzeit eingebunden wird und die Laufzeitartefakte fallen stets relativ kompakt aus. Insgesamt zeigt Micronaut schnelle Startzeiten und geringen Ressourcenverbrauch, kann aber bei der ersten Anfrage nach Inbetriebnahme zu leichten Verzögerungen führen. Benchmarks zeigen solide Ergebnisse, besonders beim Einsatz von nativen Images, stößt aber bei hoher Last auf seine Grenzen. Die Dokumentation ist im Vergleich zu anderen etablierten Frameworks noch im Aufbau. [4], [26], [27]

3.3. Java Laufzeitumgebungen

Im nächsten Abschnitt wird betrachtet, wie klassische JVMs funktionieren, wo ihre Grenzen liegen und wie moderne Alternativen wie GraalVM helfen, Java für Microservices und Cloud-native Szenarien besser nutzbar zu machen.

3.3.1. JVM

Virtuelle Maschinen (VMs) wie die JVM oder die Common Language Runtime (CLR) von .NET sind ein wichtiger Pfeiler der Softwareentwicklung geworden. Sie bieten plattformunabhängige Ausführungsumgebungen, wobei der Hauptvorteil in der Abstraktion von Hardware- und Betriebssystemdetails liegt. Dadurch wird der Ansatz „Write Once, Run Anywhere“ ermöglicht. Wichtiger Bestandteil ist der Garbage-Collection-Mechanismus, welcher die Speicherverwaltung automatisiert übernimmt [28]. Die Java HotSpot VM ist eine der verbreitetsten JVM-Implementierungen und verfolgt das Ziel, Java-Anwendungen so performant wie nativen Code auszuführen. Dies erfolgt über Just-in-Time (JIT)-Kompilierung, die häufig genutzte Codeabschnitte (Hotspots) zur Laufzeit in optimierten Maschinencode übersetzt. [28] Trotz dieser Optimierungen bringt die JVM auch Nachteile mit sich. Dynamische Mechanismen, wie das Laden von Klassen zur Laufzeit und die JIT-Kompilierung insgesamt, verursachen erhöhte Startkosten, vor allem durch Schritte wie die Bytecode-Verifizierung, Klasseninitialisierung, Profiling und dynamische Kompilierung [29], [30]. Es gibt eine Aufwärmphase bei JVM-Lösungen, welche zu höherem Speicherbedarf führt. Vor allem in Cloud-Umgebungen mit häufig startenden Microservices oder Serverless-Funktionen wirken sich erhöhte Startzeiten und Ressourcenbedarf negativ auf Skalierbarkeit und die Gesamtkosten aus. [5], [29]

3.3.2. GraalVM und Native Image

GraalVM wurde entwickelt, um die Schwächen herkömmlicher JVMs zu überwinden, und basiert auf dem OpenJDK [28]. Dabei nutzt sie wichtige Bausteine der Java HotSpot VM, wie z.B. den Interpreter, den Klassenloader und den Garbage Collector [5]. Neu ist vor allem das JVM Compiler Interface (JVMCI). Dieses macht es möglich, alternative Compiler wie den Graal Compiler, welcher selbst in Java geschrieben ist, einzubinden [28]. Der Graal Compiler setzt auf modernere Optimierungstechniken wie Methoden-Inlining oder Partial-
Esad Hrbatović

Escape-Analysen, um Speicher effizienter zu nutzen. Der Eingangscode, zum Beispiel Java-Bytecode oder Code aus anderen Sprachen, wird mit Hilfe von Truffle in eine spezielle Graal Intermediate Representation (IR) übersetzt. Auf dieser Zwischenstufe werden verschiedene Optimierungen vorgenommen, bevor der Code in Maschinencode übersetzt wird [28]. Mit Truffle lassen sich auch Interpreter für verschiedene Sprachen erstellen und GraalVM kann diese dank partieller Auswertung automatisch in schnellen Maschinencode umwandeln [28]. Es lassen sich Programme aus unterschiedlichen Sprachen wie JavaScript, Python, Ruby, R oder sogar C/C++ in einer einzigen VM gemeinsam ausführen. [5]

Neben der klassischen JIT-Kompilierung bietet GraalVM mit Native-Image auch einen ganz anderen Ansatz: die AOT mit der Substrate VM (SVM) [28], [30]. Dabei wird der gesamte Code bereits beim Build analysiert und in Maschinencode umgewandelt. Wichtig ist dabei die sogenannte Closed-World-Annahme. Alle Klassen und Abhängigkeiten müssen vorher bekannt sein [29]. Während des Build-Prozesses wird Initialisierungscode ausgeführt und der Zustand der Objekte eingefroren, was als Heap-Snapshot in das fertige Image übernommen wird [5], [29]. Dadurch entfallen typische JVM-Startaufgaben wie dynamisches Laden, Überprüfen und Kompilieren. Das Ergebnis ist eine eigenständige, plattformspezifische ausführbare Datei, die extrem schnell startet und mit den Copy-on-Write-Techniken zusätzlich Speicher spart [29]. Die Substrate VM ist ebenfalls in Java geschrieben und beinhaltet dabei alles, was zur Laufzeit nötig ist, darunter auch die Garbage Collection [30]. Mit diesen Eigenschaften werden GraalVM Native-Images für Cloud-native Szenarien, Container-Deployments und Serverless-Anwendungen immer interessanter [5], [28], [29]. Studien zeigen, dass GraalVM im Vergleich zu klassischen JVMs, je nach Anwendungsfall, wesentliche Vorteile bei Geschwindigkeit und Energieverbrauch bieten kann. [30]

3.4. Cloud-native und Serverless

Die Popularität von Cloud-Computing-Anbietern wie AWS, Azure und GCP hat die Art der Softwarebereitstellung grundlegend verändert. Allerdings bringt allein die Ausführung in der Cloud nicht automatisch nur Vorteile mit sich. Es ist wichtig zu unterscheiden, ob eine Anwendung nur in der Cloud läuft oder auch Cloud-nativ entwickelt wurde. Davis (2019) [8] definiert Cloud-native als eine Softwarearchitektur, die von Grund auf für verteilte Systeme entworfen ist. Sie passt sich an ein dynamisches Umfeld an und wird kontinuierlich weiterentwickelt. Dieser Ansatz geht über die reine Nutzung von Cloud-Infrastruktur hinaus. Cloud-native-Systeme bestehen aus modularen und lose-gekoppelten Komponenten, welche flexibel auf mehreren Servern, Rechenzentren oder sogar bei verschiedenen Cloud-Anbietern gleichzeitig ausgeführt und betrieben werden können. Damit soll auch eine hohe Verfügbarkeit gewährleistet werden. Der Betrieb in einer sich ständig verändernden Umgebung wird hier als Normalfall angenommen. Ressourcen können sich jederzeit ändern, Instanzen werden automatisch skaliert oder neu gestartet, und Systemänderungen erfolgen stetig und möglichst ohne Ausfallzeiten. Diese Prinzipien bilden die Grundlage für bewährte Praktiken wie Microservice-Architekturen, Continuous Delivery und automatische Skalierung. [8]

Wesentlich ist dabei der Einsatz von Containern und Orchestrierungsplattformen, denn Containertechnologien wie Docker kapseln Anwendungen gemeinsam mit ihren

Abhängigkeiten. Das fördert eine konsistente Ausführung in unterschiedlichen Umgebungen. Kubernetes übernimmt als Orchestrierungsplattform das automatisierte Deployment, die Skalierung und Verwaltung von Containern und vereinfacht den Betrieb verteilter Systeme maßgeblich [31]. Ebenso spielen Continuous-Integration- und Continuous-Deployment-(CI/CD) -Pipelines eine wichtige Rolle in Cloud-nativen Anwendungen, weil sie den gesamten Prozess vom Bauen über das Testen bis zur Auslieferung der Software automatisieren und beschleunigen. [31]

Cloud-native-Anwendungen sind für den globalen Einsatz konzipiert. Daten und deren Dienste werden weltweit repliziert, um möglichst niedrige Latenzen sicherzustellen. Um tausende gleichzeitige Nutzer:innen bedienen zu können, sollte ein Cloud-native-System grundsätzlich hoch skalierbar sein. Die Annahme, dass die Infrastruktur fehleranfällig ist, sollte jederzeit gelten und von Ausfällen stets ausgegangen werden. Ebenso müssen Upgrades, Tests und Deployments im laufenden Betrieb möglich sein, um eine unterbrechungsfreie Laufzeit zu garantieren. [8]

Containerisierte Microservices galten lange Zeit als der Standard für Cloud-native-Anwendungen, jedoch erweitern aktuelle Entwicklungen diesen Trend zunehmend um Serverless Computing und vollständig verwaltete Cloud-Dienste. Serverless Computing ist ein Entwicklungsmodell, bei dem es möglich ist Anwendungen zu erstellen und auszuführen, ohne sich um die zugrunde liegende Serverinfrastruktur kümmern zu müssen. In diesem Ansatz übernimmt der Cloud-Anbieter die Bereitstellung, Wartung und Skalierung der Server, sodass man sich ausschließlich auf das Schreiben und Bereitstellen von Code konzentrieren kann. Das reduziert Betriebskosten und serverseitige Verwaltungsaufgaben fallen weitestgehend weg. Immer mehr Anwendung findet diese Technologie auch im Bereich des Internet of Things (IoT). Serverless Computing bringt auch Herausforderungen mit sich, zum Beispiel eingeschränkte Kontrollmöglichkeiten über die Umgebung, von Anbietern vorgegebene Ressourcenbeschränkungen und erhöhte Komplexität bei der Fehlersuche und Verwaltung verteilter Funktionen. [9], [32], [33]

Schlussendlich bezeichnet Cloud-native die Entwicklung und Ausführung von Anwendungen, welche die Vorteile des Cloud Computing vollständig ausschöpfen. In dieser Umgebung müssen Anwendungen schnell starten, minimale Ressourcen verbrauchen und gut skalieren können. Solche Anforderungen können herkömmliche Java-Anwendungen mit langsamen Startzeiten und hohem Speicherbedarf bisher nur schwer erfüllen. Auch deshalb versucht GraalVM bei diesen Defiziten anzusetzen, vor allem mit der AOT-Kompilierung in Native-Images. Dadurch ist die Technologie besonders relevant bei Serverless Computing und Cloud-nativen Microservice-Architekturen, wo erhöhter Ressourcenverbrauch sich direkt auf das Benutzererlebnis und die Betriebskosten auswirken. [5], [29], [34], [10]

4. Orchestrierung und Choreografie

Dieses Kapitel untersucht, welche Vor- und Nachteile choreografiebasierte Microservice-Architekturen im Vergleich zu orchestrierten und monolithischen Ansätzen bieten. Als Basis dient dabei das Buch „Microservice Patterns“ von Chris Richardson [35], das zusammen mit weiterer relevanter Fachliteratur herangezogen wurde, um die wichtigsten Konzepte darzustellen und die jeweiligen Stärken und Schwächen zu beleuchten. Der Fokus liegt auf den Aspekten der Skalierbarkeit, Flexibilität und Ausfallsicherheit, um herauszuarbeiten, welche Architekturformen welche Vorteile bieten und welche Herausforderungen sich daraus für Entwicklung und Betrieb ergeben.

4.1. Von zentralen zu verteilten Transaktionen

Monolithen und Microservice-Architekturen haben unterschiedliche Vorgehensweisen, um Daten zu speichern und Transaktionen umzusetzen. Während in monolithischen Systemen meist eine zentrale Datenbank genutzt wird und ACID-Transaktionen die Datenintegrität sichern, setzen Microservices auf eine dezentrale Datenhaltung wo jeder Service seine eigene Datenbank verwaltet. Das kann zu Schwierigkeiten führen, wenn Geschäftsprozesse Daten über mehrere Services hinweg verwalten müssen. Solche Abläufe bestehen oft aus einer Kette von Schritten in verschiedenen Services und das klassische ACID-Modell kann dabei auf seine Grenzen stoßen. [35]

4.1.1. Verteilte Transaktionen - Two-Phase-Commit

Um komplexe, serviceübergreifende Abläufe dennoch konsistent zu halten, wurden in der Vergangenheit spezielle Verfahren entwickelt. Ein bekanntes Beispiel dafür sind Protokolle wie das 2PC (Two-Phase-Commit). Dabei koordiniert ein zentraler Manager in zwei Phasen den Ablauf: Zuerst prüfen alle Teilnehmer, ob sie Änderungen ausführen können (Prepare), danach folgt der endgültige Commit oder ein Rollback. [35], [36]

In folgendem beispielhaften 2PC-Szenario von Abbildung 4 kauft ein:e Kund:in über einen Online-Shop ein. Dabei steuert ein Koordinator den gesamten Ablauf. Zuerst startet die Prepare-Phase: Der Koordinator fragt den Lager-Service, ob die Ware verfügbar ist und den Zahlungs-Service, ob die Zahlung möglich ist. Beide prüfen das intern und melden zurück, ob sie bereit sind („Ready“) oder die Transaktion abgelehnt wird („Abort“).

Antworten alle mit „Ready“, startet die Commit-Phase. Der Koordinator beauftragt Lager und Zahlung an, die Reservierung und Zahlung tatsächlich auszuführen. Danach bekommt der Kunde oder die Kundin eine Bestätigung. Wenn ein Dienst abbricht, wird die ganze Bestellung storniert, damit keine Teilleistungen entstehen. So wird sichergestellt, dass entweder alles oder nichts passiert.

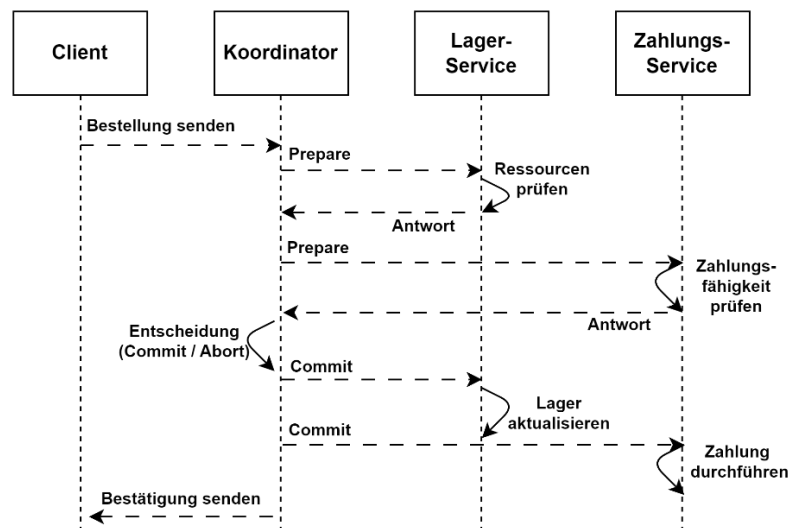


Abb. 4: Two-Phase-Commit Ablauf

In der Praxis senken solche verteilten Transaktionen jedoch die Verfügbarkeit, weil alle beteiligten Dienste gleichzeitig erreichbar sein müssen. Schon bei wenigen Services sinkt die Gesamtverfügbarkeit messbar. [35] Während 2PC Atomarität und Konsistenz garantiert, ist es grundsätzlich ein blockierendes Protokoll. Wenn der Koordinator in einem kritischen Moment abstürzt, können die beteiligten Systeme gezwungen sein, unbestimmt lange auf eine endgültige Entscheidung zu warten. [36] Nach dem CAP-Theorem ist häufig eine Entscheidung zwischen Konsistenz und Verfügbarkeit erforderlich, wobei heute meist Verfügbarkeit bevorzugt wird. Um Datenkonsistenz dennoch sicherzustellen, ohne die Nachteile verteilter Transaktionen, braucht es andere Ansätze. Genau hier setzt das Saga-Pattern an. [35]

4.1.2. Saga Pattern

In Microservice-Architekturen hat sich das Saga Pattern als Standard etabliert, um Konsistenz über mehrere Services hinweg sicherzustellen. Ein großer Geschäftsprozess wird dabei in eine Abfolge lokaler Transaktionen unterteilt, die jeweils innerhalb eines einzelnen Services ablaufen. Die Koordination kann asynchron über Nachrichten und Events erfolgen. Der erfolgreiche Abschluss einer Transaktion stößt die nächste an. Durch diesen Ansatz werden Ausfälle besser abgefangen, da Event-Broker-Nachrichten über einen längeren Zeitraum zwischenspeichern können, bis ein Service wieder verfügbar ist. [35]

Rocha et al. (2023) [37] zeigen in einer Analyse von 125 Praxisquellen, dass neben dem Proxy Pattern das Saga Pattern eines der am häufigsten eingesetzten Muster ist, um Kommunikation und Datenkoordination in Microservices zu organisieren. [37]

Sagas sichern in Microservice-Architekturen Atomarität, Konsistenz und Dauerhaftigkeit, verzichten aber auf Isolation. Das bedeutet, dass Änderungen einer lokalen Transaktion sichtbar werden können, noch bevor die gesamte Saga abgeschlossen ist. Dadurch besteht das Risiko, dass andere Transaktionen auf einen unvollständigen Zwischenzustand zugreifen. Deshalb sollten daraus resultierende Nebeneffekte schon im Design

berücksichtigt werden. [12], [35] Das folgende Beispiel in Abbildung 5 aus [35] demonstriert eine mögliche Saga wie sie in einem fiktiven Restaurant-System eingesetzt werden könnte:

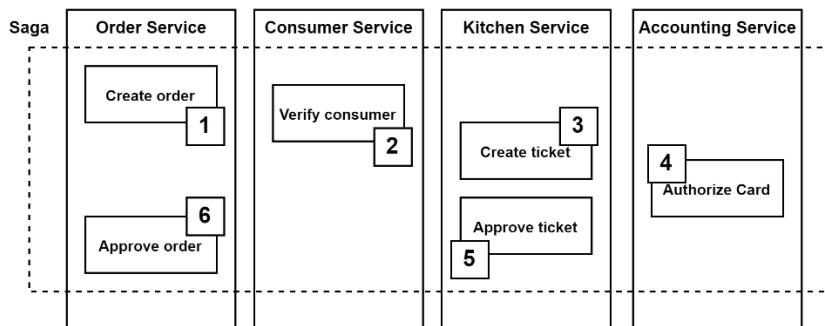


Abb. 5: Saga Aufbau - Beispiel anhand eines Restaurants [35]

1. Der Order Service legt zuerst die Bestellung an (Status: „APPROVAL_PENDING“).
2. Der Consumer Service prüft, ob eine Bestellung zulässig ist.
3. Der Kitchen Service erstellt ein Ticket („CREATE_PENDING“) für die Zubereitung.
4. Der Accounting Service autorisiert die Kreditkarte.
5. Danach ändert der Kitchen Service das Ticket auf „AWAITING_ACCEPTANCE“.
6. Am Ende setzt der Order Service die Bestellung auf „APPROVED“.

Die Services tauschen hierbei asynchrone Nachrichten aus um eine lose Kopplung zu gewährleisten. Es kann komplexer werden, wenn es darum geht Fehlerzustände zu behandeln. In diesem Fall müssen abgeschlossene Schritte mit Kompensationsaktionen wieder rückgängig gemacht werden. Wenn zum Beispiel ein Schritt fehlschlägt, machen kompensierende Operationen vorherige erfolgreiche Schritte in umgekehrter Reihenfolge rückgängig. So kann die Gesamtkonsistenz erhalten bleiben, wenn Reservierungen storniert oder Aufträge unerwartet zurückgesetzt werden müssen. Beispielsweise könnte sich die oben angeführte Saga im Fehlerfall folgendermaßen abspielen: Zunächst wird die Bestellung erstellt, die Kundendaten geprüft und ein Küchenticket angelegt. Schlägt dann die Autorisierung der Kreditkarte fehl, werden folgende Kompensations-Transaktionen durchgeführt: Der Kitchen Service storniert das Ticket wieder, und der Order Service markiert die Bestellung als abgelehnt. Im Idealfall bleiben trotz des Fehlers keine inkonsistenten Zustände bestehen. [35]

Speth et. al (2022) [38] präsentierten eine praxisnahe Referenzarchitektur, das „T2-Projekt“. Diese zeigt, wie sich das Saga-Muster in ein modernes, selbstadaptives Microservice-System integrieren lässt. Zudem wird untermauert, dass die Orchestrierung von Sagas, kombiniert mit asynchronem Messaging (Kafka) und einer modularen Architektur (Spring Boot + „Eventuate Tram“), die zuverlässige Abwicklung komplexer Geschäftsprozesse wie verteilter Bestellungen und Zahlungen ermöglicht. Das System kann Fehler simulieren und verfolgen, deren Auswirkungen auf Service-Level-Objectives (SLOs) messen und mit Autoscaling reagieren. [38]

4.2. Orchestrierung vs. Choreografie

Da nun der Stellenwert vom Saga Pattern zum Erreichen von Konsistenz über Servicegrenzen hinweg behandelt wurde, stellt sich die Frage, wie die Abfolge der Teilschritte umgesetzt werden kann. Dazu braucht es eine Verwaltungslogik oder einen Mechanismus, welcher die einzelnen lokalen Transaktionen startet, abschließt und bei Fehlern über Kompensationen wieder rückgängig macht. Grundsätzlich gibt es dafür zwei Ansätze: Choreografie setzt auf eine dezentrale Steuerung durch den Austausch von Eventnachrichten. Dabei reagiert jeder Service auf empfangene Events und löst so den nächsten Schritt aus. Orchestrierung dagegen nutzt eine zentrale Komponente, die den gesamten Ablauf kontrolliert und die Services gezielt aufruft. [35], [39] Im nächsten Abschnitt werden diese beiden Vorgehensweisen näher beleuchtet und anschließend verglichen.

4.2.1. Orchestrierung – Zentrale Steuerung durch einen Orchestrator

Bei der Orchestrierung übernimmt ein zentraler Saga-Orchestrator die komplette Steuerung, Fehlerbehandlung und den Ablauf des verteilten Geschäftsprozesses. Dieser kennt alle beteiligten Services und schickt gezielt Befehle, um lokale Transaktionen oder Kompensationen auszuführen. Die Kommunikation erfolgt im Request/Response-Muster. Der Orchestrator wartet dabei auf Rückmeldungen, bevor er den nächsten Schritt auslöst. [12], [35]

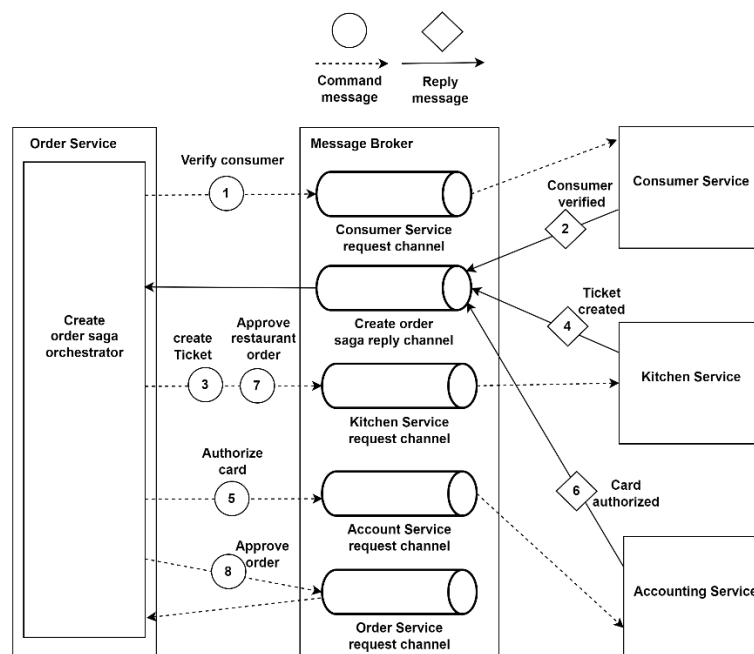


Abb. 6: Microservice Orchestrierung [35]

Am Beispiel des Restaurantprozesses aus [35] in Abbildung 7 bedeutet das: Der Order Service agiert hier als Orchestrator und beauftragt zunächst den Consumer Service, den Kunden und seine Anfrage zu prüfen. Ist das erfolgreich, wird der Kitchen Service angewiesen, ein Ticket zu erstellen. Danach folgt die Zahlungsautorisierung durch den Accounting Service. Kommt es zu einem Fehler, stößt der Orchestrator gezielt

Kompensationen an, z. B. das Stornieren des Tickets oder die Freigabe des Kreditrahmens. [35]

Ein zentraler Orchestrator macht den Ablauf einer Saga besser nachvollziehbar, da alle Schritte, Kompensationen und Fehlerbehandlungen an einer Stelle definiert sind. Das kann das Verständnis und die Wartung erleichtern. Die klare Trennung der Zuständigkeiten sorgt dafür, dass Fachlogik in den Services bleibt, während der Orchestrator nur die Abfolge steuert. Ein Nachteil ist der mögliche Single-Point-Of-Failure. Fällt der Orchestrator aus, kann der gesamte Prozess zum Stillstand kommen. [1] Redundante Instanzen des Orchestrators können helfen den Ausfall einer anderen Instanz zu kompensieren, kann aber kostenintensiv im Betrieb werden. Zudem besteht die Gefahr der Überladung eines Orchestrators, wenn zu viel Logik darin implementiert ist. Um dem entgegenzuwirken, sollte der Orchestrator nur koordinieren und die eigentliche Fachlogik immer in den Services bleiben. Insgesamt ist die Orchestrierung für komplexe Prozesse mit vielen Abhängigkeiten gut geeignet. Baklouti et al. (2017) [40] zeigen, dass Orchestrierung zwar zuverlässig funktioniert, aber viel Kommunikation erfordert, weil alle Zwischenergebnisse immer wieder an die zentrale Stelle zurückgeschickt werden müssen. In Netzwerken, die nicht immer stabile Verbindungen haben, kann das zu längeren Ausführungszeiten und mehr Verzögerungen führen. [40]

4.2.2. Choreografie – Koordination durch Events

Die Idee hinter der Choreografie ist, eine zentrale Steuerung vollständig zu vermeiden. Stattdessen reagieren die Services selbstständig auf Events, welche von anderen Services erzeugt werden können. Dabei muss jeder beteiligte Service lediglich wissen, auf welche Events er hören muss und welche Aktion daraus zu folgen hat. Typisch ist dabei das Publish/Subscribe-Muster über einen Message Broker. [35], [40]

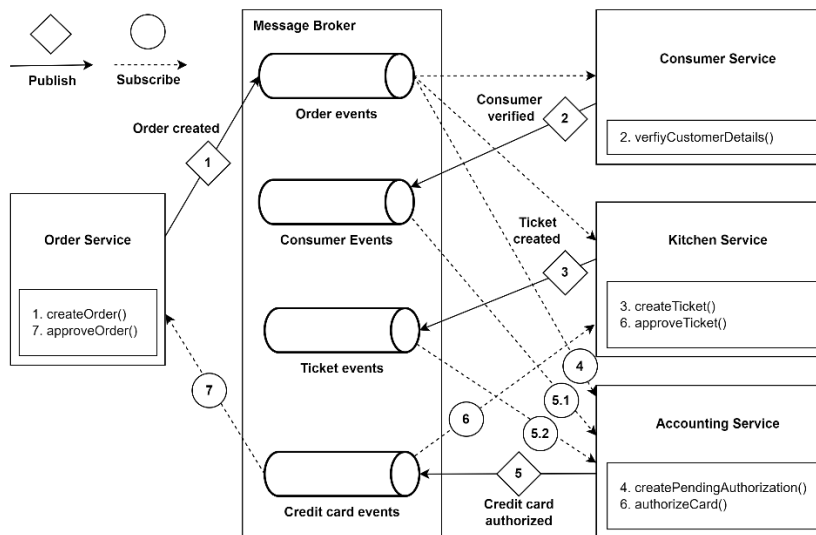


Abb. 7: Event-Choreografie Beispiel [35]

Anhand des Beispiels aus [35] in Abbildung 6 startet der Order Service mit einem OrderCreated-Event. Der Consumer Service reagiert darauf, prüft die Kundendaten und sendet bei Erfolg ein ConsumerVerified-Event. Der Kitchen Service erstellt daraufhin ein Ticket und veröffentlicht ein TicketCreated-Event. Danach autorisiert der Accounting

Service die Zahlung. Läuft alles erfolgreich, schließt der Order Service die Bestellung ab. Für den Fall, dass es zu Fehlern kommt, lösen eigene Events Kompensationen aus, z.B. das Stornieren eines Tickets. [35]

Die Choreografie fördert insgesamt eine hohe Unabhängigkeit der Microservices. Jeder Service muss nur auf bestimmte Events reagieren, passende Aktionen ausführen und bei Bedarf Folge-Events veröffentlichen. Das hält die Logik innerhalb eines einzelnen Services überschaubar. Gleichzeitig fehlt aber auch der zentrale Überblick, denn mit zunehmender Anzahl der Services wird es schwieriger, den Ablauf eines gesamten Prozesses nachzuvollziehen. Außerdem können unbeabsichtigte Abhängigkeiten, Zyklen oder Endlosschleifen entstehen, wenn Services wechselseitig Events auslösen. Es bestehen auch Risiken bei Änderungen an Event-Strukturen, da sie unerwartete Seiteneffekte in anderen Services auslösen können. Der Prozess der Fehlersuche und des Debuggings kann dadurch aufwendig werden. Die Choreografie eignet sich daher gut für einfache und überschaubare Abläufe. [12], [35] Durch das dezentrale, ereignisgesteuerte Modell, reagieren die Services autonom und koordinieren sich selbstständig. Das reduziert die Abhängigkeiten, erhöht die Parallelität und verbessert laut [1] auch Zeit-, Speicher- und Energieverbrauch messbar.

4.3. Vergleich anhand Skalierbarkeit, Flexibilität und Ausfallsicherheit

In diesem Abschnitt wird ein Vergleich zwischen orchestrierten und choreografiebasierten Architekturen im Hinblick auf Skalierbarkeit, Flexibilität und Ausfallsicherheit aufgestellt.

4.3.1. Skalierbarkeit

Im Punkt der Skalierbarkeit lässt sich festhalten, dass die dezentrale und Event-basierte Architektur von Choreografien es ermöglicht, einzelne Services unabhängig voneinander zu replizieren und bei Bedarf flexibel zu skalieren. Dadurch wird gewährleistet, dass die gesamte Lösung auf Lastspitzen oder eine erhöhte Anzahl gleichzeitiger Nutzer:innen besser reagieren kann. Außerdem kann die Skalierung genau dort stattfinden, wo sie auch benötigt wird, und es muss nicht wie im monolithischen Ansatz, das gesamte System skaliert werden. Das kann zu verbesserten Reaktionszeiten, einem höheren Gesamtdurchsatz und einer effizienteren Ressourcennutzung führen. Dies gilt besonders für Systeme, bei denen die Belastung stark schwankt. Im Gegensatz dazu kommen orchestrierte Systeme öfter an ihre Grenzen, weil der zentrale Orchestrator alle Abläufe zwischen den Diensten steuern und koordinieren muss. Diese Komponente kann somit zu einem Bottleneck werden, vor allem wenn die Zahl der Dienste und Transaktionen plötzlich stark zunimmt. Dadurch kann sich die gesamte Leistung des Systems verringern und die Reaktionszeit auf Anfragen verlängern. [11], [35], [41], [42]

4.3.2. Flexibilität

Im Hinblick auf Flexibilität und Modularität bieten choreografiebasierte Architekturen einige wichtige Vorteile. Da jeder Service eigenständig entwickelt, implementiert und bereitgestellt werden kann, ermöglicht sie eine hohe Anpassungsfähigkeit. Das führt dazu, dass Änderungen an einzelnen Services vorgenommen werden können, ohne dass eine zentrale Management-Komponente angepasst werden muss. Besonders bei Systemen, die

mehrere geschäftliche Bereiche abdecken und bei denen sich die Anforderungen oft ändern oder flexibel weiterentwickelt werden müssen, kann das von Vorteil sein. Orchestrierte Microservices haben dagegen strikt festgelegte, zentrale Abläufe. Das macht sie zwar übersichtlich und gut nachvollziehbar, schränkt jedoch ihre Flexibilität ein. Änderungen an einzelnen Diensten erfordern eine Anpassung des Orchestrators, was die Komplexität erhöhen und zu Verzögerungen im Prozess führen kann. [12], [35], [39], [43]

4.3.3. Ausfallsicherheit

Im Bereich der Ausfallsicherheit besitzen choreografiebasierte Architekturen wichtige Stärken. Weil die einzelnen Services stark voneinander getrennt sind, führt ein Ausfall in einem Service nicht automatisch dazu, dass andere Teile des Systems ebenfalls betroffen sind. Die Verwendung von asynchroner Kommunikation und Event-Streaming trägt zusätzlich zur Ausfallsicherheit des Gesamtsystems bei, da Nachrichten in den meisten Message-Broker- und Streaming-Lösungen zwischengespeichert und bei Fehlern später erneut verarbeitet werden können. Auf der anderen Seite erleichtert die zentrale Steuerung bei orchestrierten Architekturen das Monitoring, die Diagnose und die Behebung von Fehlern. Gleichzeitig aber bringt die zentrale Steuerkomponente das Risiko eines Single-Point-Of-Failure mit sich. Moderne orchestrierte Systeme versuchen das Risiko eines kompletten Ausfalls zu minimieren, indem sie Redundanzen einbauen und Verfügbarkeitslösungen nutzen, wie das Bereitstellen von Ersatzservern wenn einer abstürzt. Das resultiert darin, dass der Gesamtbetrieb auch bei Störungen fortgesetzt werden kann. Solche Absicherungen treiben jedoch gleichzeitig die Kosten in die Höhe. [12], [35], [41], [42], [43]

4.3.4. Fazit

Aus der Literaturrecherche zeigt sich zusammenfassend, dass choreografiebasierte Microservice-Architekturen vor allem dort Stärken zeigen, wo hohe Skalierbarkeit, Flexibilität und Ausfallsicherheit gefragt sind. Die lose Kopplung und asynchrone Eventkommunikation erlauben es, Services unabhängig voneinander zu entwickeln, bei Bedarf zu skalieren und Ausfälle im System besser zu behandeln. Zudem erfordert dieses Maß an Dezentralität eine saubere Planung der Events, um unerwünschte Abhängigkeiten und Fehlerzustände zu vermeiden.

Orchestrierte Ansätze bieten dafür eine klare zentrale Steuerung der Abläufe und vereinfachen das Monitoring und die Fehlerbehandlung. Sie stoßen bei wachsender Komplexität an ihre Grenzen und bringen das Single-Point-Of-Failure und Bottleneck-Risiko mit sich. Bei monolithischen Systemen sind starke Konsistenz und einfache Transaktionen im Vordergrund, es besteht aber eine geringere Anpassungsfähigkeit und sie sind insgesamt schwieriger zu skalieren.

In der Praxis hängt die Wahl der Architektur davon ab, welche Anforderungen an Skalierung, Anpassbarkeit und Stabilität im Vordergrund stehen und wie viel Komplexität beim Design und Betrieb managebar ist.

5. Entwurfsmuster und Best Practices

Zur Beantwortung der Teilforschungsfrage 2 wurde eine Literaturrecherche durchgeführt, welche auf Fachpublikationen, Fallstudien und weit verbreitete Best Practices aus Fachbüchern setzt. Ziel war es, die in der Praxis und Forschung am häufigsten empfohlenen Muster, Technologien und Konzepte zu identifizieren, welche speziell für choreografiebasierte Microservices in E-Shop-Anwendungen relevant sein können.

5.1. Event Driven Architecture

Event-Driven-Architecture (EDA) ist ein Architekturstil, bei dem Softwarekomponenten durch das Erzeugen, Empfangen und Verarbeiten von Ereignissen miteinander kommunizieren. Ein Event stellt dabei eine bedeutende Veränderung im System dar, zum Beispiel eine Benutzeraktion, ein Systemprozess oder eine Datenmanipulation. Solche Ereignisse lösen Nachrichten aus, welche wiederum eine Reaktion einzelner Komponenten oder des gesamten Systems verursachen. Der entscheidende Vorteil von EDA liegt darin, dass beteiligte Services die Events parallel verarbeiten können, ohne aufeinander warten zu müssen. Das unterscheidet sie von klassischen, zentral gesteuerten Kommunikationsmodellen. Bei einer EDA kann es jedoch genau aus diesen Gründen schwierig sein, Konsistenz sicherzustellen. [44]

Im Vergleich zu eng gekoppelten, API-basierten Ansätzen reduziert EDA den Kommunikationsaufwand zwischen Diensten und kann damit den Durchsatz, die Reaktionszeit und Fehlerbehandlung verbessern. [45] Werkzeuge wie Apache Kafka, RabbitMQ oder NATS-Streaming (Neural Autonomic Transport System) unterstützen dabei die Verwaltung des Eventflusses. [44], [46] Ein zentrales Element ist der Einsatz eines Eventlogs, welcher die Nachvollziehbarkeit von Nachrichten erhöht, Wiederholungsversuche nach Fehlerzuständen ermöglicht und das Verpassen wichtiger Events verhindert. [45]

Myllynen et al. (2023) [43] zeigt, dass die Event-Choreografie, der zentralisierten Orchestrierung vorzuziehen ist, da sie die lose Kopplung der Services stärkt und die Skalierbarkeit vereinfacht. Anhand von Fallstudien in den Bereichen E-Commerce und Finanzdienstleistungen wird verdeutlicht, wie EDA die Reaktionsfähigkeit und Konsistenz komplexer Systeme erhöht, während die Subsysteme autonom bleiben [43].

Empirische Ergebnisse von Rahmatulloh et al. (2022) [46] bestätigten den Mehrwert von EDA. In einer Benchmark Studie konnte EDA gegenüber API-gesteuerten Ansätzen eine um 19,18 % schnellere Reaktionszeit und eine um 34,40 % geringere Fehlerrate erzielen, bei einem Anstieg der CPU-Auslastung um 8,52 % aufgrund der ereignisbasierten Nachrichtenverarbeitung. Diese Ergebnisse zeigen, dass EDA insbesondere unter erhöhter Last die Performance von Microservice-Architekturen messbar verbessern kann. [46]

Zusätzlich wurden in der Literaturrecherche von Karabey Aksakalli et al. (2021) [47] insgesamt sieben verschiedene Arten von Kommunikationsmustern und drei Bereitstellungsansätze identifiziert. Die Publish/Subscribe-Kommunikation (One-to-Many-Interaktionen) erwies sich dabei als die am häufigsten verwendete Methode unter den identifizierten Kommunikationsmustern. [47]

5.2. Idempotent Consumer

“In idempotent operations, the outcome doesn’t change after the first application, even if the operation is subsequently applied multiple times. If operations are idempotent, we can repeat the call multiple times without adverse impact. This is very useful when we want to replay messages that we aren’t sure have been processed, a common way of recovering from error.” [12]

Kommunikation über Events kann manchmal dazu führen, dass Nachrichten mehrfach zugestellt werden. Typische Ursachen sind Netzwerkprobleme, Wiederholungslogiken auf Clientseite oder Fehler bei verteilten Transaktionen. [48], [49]

Mehrfach zugestellte Nachrichten können jedoch Probleme verursachen, zum Beispiel wenn finanzielle Buchungen doppelt getätigt werden. Zusätzlich kann es zu inkonsistenten Datenständen oder unnötigem Ressourcenverbrauch durch mehrfache Bearbeitung führen. Um das zu verhindern, ist das Prinzip der Idempotenz ein wichtiges Konzept. Eine Operation ist dann idempotent, wenn sie bei wiederholter Ausführung keine zusätzlichen Effekte verursacht. In verteilten Systemen bedeutet das, dass ein Service doppelte Anfragen zuverlässig ohne negative Nebenwirkungen behandeln muss. Ein bewährtes Muster, um Idempotenz umzusetzen, ist der Idempotente Konsument. Dazu sind einige Aspekte zu beachten: Jede Nachricht muss eine eindeutige Kennung besitzen, der Service muss bereits verarbeitete Nachrichten erfassen, die Verarbeitung darf nur bei erstmaliger Ankunft erfolgen und alle Operationen müssen atomar sein, um Inkonsistenzen zu verhindern. [48], [12] Yadav und Mantri (2024) [48] heben einige Vorteile des Patterns hervor, wie zum Beispiel eine verbesserte Datenintegrität, Systemstabilität und eine einfachere Fehlerbehandlung. Herausforderungen wie zusätzliche Komplexität, Leistungsaufwand und Speicheranforderungen werden ebenfalls identifiziert. Insgesamt bietet die Studie wichtige praktische Anleitungen für Entwickler:innen und Architekt:innen, die robuste und zuverlässige Microservice-Systeme aufbauen möchten. [48]

5.3. Database per Service

Daten eines Microservices sollten für diesen privat bleiben und nur über dessen eigene API zugänglich sein. Das bedeutet, dass die Datenbank als Teil der internen Implementierung betrachtet wird und andere Services keinen direkten Zugriff darauf haben dürfen. Um diese Isolation sicherzustellen, kann man verschiedene Ansätze wählen. Jeder Service kann eine eigene Tabelle in einer Datenbank besitzen, ein eigenes Datenbankschema verwenden oder sogar einen eigenen Datenbankserver betreiben. Letzteres ermöglicht es, die Datenbank unabhängig vom Service zu skalieren, etwa in Form eines Datenbank-Clusters. So bleibt die Hoheit der Daten beim Service selbst und die Architektur bleibt sauber getrennt und gut skalierbar. [50]

Laigner et al. (2020) [51] liefern eine umfassende Untersuchung zum Stand der Praxis, den Herausforderungen sowie den zukünftigen Forschungsrichtungen im Bereich des Datenmanagements bei Microservices. Ein wichtiges Ergebnis der Studie ist die detaillierte Gliederung von Motivationen und Mustern für das Datenmanagement. Erkenntnisse bestätigen frühere Beobachtungen von Pritchett (2008) zur Rolle der funktionalen Partitionierung sowie von Richards (2015) zur dezentralen Datenverantwortung. Das Muster „Database per Service“ dominiert und deckt sich mit den Prinzipien von Fowler und

Lewis (2014), welche die Datenbank Autonomie als wichtigen Grundpfeiler gut designer Microservices betonen. [51] Nogueira et al. (2024) [52] ermittelten, dass 58 % der 53 Befragten eine eigene Datenbank pro Microservice verwenden.

5.4. Event Sourcing

Event Sourcing ist ein Designpattern, bei dem der Zustand eines Systems nicht direkt gespeichert, sondern aus einer Reihe von Events rekonstruiert wird. Der komplette Verlauf dieser Events wird gespeichert, sodass man den Systemzustand zu jedem Zeitpunkt wiederherstellen oder zurückspulen kann. [35], [53] Eine Sammlung aller Events ermöglicht es auch eine lückenlose Historie zu führen, was Audits, Debugging und Nachvollziehbarkeit erleichtert. [54]

Die Arbeit von Charankar und Pandiya (2024) [55] zeigt, wie Event Sourcing die Leistung, Skalierbarkeit und Ausfallsicherheit von Microservices verbessert. Es löst typische Probleme wie ineffiziente Abfragen, Engpässe in der Performance und schlechte Fehlertoleranz. Vorteile sind unter anderem ein verringerter Overhead durch Datenbanken, bessere Skalierbarkeit durch parallele Verarbeitung, bessere Nachvollziehbarkeit, höhere Ausfallsicherheit durch wiederholbares Abspielen von Events und mehr Flexibilität bei Weiterentwicklungen. Praxisbeispiele von Netflix, Uber und verschiedenen Finanzplattformen bestätigen diese Vorteile. Insgesamt wird Event Sourcing als zukunftsicheres Muster für komplexe Microservices vorgestellt. [55]

Overeem et al. (2021) [53] untersuchten den praktischen Einsatz von Event Sourced Systems (ESSs), also Systemen die auf Event Sourcing basieren. Mithilfe der Grounded Theory befragten die Autor:innen 25 Entwickler:innen aus 19 Projekten, um Beweggründe, Merkmale und Herausforderungen von Event Sourcing in der Praxis zu ergründen. Als Hauptmotivation für den Einsatz nennen sie Nachvollziehbarkeit, Flexibilität, den Umgang mit Komplexität und Trends in der Branche. [53]

Alongi et al. [54] zeigten, wie Event Sourcing genutzt werden kann, um Softwarearchitekturen beobachtbarer zu gestalten. Beobachtbarkeit wird hierbei klar von Überwachbarkeit abgegrenzt und soll das Systemverhalten nachvollziehbarer machen. Anhand einer Fallstudie (oMVC) zeigen sie, dass Event Sourcing zwar die Nachvollziehbarkeit und Wartbarkeit verbessert, aber auch die Komplexität und Kosten erhöht. [54]

5.5. CQRS

CQRS (Command-Query Responsibility Segregation) beschreibt ein alternatives Modell zur Speicherung und Abfragen von Daten. Im Gegensatz zu klassischen Datenbanken, welche Änderungen und Abfragen in einem einzelnen System vereinen, trennt CQRS diese Verantwortlichkeiten. Ein Teil verarbeitet Commands (Zustandsänderungen), der andere Teil nur Queries (Abfragen). Commands werden geprüft und bei Erfolg auf das Datenbankmodell angewendet. Sie enthalten immer den Zweck der Änderung und können direkt oder später verarbeitet werden. Die internen Modelle für Commands und Queries sind stets voneinander getrennt. Das erlaubt es, Commands und Queries unabhängig zu skalieren oder mit unterschiedlichen Speichersystemen zu betreiben, und dem jeweiligen Anwendungsfall anzupassen. So können zum Beispiel auch verschiedene Datenformate

unterstützt werden. CQRS stellt allerdings einen Bruch mit der klassischen CRUD-Vorgehensweise dar, und selbst erfahrene Teams stoßen bei der korrekten Umsetzung auf Schwierigkeiten. [12], [56], [57], [58], [59]

Die Arbeit von Kumaresan Durvas Jayaraman und Dr. Pooja Sharma [56] untersucht, wie das CQRS-Muster die Architektur von Webanwendungen verbessern kann. Sie diskutieren, dass bei klassischen Architekturen ein gemeinsames Datenmodell für Lese- und Schreiboperationen zu Leistungsdefiziten, komplexen Modellen und begrenzter Skalierbarkeit führen kann. CQRS versucht genau dieses Problem zu lösen. Vorteile sind vor allem schnellere Abfragen, bessere Erweiterbarkeit, höhere Flexibilität bei Geschäftslogiken und einfachere Wartung. Fallstudien bei Netflix und Uber, zeigen deutliche Verbesserungen, zum Beispiel kürzere Antwortzeiten, geringere CPU-Auslastung der Datenbank und höhere Verfügbarkeit bei Lastspitzen. Befragungen bestätigen, dass die Mehrheit der Fachleute nach Einführung von CQRS bessere Performance und Skalierbarkeit erreicht hat. [56]

Long (2017) [49] zeigt ebenso auf wie die Leistungsfähigkeit von CQRS-Architekturen gezielt verbessert werden kann. Zunächst wird erläutert, welche Probleme klassische CRUD-Architekturen mit sich bringen: Da die gleichen Entitäten für Lese- und Schreibzugriffe verwendet werden, müssen oft unnötig große Datenmengen verarbeitet werden, auch wenn sich nur einzelne Felder ändern. Das „ENode-Framework“, welches auf den im Paper beschriebenen Prinzipien basiert, konnte auf einem handelsüblichen i7-Prozessor bis zu zwei Millionen Nachrichten in nur fünf bis sechs Sekunden verarbeiten. [49]

Roh et al. [60] beschäftigen sich mit der Frage, wie „MLOps-Systeme“ effizienter und besser skalierbar gestaltet werden können. Neben dem Saga Pattern entschied man sich für den Einsatz von CQRS aufgrund des Verbesserungspotenzials in Skalierbarkeit und dem reduzierten Risiko von Blockaden bei hoher Last. [60]

Rajković et al. [59] beschreiben, wie das CQRS-Muster eingesetzt wurde, um deren „Medis.NET-System“ in über 20 Gesundheitseinrichtungen zu verbessern. Die Autor:innen zeigen auch hier, dass große, oft unnötige Datenmengen die Performance von Informationssystemen belasten. Sie setzten das CQRS-Muster ein, um häufig genutzte und sich selten ändernde Daten in eine separate Lesedatenbank zu verschieben. So bleibt das System flexibel und effizient. Die Ergebnisse waren dabei um ca. 40 % schnellere Antwortzeiten und um zwei Drittel weniger Datenverkehr. [59]

5.6. Strangler Fig

Im Jahr 2004 stellte Martin Fowler mit dem sogenannten Strangler Fig Pattern eine Strategie zur schrittweisen Modernisierung von Softwaresystemen vor. Die Metapher der Würgefeige beschreibt, wie neue Systemteile um den bestehenden Monolithen herumwachsen, bis dieser vollständig ersetzt ist. Dieses Vorgehen eignet sich besonders für die Migration von monolithischen Architekturen zu Microservices. [7], [61], [62]

Domain-Driven-Design (DDD) liefert dazu einen methodischen Rahmen, um komplexe Systeme anhand fachlicher Domänengrenzen zu analysieren und sinnvoll in Microservices zu zerlegen. Damit lässt sich eine zentrale Herausforderung, die Identifikation bzw. das Definieren von geeigneten Servicegrenzen, systematisch angehen. [61]

Villaca et al. (2024) [63] heben hervor, dass das Strangler Fig Pattern zu den wirksamsten Strategien zählt, um Antipatterns bei der Migration von monolithischen Systemen zu einer Microservice-Architektur zu vermeiden. In Kombination mit DDD hilft dieses Muster, typische Probleme wie den „Big Bang Rewrite“ zu verhindern, also den Versuch, das gesamte System auf einmal zu ersetzen. [35], [63] Außerdem beugt es der Entstehung von „Mega Services“ vor, welche übergroße Dienste sind, die dem Grundgedanken von Microservices widersprechen. [63]

Die Fallstudie von Li et al. (2020) [61] zeigte, dass die Auslagerung eines ressourcenintensiven Moduls in einen eigenen Microservice die Zuverlässigkeit des Gesamtsystems erhöhen und die Belastung der verbleibenden monolithischen Teile messbar verringern konnte. Dadurch wurde die Gesamtperformance verbessert. Die Ergebnisse zeigen, dass der schrittweise Ansatz einer Migration in der Praxis umsetzbar und wirkungsvoll ist, um Altsysteme mit geringerem Risiko in eine moderne Microservice-Architektur zu migrieren. [61]

Vale et al. (2022) [64] zeigten, wie Expert:innen die Auswirkungen von 14 Microservice Designpatterns auf sieben Qualitätsmerkmale (z. B. Skalierbarkeit, Performance, Sicherheit) einschätzen. Neun Interviews belegen: Strangler Fig und Gateway Routing werden am häufigsten genutzt. Alle Befragten nutzen dieses Pattern und die Hälfte sieht bessere Skalierbarkeit als Vorteil, gefolgt von besserer Wartbarkeit. Ein großer Pluspunkt ist auch die Transparenz für die Nutzer:innen, denn Entwickler:innen können auf Microservices umstellen, ohne dass die Kund:innen etwas anpassen müssen oder überhaupt etwas davon mitbekommen. [64]

5.7. API-Gateway

Das API-Gateway Pattern ist ein Konzept, bei dem ein zentraler Einstiegspunkt alle Anfragen entgegennimmt, an die passenden Services weiterleitet und Ergebnisse zusammenfasst. So nutzen Clients nur eine Schnittstelle, statt sich mit mehreren APIs verschiedener Microservices auseinandersetzen zu müssen. [35], [62], [65], [66], [67], [68]

Zusätzlich kann das Gateway Aufgaben wie Authentifizierung, Monitoring oder Lastverteilung übernehmen. Weiters stellt es wichtige Funktionen wie Routing und Ratenbegrenzung bereit. Tools wie AWS API Gateway als „fully managed“ Lösung unterstützen Skalierbarkeit und reduzieren den Betriebsaufwand. Dadurch ist das Pattern ein zentraler Bestandteil von modernen, Cloud-betriebenen, Softwarelösungen. [62] Der Vorteil ist eine insgesamt verbesserte Kommunikation. Jedem Client kann eine für ihn passende API bereitgestellt werden und Services lassen sich leichter ändern, ohne die Clients zu beeinflussen. Nachteile sind mögliche Bottlenecks, steigende Komplexität und der Aufwand, APIs für verschiedene Clients sauber zu verwalten. [35], [62], [65]

Montesi und Weber (2016) [68] heben hervor, dass ein API-Gateway mehrere Muster wie Circuit Breaking, Lastverteilung, Monitoring und Service Discovery vereinen kann, da es strategisch zwischen Clients und Services platziert ist. Sie argumentieren, dass API-Gateways zwar Flexibilität und bessere Verwaltung ermöglichen, gleichzeitig aber auch die Architektur komplexer machen und eine Einbindung von übergreifenden Aspekten wie Sicherheit und Fehlertoleranz erfordern. [68]

5.8. Domain Driven Design

DDD stellt eine Sammlung von Methoden und Konzepten dar, mit dem Ziel Software so zu gestalten, dass sie eng mit den fachlichen Anforderungen eines Unternehmens verknüpft ist. Es stellt damit die fachliche Domäne ins Zentrum der Softwareentwicklung und schafft dafür ein gemeinsames Verständnis zwischen Entwickler:innen und Fachexpert:innen. Dabei wird ein Domänenmodell entwickelt, welches alle Geschäftsregeln und Zusammenhänge abbildet. Dieses Modell dient als Grundlage, um die Software in klar abgegrenzte, fachliche Bausteine zu gliedern. Die Erstellung von Domänenmodellen bringt zahlreiche Vorteile mit sich. Es können Subdomänen identifiziert, zusammengehörige Funktionalitäten erkannt und Aufrufe externer Systeme oder Drittanbieterdienste sichtbar gemacht werden. Durch die Zerlegung eines Gesamtsystems in Microservices und mit Unterstützung von DDD zur Abgrenzung der Verantwortlichkeiten, lassen sich Microservices mit hoher Kohäsion umsetzen. [11], [69], [70]

Velepucha und Flores (2023) [70] fassen den aktuellen Stand der Forschung zu Microservices zusammen und untersuchen, wie Unternehmen ihre monolithischen Anwendungen erfolgreich in Microservices migrieren. Zusammenfassend wurde festgestellt, dass es durchaus wichtig ist, Microservices unter Anwendung von DDD zu entwerfen. Es wird sichergestellt, dass ihre Implementierungsdetails verborgen bleiben und ausschließlich über klar definierte, intelligente Schnittstellen („Smart Endpoints“) genutzt werden. [70]

Vural und Koyuncu (2021) [71] nahmen zwei bestehende, nach DDD entworfene Fallbeispiele und veränderten sie künstlich. Einmal gröber (weniger, größere Services) und einmal feiner (mehr, kleinere Services). Anschließend wurden Kopplung und Kohäsion gemessen und mit einer Mehrzielanalyse (AHP) verglichen. Das Ergebnis zeigt, dass die Originale das beste Verhältnis von niedriger Kopplung und hoher Kohäsion hatten. Daher schließen die Autor:innen, dass in diesen Fällen DDD zur optimalen Granularität der Services führte. [71]

Sangabriel-Alarcón et al. (2024) [72] haben mit einer systematischen Literaturrecherche und einer thematischen Analyse untersucht, wie DDD praktisch in der Entwicklung von Microservices eingesetzt wird. Dabei wurden 35 relevante Fachartikel ausgewertet. Die Ergebnisse zeigen, dass die Identifikation und Abgrenzung von Microservices der Hauptgrund für den Einsatz von DDD ist. Die Hauptconclusio ist, dass DDD helfen kann, die Komplexität von Microservices zu bewältigen, gleichzeitig aber Herausforderungen bestehen bleiben, wie die praktische Umsetzung der Domainmodelle und die Kommunikation zwischen Teams. [72]

5.9. Service Discovery

Microservices müssen Adressen voneinander auf eine dynamische Art und Weise finden um miteinander kommunizieren können. Das Service Discovery Pattern stellt Mechanismen bereit, mit denen sich Services registrieren und ihre Standorte zur Laufzeit auflösen können. Dadurch wird die Kommunikation der Dienste von festen Netzwerkadressen entkoppelt [62], [68], [73]. Beim clientseitigen Service Discovery ermittelt der Client selbst den Standort eines anderen, indem er ein Service Registry abfragt und anhand bestimmter Kriterien (z. B. Lastverteilung) eine passende Instanz auswählt. Ein bekanntes Beispiel dafür ist Netflix

Eureka, bei dem sich Clients registrieren und über den Eureka-Server andere Dienste finden. [8], [62] Im serverseitigen Service Discovery sendet der Client seine Anfrage an einen Load Balancer, der die passenden Instanzen über das Service Registry ermittelt und die Anfrage weiterleitet. So entfällt die Discovery-Logik auf der Client-Seite. Beispiele sind AWS Elastic Load Balancing (ELB) und die integrierte Service Discovery von Kubernetes. Bei der DNS-basierten Discovery registrieren sich Dienste bei einem DNS-Server (Domain Name System) und Clients lösen die Standorte per DNS-Abfrage auf. Ein Beispiel dafür ist Consul, welches ein DNS- und HTTP-basiertes Discovery, Health Checks, Key-Value-Speicher und Multi-Datcenter-Unterstützung bietet. [62] Ein weiterer Vorteil von Service Discovery ist, dass es die Skalierung und dynamische Replikation von Diensten in Cloud-Umgebungen unterstützt, ohne dass Clients oder andere Services manuell angepasst werden müssen. [68]

5.10. Circuit Breaker

Das Circuit Breaker Pattern funktioniert, indem es Aufrufe an Microservices überwacht und diese unterbricht, sobald eine definierte Fehlerquote überschritten wird. So werden weitere fehlerhafte Aufrufe verhindert und der betroffene Service kann sich erholen. Das Gesamtsystem bleibt funktionsfähig, selbst wenn einzelne Komponenten ausfallen. Dadurch wird verhindert, dass der Ausfall eines Services keine anderen abhängigen Services beeinträchtigt. [62]

Grundsätzlich agiert ein Circuit Breaker als Vermittler zwischen Client und Zielservice. Dabei arbeitet er typischerweise mit drei Zuständen. Im geschlossenen Zustand leitet er alle Anfragen wie gewohnt weiter, bis die festgelegte Fehlergrenze erreicht ist. Im offenen Zustand blockiert er alle Zugriffe und meldet dem Client einen Fehler oder startet eine alternative Fehlerbehandlung. Nach einer festgelegten Zeit prüft er im halb-offenen Zustand regelmäßig, ob der Zielservice wieder erreichbar ist, und stellt den Normalbetrieb wieder her, sobald eine erfolgreiche Antwort angekommen ist. [68], [74]

Der Circuit Breaker ist ein einfaches und effektives Muster zur Fehlerbehandlung. Laut Aquino et al. (2019) [75] steigert es Verfügbarkeit und Zuverlässigkeit, da es Fehler schnell abfängt, Fallbacks erlaubt, und Überlastung vermeidet, beispielsweise durch einen lokalen Cache. [75]

Falahah et al. (2021) [74] beschreiben die Vorteile eines Circuit Breakers folgendermaßen: Er verhindert den Zugriff auf fehlerhafte Komponenten und ermöglicht eine schnelle und elegante Fehlerbehandlung. Zudem verhindert er, dass Clients bei nicht verfügbaren Diensten hängen bleiben, und erlaubt die Definition individueller Fallback-Aktionen. Schlussendlich schützt er Services davor, durch zu viele Anfragen überlastet zu werden. [74]

5.11. Token based authentication

JWT (JSON Web Token) ist ein Format für clientseitige Tokens, das 2015 standardisiert wurde. Seitdem ist es in vielen APIs und Authentifizierungsprotokollen wie OpenID Connect (OIDC) weit verbreitet. Ein JWT besteht immer aus drei Teilen: einem Header, einem Payload mit den Claims und einem Signatur- oder Authentifizierungstag. Diese Teile

werden mittels Base64url kodiert und durch Punkte getrennt. Das beschriebene Format nennt man JWS (JSON Web Signature) Compact Serialization. [66], [76]

Der Header enthält Metadaten über den Token, vor allem welcher Algorithmus zur Signierung oder Verschlüsselung verwendet wurde. Das Payload enthält die Claims, also die eigentlichen Informationen über die Benutzer:innen oder das Token selbst. Es gibt dafür eine Reihe standardisierter Felder: „sub“ (Subject) für den Benutzernamen, „exp“ (Expired) für den Ablaufzeitpunkt, „iss“ (Issuer) für den Aussteller, „aud“ (Audience) für die Zielsysteme und „jti“ (JWT ID) als eindeutige ID um Replay-Angriffe zu verhindern. Die Möglichkeit, den Algorithmus blind aus dem Header zu übernehmen darf nicht gegeben sein. Dieser sollte eher serverseitig an den Schlüssel gebunden sein (key-driven cryptographic agility). So kann ein Angreifer den Algorithmus nicht einfach austauschen. Sensible Daten im Payload können mit JSON Web Encryption (JWE) verschlüsselt werden. Dabei wird Authenticated Encryption genutzt, eine Kombination aus Verschlüsselung und Integritätsprüfung. Damit wird gewährleistet, dass die Daten vertraulich bleiben und nicht manipuliert werden können. Gängige Verfahren dafür sind AES (Advanced Encryption Standard) im Galois/Counter Mode (GCM) oder AES im CBC-Modus (Cipher Block Chaining) kombiniert mit HMAC (Hash-based Message Authentication Code). Wichtig ist, dass ein Initialisierungsvektor (IV) verwendet wird, um gleiche Nachrichten immer unterschiedlich zu verschlüsseln. [76] Unter [77] findet man die RFC 7519, die offizielle Spezifikation, welche den JWT-Standard definiert.

Venčkauskas et al. (2023) [78] beschreiben, dass der Einsatz von JWT-Tokens in einer Microservice-Architektur mehrere Vorteile bietet. Da die Tokens zustandslos sind, müssen Session-Daten nicht auf Servern gespeichert werden [67], [78]. Dadurch werden diverse Sicherheitsrisiken verringert. Außerdem ermöglicht der Token-Ansatz eine Entkopplung von Authentifizierung und Autorisierung. Interne Tokens stellen sicher, dass jede Anfrage geprüft wird, was den unbefugten Zugriff erschwert, auch wenn jemand die äußere Sicherheitsgrenze überwindet. Durch die Kombination von externer Authentifizierung und interner Autorisierung lassen sich Berechtigungen nach dem Prinzip minimaler Rechte steuern. Außerdem kann der dezentrale Ansatz die Skalierbarkeit erhöhen und Ausfälle einzelner Dienste besser abfangen, weil die Entscheidung auf mehrere Microservices verteilt wird. [78]

Almeida und Canedo (2022) [66] beschreiben einen weiteren Vorteil gegenüber herkömmlichen Tokens. Die Überprüfung kann direkt auf dem Zielsystem erfolgen, ohne einer notwendigen Verbindung zu einem Authentifizierungsserver. Mit einem JWT lassen sich außerdem Benutzerinformationen direkt aus dem Token selbst auslesen. [66]

In ihrer Studie untersuchen Nasab et al. (2023) [79], wie Sicherheit von Microservice-Systemen in der Praxis umgesetzt wird. Dafür wurden 861 sicherheitsrelevante Punkte aus Open-Source-Projekten, Dokumentationen und Foren ausgewertet. Daraus entwickelten die Autor:innen einen Katalog von 28 Sicherheitspraktiken, die dann in einer Umfrage mit 74 Fachleuten bewertet wurden. Bezogen auf JWT kommt die Studie zu dem Ergebnis, dass der Einsatz von JSON Web Tokens in Microservices als nützlich angesehen wird. Vor allem hervorgehoben wurden die damit verbundene Sicherheit der Kommunikation, Session-Verwaltung (Ablauf und Widerruf) und die Verifizierung von Tokens direkt in den Services. Die Mehrheit der Befragten bestätigte den praktischen Nutzen, betonte aber auch,

dass JWT oft mit zusätzlichen Verfahren wie OAuth kombiniert werden sollte, um generelle Sicherheit zu gewährleisten. [79]

5.12. Observability

Bei Microservices ist es wichtig, eine umfangreiche Observability (Beobachtbarkeit) sicherzustellen, um Zuverlässigkeit, Leistungsfähigkeit und effektive Fehlerdiagnose zu fördern. Beobachtbarkeit bedeutet dabei, dass der interne Zustand eines Systems über äußere Signale wie Protokolle, Metriken oder Ablaufverfolgungen (Traces) nachvollziehbar ist. Im Unterschied zum klassischen Monitoring, das oft nur einzelne Kennzahlen wie die CPU-Auslastung oder Verfügbarkeit überwacht, zielt Observability darauf ab, das Verhalten verteilter, asynchroner Systeme im gesamten zu verstehen. Durch gute Observability lässt sich frühzeitig erkennen, wo und warum Probleme auftreten. Fehlerquellen können so schnell identifiziert und gezielt behoben werden, bevor sie die Zuverlässigkeit des Gesamtsystems gefährden. Gleichzeitig werden Engpässe, langsame Antwortzeiten oder ungewöhnliche Lastspitzen sichtbar gemacht und Ressourcen können je nach Bedarf angepasst und Bottlenecks beseitigt werden [80], [81].

5.12.1. Log-Aggregation

Log-Aggregation beschreibt die zentrale Sammlung und Zusammenführung von Protokolldaten, die in unterschiedlichen Microservices entstehen. Eine zentrale Ablage erlaubt es, Protokolle besser zu durchsuchen, Zusammenhänge nachzuvollziehen und auszuwerten. Dieses Vorgehen vereinfacht die Fehlersuche deutlich und die durchschnittliche Zeit bis zur Wiederherstellung im Störfall kann reduziert werden. [80]

Wenn Logs einheitlich strukturiert sind, liefern sie auch wertvolle Informationen zum Kontext, die gerade beim Analysieren und Lösen von Problemen in verteilten Systemen von großer Bedeutung sind. Newman [12] hebt hervor, dass eine geordnete Protokollierung die Log-Aggregation noch wirkungsvoller macht, weil sich Suchvorgänge beschleunigen und Verknüpfungen über verschiedene Dienste hinweg genauer herstellen lassen [12].

Cândido et al. [82] weisen jedoch darauf hin, dass bei großem Umfang und uneinheitlichen Formaten, Schwierigkeiten bei der Auswertung auftreten können. Sie empfehlen deshalb den Einsatz standardisierter Logging Frameworks und strukturierten Schemata, um über alle Services hinweg eine konsistente und verlässliche Protokollierung herzustellen [82].

Für Microservices ist Log-Aggregation von wichtiger Bedeutung, da sie das Problem von zersplitterten Log-Informationen löst. Ohne eine zentrale Sammlung müssten verteilte Protokolle manuell abgeglichen werden. Das ist ein zeitaufwendiger Prozess, der die Diagnose des Gesamtsystems insgesamt erschwert. Schlussendlich ist Log-Aggregation ein wichtiges Pattern für gute Beobachtbarkeit in Microservice-Architekturen. [12], [82]

5.12.2. Sidecar Pattern

Beim Sidecar Pattern wird ein zusätzlicher Container gemeinsam mit dem Container der Hauptanwendung im selben Pod oder auf demselben Host betrieben. Diese Sidecar-Komponente übernimmt dabei übergreifende Aufgaben wie das Sammeln von Laufzeitinformationen, Sicherheitsfunktionen, Konfigurationsverwaltung oder auch die Steuerung des Datenverkehrs. Dabei muss der Code der eigentlichen Anwendung nicht angepasst werden. [35]

In der Praxis wird dieses Muster häufig in Service Meshes wie Istio oder Linkerd eingesetzt. In diesen fangen die Sidecar Container den Netzwerkverkehr zwischen den Microservices ab. So können Metriken, Protokolle und Traces automatisch, einheitlich und konsistent für alle Services erfasst werden. Insgesamt fördert das eine bessere Beobachtbarkeit. [35], [83]

In Microservices ist das Sidecar Pattern relevant, weil es diese Verantwortlichkeiten sauber trennt und organisiert. Die Logik für die Überwachung bleibt vom fachlichen Code der eigentlichen Anwendung getrennt. Zudem lassen sich Funktionen zur Beobachtbarkeit unabhängig weiterentwickeln. Levin und Benson (2020) [83] betonen jedoch, dass trotz dieser Vorteile mögliche Auswirkungen auf die Performance beachtet werden müssen, wie zum Beispiel zusätzliche Latenzen oder erhöhter Ressourcenverbrauch. Es ist wichtig, Sidecars so zu optimieren, dass sie den Betrieb der Microservices nicht unnötig belasten. [83]

5.12.3. Distributed Tracing

Distributed Tracing schafft Transparenz darüber, welchen Weg Anfragen in verteilten Systemen nehmen und wie viel Zeit ihre Verarbeitung in den einzelnen Services beansprucht. Auf diese Weise lassen sich Zusammenhänge besser nachvollziehen, Engpässe identifizieren und Performanceprobleme in komplexeren Serviceaufrufen analysieren. [81], [84]

Dabei wird jeder eingehenden Anfrage am Eintrittspunkt eine eindeutige Kennung vergeben, welche dann über alle beteiligten Services hinweg weitergereicht wird. Jeder Service protokolliert Informationen zum Trace, wie zum Beispiel einen Zeitstempel oder weitere Metadaten. Anschließend werden diese an ein zentrales Tracing Backend, wie Jaeger oder Zipkin, übermittelt. Dort werden die einzelnen Daten zu einem Gesamtbild zusammengesetzt und ggf. visualisiert, was zudem die Analyse von Performanceproblemen und die Fehlersuche erleichtert [84].

In Microservice-Architekturen bringt Distributed Tracing einen Mehrwert, weil diese Systeme verteilt und häufig asynchron arbeiten. Ohne eine entsprechende Nachverfolgung kann die Ermittlung von Ursachen für Verzögerungen oder fehlerhafte Anfragen mit erheblichem Aufwand verbunden sein. Li et al. (2022) heben hervor, wie wichtig Distributed Tracing insbesondere in großen, komplexen Umgebungen ist, um Probleme schneller zu identifizieren. Die Effektivität dieser Verfahren lässt sich künftig durch den Einsatz von fortgeschrittenen Analysemethoden oder maschinellem Lernen noch steigern. Dadurch könnten Auffälligkeiten und Performance Einbrüche automatisch erkannt werden [84].

5.13. Zusammenfassung

In der folgenden Tabelle 1 wird das Ergebnis der Literaturrecherche zusammengefasst und mögliche Vorteile im Kontext von E-Shop Anwendungen dargestellt.

Muster / Technologie	Zweck	Vorteile	Quellen
Event-Driven Architecture (EDA)	Lose Kopplung durch asynchrone Events, Dienste publizieren/abonnieren selbstständig und ohne zentrale Orchestrierung	Erhöhte Skalierbarkeit, schnelle Reaktion auf Events, unabhängige Domains, Vorteilhaft in E-Commerce (z.B. Bestell- und Lager-Events)	[43], [44], [45], [46], [47]
Idempotent Consumer	Sicherstellung, dass mehrfach zugestellte Events keine Doppelverarbeitung verursachen	Verhindert doppelte Buchungen z.B. bei Bestellungen/Zahlungen im Shop	[12], [48], [49]
Database per Service	Jeder Service hat seine eigene Datenhaltung und API	Klare Datenhoheit, bessere Skalierung, minimiert Abhängigkeiten	[50], [51], [52]
Event Sourcing	Speicherung aller Zustandsänderungen als Events	Volle Nachvollziehbarkeit, einfache Rückverfolgbarkeit, Wiederherstellung früherer Zustände (z.B. Bestellhistorie)	[35], [53], [54], [55]
CQRS	Trennung von Lese- und Schreibmodellen	Optimiert Abfragen, skaliert unabhängig, vermeidet Performance-Engpässe bei hohem Traffic	[12], [49], [56], [57], [58], [59], [60]
Strangler Fig	Schrittweise Migration eines Monolithen zu Microservices	Minimiert Risiko, ermöglicht schrittweise Umstellung eines bestehenden E-Shop-Systems	[7], [35], [61], [62], [63], [64]
API-Gateway	Zentraler Einstiegspunkt für Anfragen, Routing, Authentifizierung	Vereinheitlicht Schnittstellen für Frontend/Apps, schützt interne Services, ermöglicht Client-spezifische APIs	[35], [62], [65], [66], [67], [68]
Domain Driven Design (DDD)	Services werden entlang fachlicher Domänengrenzen geschnitten	Klare Verantwortlichkeiten, hohe Kohäsion, saubere Entkopplung, besonders relevant für E-Shop-Domänen wie Warenkorb, Bezahlung	[11], [69], [70], [71], [72]
Circuit Breaker	Verhindert Fehlerausbreitung, wenn ein Service ausfällt	Erhöht Verfügbarkeit, schützt E-Shop bei Ausfällen einzelner Dienste (z.B. Zahlungsprovider offline)	[62], [68], [74], [75]
Token-based Authentication (JWT)	Zustandslose Authentifizierung & Autorisierung	Einfache Skalierung, sichere Benutzer-Sessions, geeignet für APIs im Frontend & Mobile	[66], [67], [76], [77], [78], [79]
Log Aggregation	Zentrale Sammlung und Auswertung von Logs	Fehler schnell identifizieren, Audit-Trails für Bestellungen und Zahlungen	[12], [80], [82]
Distributed Tracing	Nachvollziehbarkeit von Anfragen durch mehrere Microservices	Identifiziert Performance-Engpässe im E-Shop-Prozess (z.B. langsame Bezahlstrecken)	[81], [84]
Sidecar Pattern	Infrastrukturaufgaben wie Monitoring oder Traffic-Steuerung als Sidecar neben Service	Erleichtert Analyse von Laufzeitinformationen, Monitoring und Sicherheitsaspekte ohne Anwendungscode zu ändern	[35], [83]
Service Discovery	Dynamische Lokalisierung von Diensten	Flexibilität bei Skalierung, automatische Registrierung neuer Shop-Services	[8], [62], [68], [73]

Tab. 1: Zusammenfassende Darstellung der Literaturrecherche

6. Prototyp Implementierung

Dieses Kapitel beschreibt die Prototyp-Implementierung eines E-Shops für digitale Güter. Es werden die funktionalen Anforderungen, die High-Level-Architektur und die eingesetzten Design Patterns und Technologien erläutert.

6.1. Lösungsdesign

Um einen aussagekräftigen und realistischen Vergleich der ausgewählten Technologien Quarkus, Micronaut und Spring Boot zu gewährleisten, wurde in dieser Arbeit ein konsistenter Prototyp eines Online-Shops für digitale Güter jeweils mit allen drei Frameworks implementiert. Der Prototyp dient als Proof-of-Concept, um die praktische Umsetzbarkeit einer Event-basierten Microservice-Architektur mit Java für Cloud-native Umgebungen zu validieren. Das Hauptziel des Prototyps ist die Nachbildung eines typischen online Kaufablaufs für digitale Güter. Durch die Abdeckung aller funktionalen Anforderungen schafft der Prototyp eine Basis zur Messung von Startzeit, Ressourcenverbrauch und Verarbeitungszeit unter realistischen Bedingungen.

6.1.1. Überblick über funktionale Anforderungen

Die in Anhang 1 enthaltene Übersicht dient als Referenz für alle funktionalen Anforderungen und unterteilt sich in ID, Teilanforderung und konkreter Zielsetzung. Die Benutzer-Authentifizierung und Autorisierung, muss sowohl eine sichere Registrierung (ID 1.1–1.3), als auch eine rollenbasierte Zugriffskontrolle (ID 1.2.3) umfassen. Die passwortbasierte Authentifizierung soll durch den Einsatz von Hashing-Algorithmen umgesetzt werden.

Die Verwaltung der Benutzerprofile (ID 2) soll die automatisierte Profilerstellung bei der Registrierung und die nachträgliche Änderung und Validierung von eingegebenen Benutzerdaten abdecken. Zusätzlich sollen administrative Tätigkeiten wie die Produktverwaltung (ID 3) implementiert werden. Weitere Anforderungen betreffen die Darstellung, Suche und Filterung von Produkten (ID 4). Die Anbindung eines Warenkorbs und eines vollständigen Bestellprozesses (ID 5 und 6) muss ebenfalls gewährleistet sein. Ziel der Lizenz-Generierung (ID 8) ist, dass zu jedem bestellten digitalen Produkt, eine eindeutige Seriennummer generiert wird.

Abschließend sollen E-Mail-Benachrichtigungen (ID 9) und Tracking (ID 10) umgesetzt werden, damit Nutzer:innen automatisiert über abgeschlossene oder fehlerhafte Bestellungen informiert werden. Zusätzlich sind Betriebs- und Nutzungsdaten für Analyse- und Monitoring-Zwecke vorhanden.

6.1.2. High Level Architecture

Die Lösung des umgesetzten E-Shops setzt sich aus zehn Java Microservices zusammen, wobei alle bis auf das API-Gateway mit einer eigenen MongoDB Datenbank verbunden sind. Dazu kommt, dass jeder der Services, bis auf das API-Gateway über Apache Kafka als Message Bus miteinander kommunizieren. Das bietet die Grundlage für das Choreografie Muster mit asynchroner, Event-basierter Kommunikation.

Abbildung 8 zeigt die übergeordnete Architektur, die aus den folgenden Services besteht:

- **ApiGateway:** Bietet einen zentralen Einstiegspunkt für alle Clientanfragen und übernimmt das Routing zu nachgelagerten Diensten.
- **AuthService:** Verwaltet Benutzerauthentifizierung, Registrierung und JWT-Token-Generierung.
- **UserService:** Speichert und verwaltet Benutzerprofile und Informationen zu Rollen sowie Metadaten der Benutzer:innen.
- **ProductService:** Verwaltet den Shop-Katalog der angebotenen digitalen Waren.
- **CartService:** Ermöglicht Benutzer:innen das Hinzufügen, Ändern und Entfernen von Artikeln in ihren Warenkorb und startet den Bestellvorgang.
- **OrderService:** Verwaltet Kundenbestellungen und verfolgt deren Status.
- **PaymentService:** Validiert Zahlungsmethoden, verarbeitet Transaktionen und veröffentlicht Events bei erfolgreicher oder fehlgeschlagener Zahlung.
- **LicenseService:** Generiert und verwaltet digitale Lizenzen für gekaufte Waren.
- **NotificationService:** Versendet Benutzerbenachrichtigungen, z. B. Bestätigungs-E-Mails mit Lizenzen.
- **TrackingService:** Erfasst, speichert und analysiert alle wichtigen Geschäftsereignisse für Monitoring von Events und Benchmarking der Verarbeitungszeit.

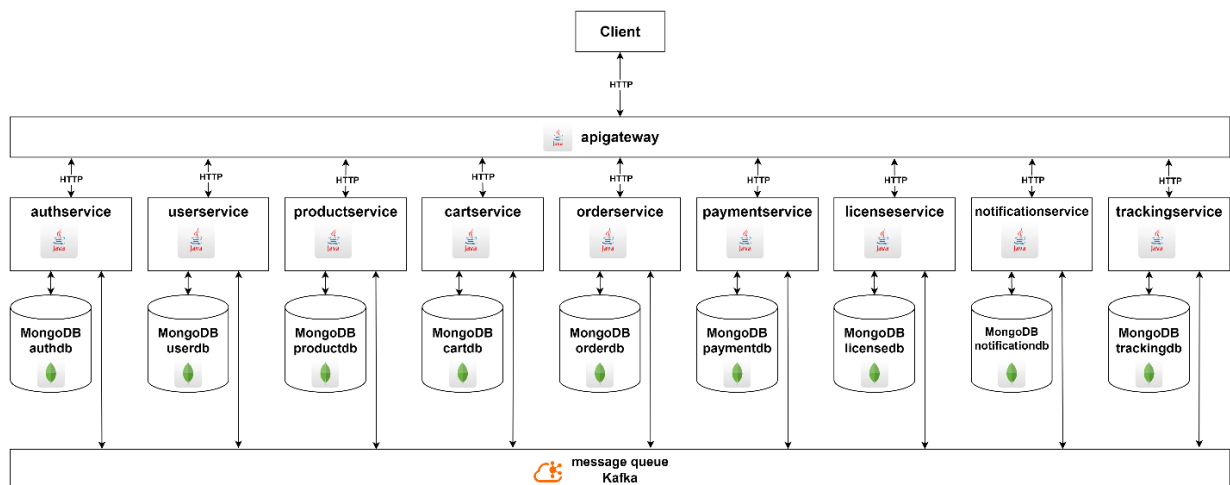


Abb. 8: High-Level-Architektur der E-Shop-Lösung

6.2. Einsatz von Design Patterns

Um den Prototyp an moderne Cloud-native Best Practices anzupassen, integriert die Lösung bewusst ausgewählte Architekturmuster welche im Kapitel davor diskutiert wurden.

6.2.1. Event-Messaging und Choreografie

Die Choreografie ist hier zentraler Bestandteil. In Abbildung 9 wird die Umsetzung anhand des Kaufprozesses visualisiert. Ein:e registrierte Benutzer:in gibt eine Bestellung auf, indem der Checkout Prozess über den CartService gestartet wird. Dies löst eine Reihe von Events aus: CheckoutStartedEvent, OrderCreatedEvent, PaymentSuccessEvent (oder PaymentFailEvent), LicensesGeneratedEvent und NotificationSentEvent. Mehrere Dienste

führen lokale Transaktionen durch und erreichen durch das Saga Pattern letztendliche Konsistenz (Eventual Consistency).

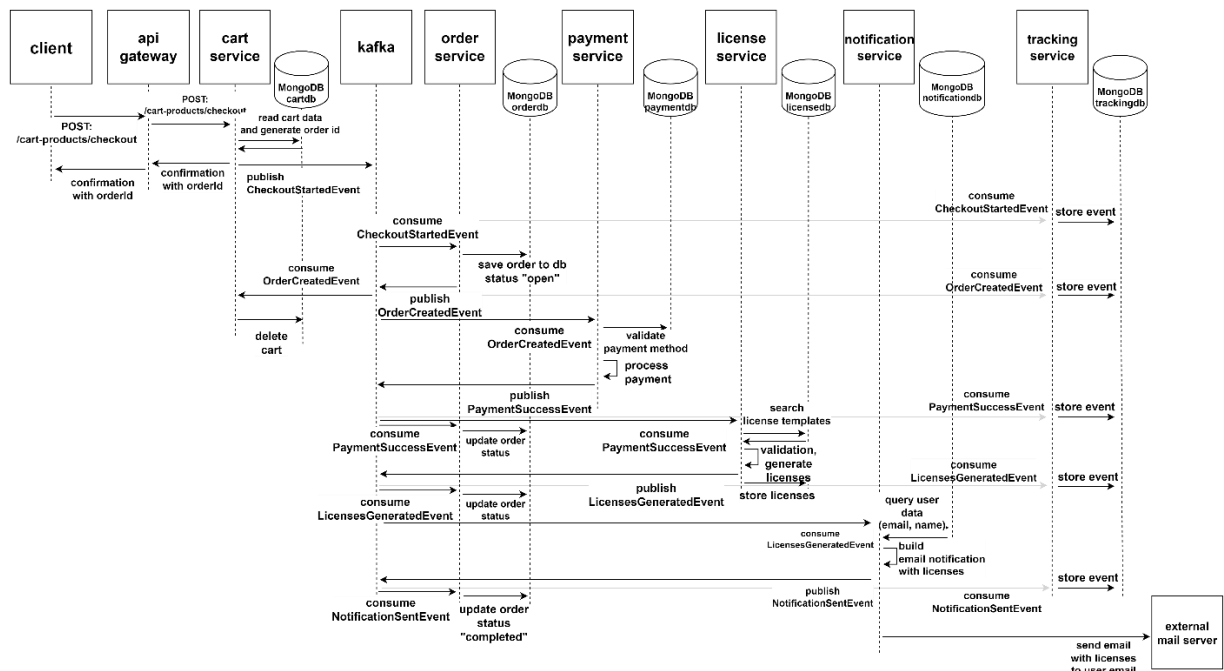


Abb. 9: Event-Choreografie des Einkaufsprozesses

6.2.2. Saga Pattern

Jede lokale Transaktion löst Domain Events aus, die nachfolgende lokale Transaktionen in anderen Services auslösen. Tritt während des Prozesses ein Fehler auf, zum Beispiel eine fehlgeschlagene Zahlung, machen kompensierende Transaktionen die vorherigen Vorgänge rückgängig oder passen sie an. Dieser Ansatz spielt mit der ereignisgesteuerten Choreografie zusammen und fördert die Gesamtkonsistenz. Eine fehlgeschlagene Zahlung in diesem System folgt einer Saga, die aus den folgenden Schritten besteht:

1. Der Client sendet eine POST /cart-products/checkout-Anfrage an das ApiGateway und leitet den Bestellvorgang ein.
2. Das ApiGateway leitet die Anfrage an den CartService weiter.
3. Der CartService liest die Warenkorbdaten aus der MongoDB cartdb, generiert eine Bestellnummer und gibt diese über das ApiGateway an den Client zurück.
4. Der CartService generiert ein CheckoutStartedEvent und veröffentlicht es über Kafka.
5. Der OrderService verarbeitet das CheckoutStartedEvent, erstellt eine Bestellung mit dem Status „OFFEN“, speichert sie in der MongoDB orderdb und gibt ein OrderCreatedEvent aus.
6. Der CartService verarbeitet das OrderCreatedEvent und löscht den entsprechenden Warenkorb aus der Datenbank, da die Bestellung nun aussteht und der Warenkorb nicht mehr benötigt wird.
7. Der PaymentService verarbeitet das OrderCreatedEvent, validiert die Zahlungsmethode und versucht, die Zahlung abzuwickeln.

Wenn die Zahlung fehlschlägt, wird ein `PaymentFailEvent` ausgelöst. Von hier an werden Ausgleichstransaktionen, wie in Abbildung 10 gezeigt, durchgeführt:

1. Der `OrderService` verarbeitet das `PaymentFailEvent`, lehnt die Bestellung ab und aktualisiert den Bestellstatus in der `MongoDB-OrderDB` auf „`PAYMENT_FAILED`“.
2. Der `NotificationService` verarbeitet das `PaymentFailEvent`, fragt Benutzerdaten (E-Mail und Name) aus der Datenbank ab, erstellt eine E-Mail-Benachrichtigung mit den Details zur fehlgeschlagenen Zahlung und sendet ein `NotificationSentEvent`.
3. Der `NotificationService` kommuniziert mit einem externen Mailserver, um den Benutzer:innen eine E-Mail über die fehlgeschlagene Zahlung zu senden.

Während des gesamten Prozesses verarbeitet der `TrackingService` alle relevanten Ereignisse und protokolliert sie für Analyse- und Auditzwecke in der `MongoDB TrackingDb`.

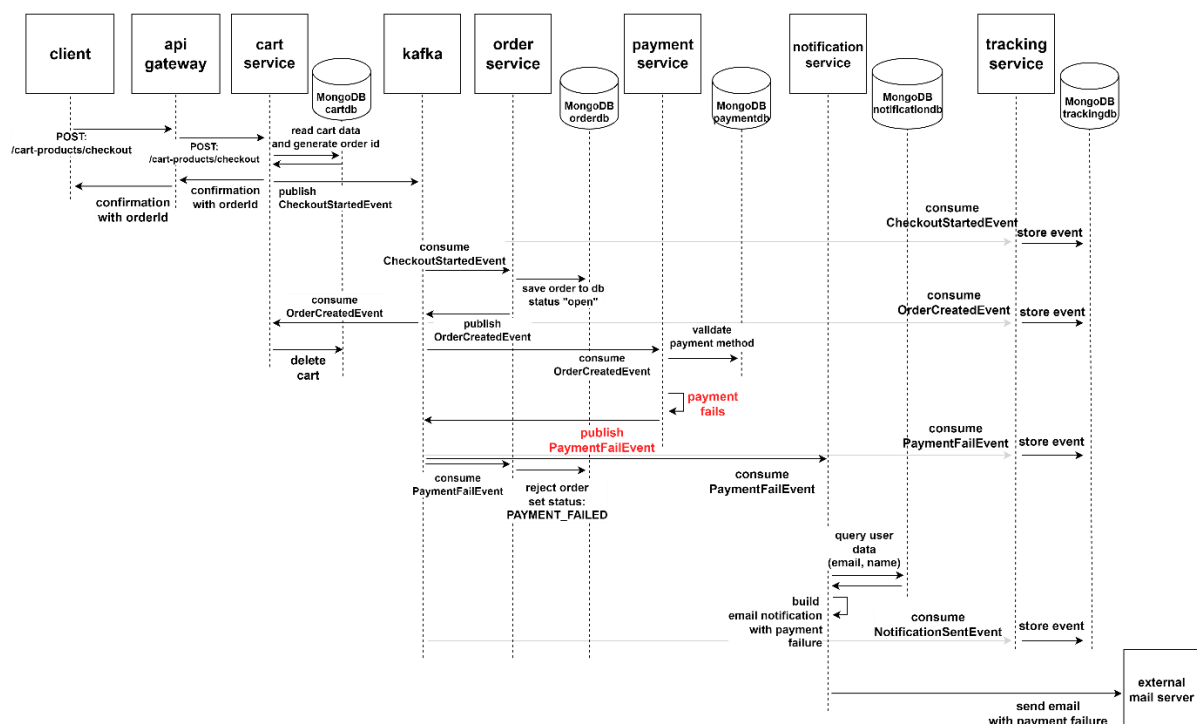


Abb. 10: Saga-Ablauf im Fehlerfall mit Ausgleichstransaktionen

6.2.3. Database per Service – CQRS

Jeder Microservice wendet das Database per Service Pattern an und stellt sicher, dass er der Eigentümer seiner Daten bleibt. Diese Trennung unterstützt die unabhängige Bereitstellung und Versionierung von Services und minimiert gleichzeitig unbeabsichtigte Nebeneffekte, die durch gemeinsam genutzte Datenbanken entstehen können.

Um komplexe Abfragen zu verarbeiten und die Leseleistung zu verbessern, ohne die Autonomie der Dienste zu beeinträchtigen, nutzt die Lösung außerdem das CQRS-Muster. Beispielsweise speichert der `OrderService` eine lokale Kopie bestimmter Benutzerdaten, wie die Rolleninformationen, und aktualisiert diese durch das Abonnieren relevanter Domänenevents wie `UserUpdatedEvent`. Das vermeidet ineffiziente synchrone Suchvorgänge zur Laufzeit und stellt gleichzeitig sicher, dass die einzelnen Dienste losegekoppelt bleiben.

6.2.4. API-Gateway

Das API-Gateway fungiert als zentraler Einstiegspunkt für alle externen Client-Anfragen. Es vereinfacht die Kommunikation mit den zugrunde liegenden Microservices, da die Clients die Adressen der einzelnen Dienste nicht kennen müssen. Durch Routing, Aggregation und Weiterleiten von Client-Anfragen verringert das API-Gateway insgesamt die Komplexität des verteilten Systems.

Zudem ermöglicht dieser Ansatz, die Dienste in einem privaten Netzwerk zu verbergen und sie nicht über das öffentliche Internet zugänglich zu machen. Schlussendlich können unterschiedliche API-Gateways eingesetzt werden, beispielsweise beim Erstellen einer Benutzeroberfläche für Administrator:innen und einer für Kund:innen.

6.3. Cross-Cutting Concerns

Über die zentralen Architekturmuster hinaus sind mehrere übergreifende Aspekte in den Prototyp integriert, um dessen praktische Relevanz zu erhöhen und ihn an reale Anforderungen anzupassen.

6.3.1. JWT-Token

Die Sicherheit wird durch JWTs gewährleistet, welche die zustandslose und skalierbare Benutzerauthentifizierung und -autorisierung ermöglichen. Durch die direkte Einbettung von Claims und Rollen in den Token kann jeder Dienst Anfragen unabhängig validieren, ohne dass wiederholte Aufrufe eines zentralen Authentifizierungsservers erforderlich sind.

6.3.2. Hashing

Zum Schutz sensibler Benutzerdaten implementiert das System Hashing-Mechanismen mit dem BCrypt-Algorithmus (Blowfish Crypt). Dadurch wird sichergestellt, dass sensible Daten wie Passwörter nicht im Klartext gespeichert werden und die Passwortüberprüfung im Falle einer gehackten Datenbank sicher bleibt. Die Verwendung von BCrypt in allen drei Frameworks bietet zudem eine faire Grundlage für den Vergleich der Gesamtleistung in den verschiedenen Frameworks.

6.3.3. Monitoring

Das Monitoring erfolgt über einen dedizierten TrackingService, der Events im gesamten System erfasst und Metadaten wie Zeitstempel, Session IDs und Korrelationskennungen aller Anfragen speichert. Das ist angelehnt an die zuvor diskutierten Patterns der Log-Aggregation und Distributed Tracings. Dieses Design ermöglicht eine Rückverfolgbarkeit von Benutzeraktionen und Workflows. Weiters ermöglicht es Leistungsanalysen, wie beispielsweise die Messung der Verarbeitungszeit von Bestellungen und den dazugehörigen Zwischenschritten. Zusätzlich helfen sie, Engpässe beim Testen nativer und JVM-basierter Deployments zu identifizieren.

6.3.4. API-First und Code Generation

Die Lösung verfolgt einen vertragsorientierten Entwicklungsansatz mithilfe von OpenAPI-Spezifikationen [85]. Durch die Definition der Serviceschnittstellen im Vorhinein bleiben die APIs über die drei verschiedenen Implementierungen hinweg konsistent. Die automatisierte Codegenerierung sowohl für Server Stubs als auch für REST-Clients reduziert den manuellen Aufwand und minimiert Implementierungsfehler. Außerdem beschleunigt es

insgesamt die Entwicklung. Es wird sichergestellt, dass alle Microservices einem gemeinsamen, gut dokumentierten Schnittstellenstandard entsprechen.

6.4. Einschränkungen

Obwohl der E-Shop-Prototyp so konzipiert wurde, dass er eine realistische Microservice-Architektur widerspiegelt, sollen einige bewusste Vereinfachungen den Umfang reduzieren, damit das Projekt im Rahmen dieser Masterarbeit durchführbar bleibt.

Eine wesentliche Einschränkung besteht darin, dass bestimmte externe Abhängigkeiten, wie beispielsweise echte Zahlungsgateways nicht vollständig integriert sind. Stattdessen wird der Zahlungsvorgang simuliert, um einen realistischen Prozess ohne tatsächliche Finanztransaktionen nachzubilden (siehe Anforderung ID 7).

Eine weitere Einschränkung ist das Fehlen von Maßnahmen zur Ausfallsicherheit. Während das System beispielsweise für die Event-Verwaltung auf Kafka setzt, sind Mechanismen zur Bewältigung von Broker-Ausfällen, zur Nachrichtenwiedergabe oder zur Partitionsneuverteilung nicht implementiert. Dadurch liegt der Fokus auf der Choreografie-Logik und der Bewertung der Frameworks, anstatt auf dem Aufbau eines produktionsreifen verteilten Systems mit hohen Garantien an Verfügbarkeit.

Schließlich wurden einige nicht wesentliche Geschäftsfunktionen bewusst weggelassen. Das betrifft unter anderem benutzergenerierte Produktbewertungen, personalisierte Empfehlungsmechanismen oder ein Frontend mit Dashboards. Diese sind zwar für einen vollwertigen E-Shop relevant, tragen aber nicht direkt zum primären Forschungsziel bei.

7. Vergleich der Implementierungen

Um Teilfrage 3 zu behandeln, werden im Folgenden wesentliche Unterschiede in der Umsetzung mit den Frameworks anhand ausgewählter Aspekte veranschaulicht. Dabei werden kritische Codestellen dargestellt und ggf. untereinander verglichen. In einem abschließenden Fazit werden Herausforderungen, Vorteile und Defizite sowie die im Zuge der Implementierung gewonnenen Learnings aufgezeigt.

7.1. API

Alle drei Prototypen unterstützen bei der Gestaltung der API-Schicht generierte Schnittstellen, und arbeiten mit einheitlich definierten Routen und einer Trennung von Definition (Interface) und Implementierung.

Quarkus nutzt für die Umsetzung von REST-APIs die Jakarta RESTful Web Services (JAX-RS)-Spezifikation zusammen mit RESTEasy Reactive. Routen werden mit `@Path` definiert, HTTP-Methoden mit `@POST`, `@PUT` oder anderen HTTP-Annotationen. Die Formate für Ein- und Ausgabe steuert Quarkus mit `@Consumes` und `@Produces`, zum Beispiel JSON. [86]

Ein Endpunkt wird dabei in einem Interface beschrieben, während die konkrete Logik in einer Serviceklasse implementiert ist. Diese Trennung erleichtert die Wartung und Erweiterungen. Die Validierung von Anfragen erfolgt direkt an den Methodensignaturen mit `@Valid` oder `@NotNull`. Die Verarbeitung wird mittels RESTEasy Reactive effizient umgesetzt und unterstützt eine deklarative, klar nachvollziehbare API-Definition. [86]

```
@Path("/auth")
public interface AuthApi {
    @POST
    @Path("/register")
    @Consumes({ "application/json" })
    @Produces({ "application/json" })
    RegisterResponse register(@Valid @NotNull RegisterRequest registerRequest);
}

@RequestScoped
public class AuthApiService implements AuthApi {
    ...
    @Override
    public RegisterResponse register(RegisterRequest registerRequest) {
        // Implementierung
    }
}
```

Code 1: Rest Interface – Quarkus

Micronaut bindet seine HTTP-Endpunkte auch über einen annotationsbasierten Ansatz an den internen HTTP-Server Netty. Der Einstiegspunkt ist die Annotation `@Controller`, die eine Klasse als REST-Controller markiert. Methoden werden mit `@Post` oder `@Put` an die jeweiligen HTTP-Operationen zugewiesen. Eingehende Daten übernimmt `@Body`, während `@PathVariable` und `@QueryValue` das Einbinden von Pfad- und Query-Parametern ermöglicht. Die Implementierung der Endpunkte erfolgt meist in einer `@Singleton-Bean`, die alle Abhängigkeiten, wie Authentifizierung oder Mappings, enthält. [87], [88]

```

@Controller
public interface AuthenticationApi {
    @Post("/auth/login")
    LoginResponse login(
        @Body @NotNull @Valid LoginRequest loginRequest
    );
}

@Controller
@Singleton
public class AuthApiService implements AuthenticationApi, CredentialsApi {
    @Override
    @Secured(SecurityRule.IS_ANONYMOUS)
    public LoginResponse login(LoginRequest loginRequest) {
        //Implementierung
    }
}

```

Code 2: Rest Interface – Micronaut

Spring Boot setzt auf das Spring MVC-Modul. REST-Controller werden mittels der Annotation `@RestController` markiert, die automatisch JSON-Antworten liefert. Routen definiert man mit `@RequestMapping` oder den spezifischen Kurzformen wie `@PostMapping`. Für JSON-Bodies wird `@RequestBody` genutzt, für Pfadparameter `@PathVariable`. [89]

Die Schnittstellen und ihre Implementierungen können auch getrennt bleiben. Die Deklaration legt fest, welche Routen verfügbar und wie sie aufgebaut sind, und die Serviceklasse enthält die fachliche Logik.

```

public interface AuthApi {
    ...
    @RequestMapping(
        method = RequestMethod.POST,
        value = "/auth/login",
        produces = { "application/json" },
        consumes = { "application/json" }
    )
    @ResponseStatus(HttpStatus.OK)
    LoginResponse login(
        @Parameter(name = "LoginRequest", description = "User login credentials", required =
true) @Valid @RequestBody LoginRequest loginRequest
    );
}

@RestController
public class AuthApiService implements AuthApi {
    @Override
    public LoginResponse login(LoginRequest loginRequest) {
        //Implementierung
    }
}

```

Code 3: Rest Interface – Spring Boot

7.2. Persistenz - Datenbankzugriff

Die Implementierung der Persistenz zeigt, wie Quarkus, Micronaut und Spring Boot den Zugriff auf MongoDB handhaben. Alle drei Lösungen arbeiten dabei deklarativ, um CRUD-Operationen mit wenig Boilerplate-Code zu ermöglichen. Die Unterschiede liegen vor allem in der Abstraktionsebene, den verwendeten Patterns und den eingesetzten Technologien.

Quarkus verwendet für den Datenzugriff das Panache-Framework. Diese Erweiterung stellt eine domainspezifische API bereit, mit der CRUD-Operationen sehr kompakt formuliert werden können. Die Entitäten (z. B. `RegistrationEntity`) erben von

PanacheMongoEntityBase. Damit ist das Active-Record-Prinzip umgesetzt: Jede Entität enthält standardmäßig Methoden, um die Daten direkt zu suchen, zu speichern, zu verändern oder zu löschen. Es wird jedoch auch der Repository Pattern unterstützt. [90], [91]

```
@MongoEntity(collection = "registrations")
public class RegistrationEntity extends PanacheMongoEntityBase {
    @BsonId
    public UUID id;
    ...
    public static RegistrationEntity findById(UUID userId) {
        PanacheQuery<RegistrationEntity> result = RegistrationEntity.find("userEntity._id",
        userId);

        return result.firstResult();
    }
    ...
}
```

Code 4: Datenbankzugriff mit Panache – Quarkus

Methoden wie `persist()`, `update()` oder `delete()` sind direkt verfügbar. Die MongoDB-Verbindung wird hierbei über Konfigurationsdateien gesteuert. Quarkus übernimmt die Verwaltung der Datenbankkommunikation automatisch. [90], [91]

Micronaut setzt auf Micronaut Data MongoDB. Dieses Modul nutzt eine Abstraktion durch Annotationen, um MongoDB-Dokumente mit Java-Klassen abzubilden. Die Entitäten sind mit `@MappedEntity` versehen und Repositories werden via dem Interface `CrudRepository` definiert. Abfragen können mit `@MongoFindQuery` genau beschrieben werden. [92]

```
@MongoRepository
public interface RegistrationRepository extends CrudRepository<RegistrationEntity, UUID> {

    @MongoFindQuery("{ 'userEntity._id': :userId }")
    Optional<RegistrationEntity> findById(UUID userId);
    ...
}
```

Code 5: Datenbankzugriff mit CRUD Mongorepository – Micronaut

Die `@Serdeable`-Annotation auf den Entity-Klassen sorgt dafür, dass die Objekte bei Bedarf JSON-serialisierbar sind. Entwickler:innen erhalten so eine Kombination aus Repository-Pattern und vereinfachter Query-Definition. [92], [93]

Spring Boot verwendet das Modul Spring Data MongoDB. Dieses stellt über den `spring-boot-starter-data-mongodb` alle nötigen Komponenten bereit, darunter Treiber, Konfigurationen und Repositories. Datenbankmodelle werden mit `@Document` gekennzeichnet. Repositories basieren auf `MongoRepository` und bieten CRUD-Operationen, ohne zwingend manuell implementiert werden zu müssen. Abfragen können über Methodennamen abgeleitet oder mit `@Query` formuliert werden. [94], [95]

```

@Document
public class UserEntity {
    private UUID id;
    ...
}

@Repository
public interface RegistrationRepository extends MongoRepository<RegistrationEntity, UUID> {

    @Query("{ 'userEntity._id': ?0 }")
    Optional<RegistrationEntity> findByUserId(UUID userId);

    @Query("{ 'credentialsEntity.email': ?0 }")
    Optional<RegistrationEntity> findByCredentialsEntityEmail(String email);

}

```

Code 6: Datenbankzugriff mit Repository – Spring Boot

In allen drei Frameworks wird MongoDB durch eine annotationsbasierte Abstraktion eingebunden. Der Fokus liegt in jedem Fall auf Lesbarkeit, wenig Boilerplate-Code und einer unkomplizierten Verknüpfung mit MongoDB.

7.3. JWT und rollenbasierte Zugriffskontrolle

Die Frameworks unterstützen JWTs und eine damit umgesetzte rollenbasierte Zugriffskontrolle. Während das Prinzip in allen Fällen ähnlich ist, unterscheiden sich die Details in der technischen Umsetzung.

Im Quarkus-Prototyp wird JWT-Authentifizierung mit der MicroProfile JWT-Spezifikation realisiert. Die vorliegende entwickelte Klasse JwtUtil erzeugt hier Tokens mit der SmallRye JWT-API [96]:

```

@ApplicationScoped
public class JwtUtil {
    ...
    @Inject
    @ConfigProperty(name = "jwt.issuer")
    String jwtIssuer;

    public String buildJwtToken(RegistrationEntity registrationEntity) {
        ...
        groups.add(registrationEntity.getUserEntity().getRole());
        return Jwt
            .issuer(jwtIssuer)
            .upn(registrationEntity.getCredentialsEntity().getEmail())
            .groups(groups)
            .claim("sid", UUID.randomUUID()) //sessionId
            .claim(Claims.sub, registrationEntity.getUserEntity().id)
            ...
            .sign();
    }
}

```

Code 7: Erstellen eines JWT-Tokens in Quarkus

Zur Laufzeit können in den Serviceklassen, in der die Businesslogik definiert ist, Claims mit @Claim direkt injiziert werden:

```

@Inject
@Claim(standard = Claims.email)
String emailClaim;

@Inject
@Claim("sid")
String sessionIdClaim;

```

Code 8: Zugriff auf JWT-Claims in Quarkus

Die Zugriffskontrolle erfolgt deklarativ mit `@RolesAllowed`:

```
@Override
@RolesAllowed({"admin", "customer"})
public UpdateCredentialsResponse updateCredentials(UserUpdateCredentialsRequest
updateCredentialsRequest) {
    ...
}
@Override
@RolesAllowed({"admin"})
public SuccessResponse adminUpdateCredentials(UUID userId, AdminUpdateCredentialsRequest
updateCredentialsRequest) {

...
}
```

Code 9: Rollenbasierte Zugriffskontrolle mit Quarkus

Die hinterlegten Rollen werden beim Erstellen der Tokens als groups-Claim definiert. Quarkus kontrolliert diese Rollen automatisch für jede annotierte Methode. [96]

Micronaut nutzt für die JWT-Erstellung den eigenen TokenGenerator aus dem Micronaut-Security-Modul. Claims wie sub, roles oder sid werden beim Aufruf mittels Tokens übergeben. Die Absicherung der Endpunkte basiert auf einer Kombination aus `@Secured`- und `@RolesAllowed`-Annotationen. Die Klasse `JwtUtil` verwendet in diesem Fall den `SecurityService`, um Claims oder Rollen aus dem aktuellen JWT im Request-Context auszulesen. Bei jeder Anfrage prüft Micronaut automatisch, ob die Rolle im JWT mit der erlaubten Rolle übereinstimmt. [97]

```
@Override
@RolesAllowed({"admin"})
public SuccessResponse adminUpdateCredentials(UUID id, AdminUpdateCredentialsRequest
adminUpdateCredentialsRequest) {
    ...
}
@Singleton
public class JwtUtil {
    @Inject
    private TokenGenerator tokenGenerator;
    @Inject
    private SecurityService securityService;

    public JwtTokenContainer buildJwtToken(RegistrationEntity registrationEntity) {
        ...
        String jwtTokenAsString = tokenGenerator.generateToken(claims)
            .orElseThrow(() -> new RuntimeException("Failed to generate token"));
        return new JwtTokenContainer(jwtTokenAsString, claims);
    }
    public String getClaimFromSecurityContext(String claimName) {
        return securityService.getAuthentication()
            .map(auth -> auth.getAttributes().get(claimName))
            .map(Object::toString)
            .orElse(null);
    }
    public List<String> getRoles() {
        return securityService.getAuthentication()
            .map(auth -> auth.getAttributes().get("roles"))
            .filter(Objects::nonNull)
            .map(roles -> (List<String>) roles)
            .orElse(Collections.emptyList());
    }
}
```

Code 10: Zugriffskontrolle und JWT-Erstellung – Micronaut

Spring Boot verwendet für JWT die jjwt-Bibliothek (io.jsonwebtoken).

```
String jwtTokenAsString = Jwts.builder()
    .subject(registrationEntity.getCredentialsEntity().getEmail())
    .claim("roles", List.of(registrationEntity.getUserEntity().getRole()))
    .addClaims(claims)
    .expiration(new Date(System.currentTimeMillis() + EXPIRATION_TIME))
    .signWith(SignatureAlgorithm.HS512, SECRET)
    .compact();
return new JwtTokenContainer(jwtTokenAsString, claims);
```

Code 11: Erstellen eines JWT-Tokens – Spring Boot

Es muss ein eigener `JwtAuthenticationFilter` implementiert werden, welcher eingehende Anfragen prüft. Dieser liest den `Authorization-Header`, validiert den Token und extrahiert wichtige Claims. Diese werden anschließend im Spring Security Context verfügbar gemacht [98]:

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    ...
    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
    FilterChain filterChain)
        throws ServletException, IOException {
        String header = request.getHeader("Authorization");
        ...
        String token = header.substring(7);
        try {
            Claims claims = Jwts.parser().setSigningKey(SECRET).build().parseClaimsJws(token).getBody();

            String username = claims.getSubject();
            List<String> roles = claims.get("roles", List.class);

            List<SimpleGrantedAuthority> authorities = roles.stream()
                .map(SimpleGrantedAuthority::new)
                .collect(Collectors.toList());
            JwtAuthentication jwtAuthentication = new JwtAuthentication(username, authorities);

            jwtAuthentication.setAuthenticated(true);

            jwtAuthentication.setEmail((String)claims.get("email"));
            jwtAuthentication.setFirstName((String) claims.get("given_name"));
            jwtAuthentication.setLastName((String) claims.get("family_name"));
            jwtAuthentication.setSessionId(UUID.fromString((String)claims.get("sid")));
            jwtAuthentication.setUserId(UUID.fromString((String)claims.get("sub")));

            SecurityContextHolder.getContext().setAuthentication(jwtAuthentication);
        } catch (Exception e) {
            SecurityContextHolder.clearContext();
        }
        filterChain.doFilter(request, response);
    }
}
```

Code 12: JWT-Authentication Filter – Spring Boot

So stehen die Benutzerdaten inklusive der Rollen für den rollenbasierten Zugriffsschutz im Security Context zur Verfügung. Die Absicherung erfolgt anschließend über Annotationen wie `@PreAuthorize`. Spring Security wertet dabei automatisch die im JWT hinterlegten Rollen aus und entscheidet, ob Benutzer:innen Zugriff auf Endpoints bekommen. [98]

```
@Override
@PreAuthorize("hasAnyRole('ADMIN', 'CUSTOMER')")
public UpdateCredentialsResponse updateCredentials(UserUpdateCredentialsRequest
userUpdateCredentialsRequest) {
    ...
}
```

Code 13: Rollenbasierte Zugriffskontrolle – Spring Boot

Insgesamt war die Umsetzung in Spring Boot durch den zusätzlichen Boilerplate-Code, welcher in den anderen Frameworks erspart blieb, mit dem meisten Aufwand verbunden.

7.4. Messaging und Kafka Integration

Die Event-Kommunikation wird bei Quarkus mit dem Modul SmallRye Reactive Messaging gewährleistet. SmallRye ist eine Spezialisierung aus dem MicroProfile-Ökosystem und erweitert Quarkus um deklaratives Messaging mithilfe von Apache Kafka. Die Producer-Logik wird mit dem CDI (Contexts and Dependency Injection)-Annotationen-System (`@Inject`) und einem Emitter realisiert. [99]

```
@Inject
@Channel("user-registered-out")
Emitter<UserRegisteredEvent> userRegisteredEmitter;
...
userRegisteredEmitter.send(buildUserRegisteredEvent(...));
```

Code 14: Kafka-Integration – Event Emitter in Quarkus

Die Emitter-Instanz sendet Nachrichten an den Kafka Connector, der über die Quarkus-Messaging-API (`mp.messaging.*`) konfiguriert ist. Die Serialisierung der Nachrichten übernimmt dabei `ObjectMapperSerializer` (Jackson). Die Kafka-Anbindung wird von Quarkus verwaltet und die Verbindung, Serialisierung und die Integration in den CDI-Lifecycle erfolgt automatisch. Für Consumer verwendet Quarkus die Annotation `@Incoming`, welche eingehende Events einem Kafka-Topic zuordnet [99]:

```
@ApplicationScoped
public class MessageConsumer {
    ...
    @Incoming("user-updated-in")
    public void onUserUpdated(UserUpdatedEvent userUpdatedEvent) {
        //Verarbeiten der Nachricht
    }
}
```

Code 15: Kafka-Integration – Message Consumer in Quarkus

Die gesamte Konfiguration liegt in der `application.properties` und wird vom SmallRye-Connector aufgelöst. Hier können auch Umgebungsvariablen eingesetzt werden. [99], [100]

```
mp.messaging.outgoing.user-registered-out.connector=smallrye-kafka
mp.messaging.outgoing.user-registered-out.value.serializer=io.quarkus.kafka.client.serialization.ObjectMapperSerializer
mp.messaging.outgoing.user-registered-out.topic=user-registered
```

Code 16: Kafka-Konfiguration in Quarkus `application.properties`

Micronaut nutzt für Kafka das Micronaut Kafka-Modul. Der Producer wird über die Annotation `@KafkaClient` definiert. Diese deklariert ein Interface, das Micronaut automatisch zur Laufzeit als Kafka-Producer generiert: [101]

```
@KafkaClient(id = "user-credentials-updated-out")
public interface UserCredentialsUpdatedProducer {

    @Topic("user-credentials-updated")
    void send(UserCredentialsUpdatedEvent userCredentialsUpdatedEvent);
}
```

Code 17: KafkaClient mit Micronaut – Events senden

Die `KafkaClient`-API stützt sich dabei auf die `micronaut-serde`-Bibliothek für JSON-De- und Serialisierung. Für Consumer kommt die Annotation `@KafkaListener` zum Einsatz: [101]

```

@KafkaListener(groupId = "auth-events-group")
public class MessageConsumer {
    @Inject
    RegistrationRepository registrationRepository;
    ...
    @Topic("user-updated")
    public void onUserUpdated(UserUpdatedEvent userUpdatedEvent) {
        //Verarbeiten der Nachricht
    }

    @Topic("user-deleted")
    public void onUserDeleted(UUID id) {
        //Verarbeiten der Nachricht
    }
}

```

Code 18: KafkaListener mit Micronaut – Events empfangen

Micronaut verwaltet die Verbindung zum Kafka-Broker automatisch. Die Konfiguration der Topics, Consumer Gruppen und Bootstrap Server erfolgt zentral über das application.yaml. [101]

Spring Boot setzt für Kafka Messaging auf die Bibliothek spring-kafka. Für Producer stellt Spring-Kafka die Klasse KafkaTemplate bereit, die wie alle Spring Beans mit @Service injiziert wird: [102]

```

@Service
public class UserRegisteredEventProducer {

    private final KafkaTemplate<String, UserRegisteredEvent> kafkaTemplate;
    private final String topic = "user-registered";

    public UserRegisteredEventProducer(KafkaTemplate<String, UserRegisteredEvent>
kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void send(UserRegisteredEvent userRegisteredEvent) {
        kafkaTemplate.send(topic, userRegisteredEvent);
    }
}

```

Code 19: KafkaTemplate mit Spring Boot – Events senden

Spring-Kafka übernimmt dabei die Verwaltung der Kafka-Producer-Instanz, der Serialisierung über Jackson und der Verbindungsparameter in der application.properties Datei. Consumer werden mit der Annotation @KafkaListener deklariert: [102]

```

@KafkaListener(groupId = "auth-group", topics = "user-updated", containerFactory =
"userUpdatedFactory")
public void onUserUpdated(UserUpdatedEvent userUpdatedEvent) {
    System.out.println("Recieved user-updated-in event: " + userUpdatedEvent);
    RegistrationEntity registrationEntity =
registrationRepository.findByUserEntityId(userUpdatedEvent.getUserPayload().getId()).orElse(nu
ll);

    if(registrationEntity == null){
        return;
    }

    mapper.update(userUpdatedEvent.getUserPayload(), registrationEntity.getUserEntity());
    registrationRepository.save(registrationEntity);
}
...

```

Code 20: KafkaListener mit Spring Boot – Events empfangen

Die Verarbeitung wird durch eine ConcurrentKafkaListenerContainerFactory gesteuert und die Deserialisierung regelt Spring über den JsonDeserializer: [102]

```

@Configuration
public class MessageConsumersConfig {
    ...
    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, UserUpdatedEvent>
    userUpdatedFactory() {
        ConcurrentKafkaListenerContainerFactory<String, UserUpdatedEvent> factory = new
        ConcurrentKafkaListenerContainerFactory<>();
        var consumerFactory = generateFactory(new StringDeserializer(), new
        JsonSerializer<>(UserUpdatedEvent.class));
        factory.setConsumerFactory(consumerFactory);
        return factory;
    }
}

```

Code 21: Spring Boot Kafka-Konfiguration für Deserialisierung

Alle drei Lösungen abstrahieren die Kafka API weitestgehend. Entwickler:innen definieren Topics, Producer und Consumer einfach und deklarativ. Konnektoren, Serialisierung und Gruppen-Management übernimmt das jeweilige Framework.

7.5. Native-Build und GraalVM-Kompatibilität

Bei Quarkus wurden viele Komponenten wie Dependency Injection, REST-Schnittstellen oder Messaging bereits während des Builds analysiert. Es erstellt dabei alle nötigen Metadaten und baut sie ins Native-Image ein. Wo Reflection dennoch nötig sein sollte, wie zum Beispiel für JSON-Serialisierung mit Jackson oder bei Event-Klassen für Kafka, kann die Annotation `@RegisterForReflection` eingesetzt werden: [103]

```

@registerForReflection
public class UserPayload implements Serializable {

    private UUID id;
    private String firstName;
    ...
}

```

Code 22: Reflection-Hinweis für Quarkus mit `@RegisterForReflection`

Alternativ können Reflection-Hinweise auch über eine `reflect-config.json` definiert werden. Beim Einsatz von Entitäten mit Panache gab es keine Probleme mit dem nativen Build und funktionierte von Anfang an ohne weitere Anpassungen. Für Messaging gilt das Gleiche, denn alle Event-Klassen wurden im Rahmen des Builds richtig erkannt und es kam zu keinen Fehlern, auch nicht während der Laufzeit. Damit stellte sich Quarkus als eine sehr gut vorbereitete und unkomplizierte Plattform für native Java-Images heraus. [103]

Micronaut wurde von Anfang an mit dem Ziel entworfen, Reflection möglichst zu vermeiden und stattdessen auf Annotation Processing zur Compile-Time zu setzen [104]. Der native Build-Prozess erfolgte über das Micronaut Gradle Plugin mit dem AOT-Modul. Ein wichtiges Werkzeug ist die Annotation `@Serdeable`. Diese kennzeichnet Klassen, die serialisiert oder deserialisiert werden sollen (z. B. über REST oder Kafka). [105]

```

@Serdeable
public class AddressPayload implements Serializable {

    private String street;
    private String city;
}

```

Code 23: Reflection-Hinweis für Micronaut mit `@Serdeable`

Fehlte in der Umsetzung `@Serdeable`, musste Micronaut auf einen Jackson-Mapper zurückgreifen, was im Native-Image Laufzeitprobleme verursacht hat. Auch Datenbank-Entities (z. B. `RegistrationEntity`, `UserEntity`) mussten mit `@Serdeable` versehen werden.

Abgesehen davon waren die Anpassungen, um Native-Images mit Micronaut lauffähig zu machen, überschaubar.

Spring Boot ist eher Reflection-abhängig was im nativen Modus mit GraalVM zur Herausforderung wurde. Spring bietet zwar ein AOT-Processing, welches viele Beans, Proxies und Reflection-Zugriffe erkennt und vorbereitet. Doch bei dynamischen Elementen wie JWT-Bibliotheken (io.jsonwebtoken) oder Jackson mussten hier explizite Hints definiert werden. Dies erfolgte in der reflect-config.json: [106]

```
{
  "name": "io.jsonwebtoken.impl.DefaultJwtBuilder",
  "allDeclaredMethods": true,
  "allDeclaredConstructors": true
},
```

Code 24: Reflection-Hinweis für Spring Boot in reflect-config.json

Das resultierte in einem sehr langen Trial-and-Error Prozess, bei dem Fehler während der nativen Laufzeit analysiert und behoben werden mussten.

7.6. Hashing

In Quarkus kann Passwort-Hashing über das Elytron Security-Modul umgesetzt werden. Dieses bringt mit BcryptUtil, eine Bcrypt-Implementierung mit, die ohne zusätzliche externe Abhängigkeiten funktioniert. Die Hasher-Klasse des Prototyps ist als @ApplicationScoped-Bean deklariert. Quarkus verwaltet ihn einmalig für die gesamte Anwendungslebensdauer und stellt ihn per @Inject überall zur Verfügung, wo Hashing benötigt wird. [107]

```
@ApplicationScoped
public class Hasher {
    public String hash(String text) {
        return BcryptUtil.bcryptHash(text, 10);
    }
    public boolean check(String plainText, String hashedText) {
        return BcryptUtil.matches(plainText, hashedText);
    }
}
```

Code 25: Bcrypt-Hashing mit Quarkus

Micronaut nutzt für das Passwort-Hashing den BCryptPasswordEncoder aus dem Spring-Security-Paket. Der Encoder wird als @Singleton-Bean verwaltet und kann überall per Dependency Injection genutzt werden. [108] Die Implementierung wird anhand des folgenden Beispiels veranschaulicht:

```
@Singleton
public class Hasher {
    private PasswordEncoder passwordEncoder;

    public String hash(String text) {
        passwordEncoder = new BCryptPasswordEncoder(10);
        return passwordEncoder.encode(text);
    }
    public boolean check(String plainText, String hashedText) {
        passwordEncoder = new BCryptPasswordEncoder(10);
        return passwordEncoder.matches(plainText, hashedText);
    }
}
```

Code 26: Bcrypt-Hashing mit Micronaut

In Spring Boot wird Hashing über Spring Security Crypto realisiert. Der Hasher wird als Spring Bean (@Component) deklariert und nutzt intern ebenfalls den BCryptPasswordEncoder. [109]


```

@Component
public class Hasher {
    private PasswordEncoder passwordEncoder;

    public String hash(String text) {
        passwordEncoder = new BCryptPasswordEncoder(10);
        return passwordEncoder.encode(text);
    }
    public boolean check(String plainText, String hashedText) {
        passwordEncoder = new BCryptPasswordEncoder(10);
        return passwordEncoder.matches(plainText, hashedText);
    }
}

```

Code 27: Bcrypt-Hashing mit Spring Boot

Spring kümmert sich um die Verwaltung der Bean und stellt sicher, dass der Encoder in allen relevanten Serviceklassen verfügbar ist.

7.7. Vergleichsfazit und Learnings

Die parallele Entwicklung der drei Prototypen hat gezeigt, dass die theoretischen Versprechen der Frameworks in der Praxis dann ausgeschöpft werden können, wenn ihre Eigenheiten beachtet werden, vor allem bei nativer Kompilierung mit GraalVM.

Quarkus zeigte sich in der Praxis in den meisten Aspekten als besonders unkompliziert. Viele Kernfunktionen wie Dependency Injection, REST APIs, Panache für MongoDB oder SmallRye Reactive Messaging sind so entworfen, dass sie standardmäßig mit GraalVMs AOT-Kompilierung kompatibel waren. Positiv hervorzuheben ist die gut strukturierte Dokumentation. Zu fast jeder Angelegenheit gab es auf der offiziellen Quarkus-Seite diverse Anleitungen mit konkreten Beispielen. Das hat mehrfach dabei geholfen, Probleme schnell zu lösen. Positiv war auch der Build-Prozess selbst. Ein nativer Build ließ sich mit `mvn package -Pnative` oder per Container Build vergleichsweise unkompliziert erstellen.

Micronaut erwies sich im Prototyp ebenfalls sehr entwicklerfreundlich, was zu einer nahezu reibungslosen Umsetzung führte. Die einzige Ausnahme war die Suche nach den richtigen Stellen, welche mit `@Serdeable`-Annotationen zu versehen waren. Laufzeitfehler traten ausschließlich beim JSON-Mapping auf, sodass die Fehlersuche relativ schnell abgeschlossen werden konnte. Allerdings ist die Dokumentation in manchen Punkten der von Quarkus unterlegen, sodass einige Lösungen durch Analyse des Source Codes oder Recherchen in Internetforen gefunden werden mussten.

Spring Boot war im Vergleich der flexibelste, zugleich jedoch auch der aufwendigste Ansatz, vor allem im Hinblick auf GraalVM und der Umsetzung von JWTs mit rollenbasierter Zugriffskontrolle. Im klassischen JVM-Betrieb funktionierte Spring Boot problemlos, denn die Autoconfiguration, dynamische Bean-Erzeugung, Reflection und Proxying deckten alle Anwendungsfälle ab. Im Native-Build zeigte sich jedoch, dass alles was Spring Boot dynamisch zur Laufzeit macht, für GraalVM statisch vorbereitet werden muss. Das betraf bei der Umsetzung des E-Shop-Prototypen vor allem den JWT-Parser, Jackson und dynamische Proxy Beans wie den `JwtAuthenticationFilter`. Der größte Aufwand entstand beim Erstellen von vollständigen `reflect-config.json` Dateien, die mehrfach manuell angepasst werden mussten. Insgesamt erwies sich die Umsetzung dadurch als deutlich aufwendiger als bei Quarkus oder Micronaut.

8. Benchmarking der Prototypen

Dieses Kapitel behandelt das Benchmarking der entwickelten Prototypen. Beschrieben werden der Aufbau der Testumgebung, die verwendeten Methoden und Tools für die Messungen sowie die Durchführung der Messungen hinsichtlich Startzeiten, Speicherverbrauch, CPU-Auslastung und Verarbeitungszeiten. Die Ergebnisse werden anschließend dargestellt und analysiert.

8.1. Testumgebung

Zur Evaluierung und zum Vergleich der Ressourceneffizienz der implementierten Microservice-Architektur wurden alle Messungen in einer lokalen Testumgebung durchgeführt. Die Tests erfolgten auf einem Entwicklungsrechner mit folgenden Hardware-Spezifikationen: AMD Ryzen 7 5800X 8-Core Prozessor mit einer Grundtaktfrequenz von 3,80 GHz, 32 GB Arbeitsspeicher und ein Windows 10 64-Bit-Betriebssystem auf Basis einer x64-Architektur. Alle Microservices wurden ohne Virtualisierung oder Containerisierung ausgeführt, um den Konfigurationsaufwand in der Testumgebung möglichst gering zu halten. Während der Messungen wurden keine weiteren Anwendungen gestartet und automatische Updates sowie jegliche Hintergrundprozesse waren deaktiviert. Die Java-Version bei JVM-Tests war *"21.0.5" 2024-10-15 LTS, Java(TM) SE Runtime Environment (build 21.0.5+9-LTS-239), Java HotSpot(TM) 64-Bit Server VM (build 21.0.5+9-LTS-239, mixed mode, sharing)*. Bei GraalVM wurde beim Kompilieren von nativen Images folgende Version eingesetzt: *java version "21.0.5" 2024-10-15 LTS, Java(TM) SE Runtime Environment Oracle GraalVM 21.0.5+9.1 (build 21.0.5+9-LTS-jvmci-23.1-b48), Java HotSpot(TM) 64-Bit Server VM Oracle GraalVM 21.0.5+9.1 (build 21.0.5+9-LTS-jvmci-23.1-b48, mixed mode, sharing)*.

8.2. Verwendete Tools

Für die Durchführung der Last- und Performancetests wurde die Open-Source-Software Grafana k6 eingesetzt. Mit diesem Framework lassen sich realistische Benutzerinteraktionen in JavaScript definieren und automatisiert ausführen. [110]

Die Messung und Protokollierung von relevanten Systemressourcen erfolgte parallel durch PowerShell-Skripte, die alle laufenden Microservices in festen Zeitintervallen überwachten. Die gesammelten Rohdaten wurden in CSV-Dateien (Comma-Separated Values) gespeichert und anschließend mit Python ausgewertet und visualisiert. Auf diese Weise lassen sich sowohl Einzelwerte als auch Trends im Durchschnitt über die gesamte Testdauer hinweg nachvollziehen.

8.3. Durchführung der Messungen

In diesem Abschnitt wird beschrieben, wie die jeweiligen Metriken gemessen wurden.

8.3.1. CPU-Effizienz und RAM-Verbrauch

In dieser Arbeit ist die zentrale Kennzahl zur Erfassung der CPU-Nutzung die vom Betriebssystem bereitgestellte „CPU-Time“. Die gemessene und angesammelte Zeit ermöglicht eine Quantifizierung des Ressourcenbedarfs der Prototypen. Im Rahmen der Messung wurde dieser Wert in festen Zwei-Sekunden-Intervallen aufgezeichnet, wodurch der Ressourcenverbrauch während der gesamten Testdauer nachvollziehbar ist.

Zusätzlich wurde für jeden Service die Speichernutzung abgefragt und erfasst. Diese Metrik stellt sicher, dass ein genaues Bild über den RAM-Bedarf der Services im Durchschnitt sowie in Lastspitzen erkannt werden kann.

Die Durchführung dieser Messungen erfolgte technologieübergreifend. Um eine faire Vergleichbarkeit zu ermöglichen, wurde der gleiche Aufbau identisch für alle Prototyp-Varianten umgesetzt. Konkret bedeutet dies, dass der gesamte Messprozess sowohl für Quarkus, Spring Boot als auch Micronaut jeweils in der JVM-Variante und als Native-Images ausgeführt wurde. Um Schwankungen zu minimieren, wurde die Anzahl der Durchläufe stetig erhöht bis sich die Mittelwerte zwischen den Messreihen nur noch geringfügig veränderten. Das Erkennen von klaren Trends war ein weiteres Kriterium für das Beenden der Messungen. Schlussendlich wurden für jede dieser Ausprägungen 100 vollständige Testdurchläufe unter den gleichen Rahmenbedingungen durchgeführt. Abbildung 11 zeigt eine Übersicht des Benchmark-Ablaufs.

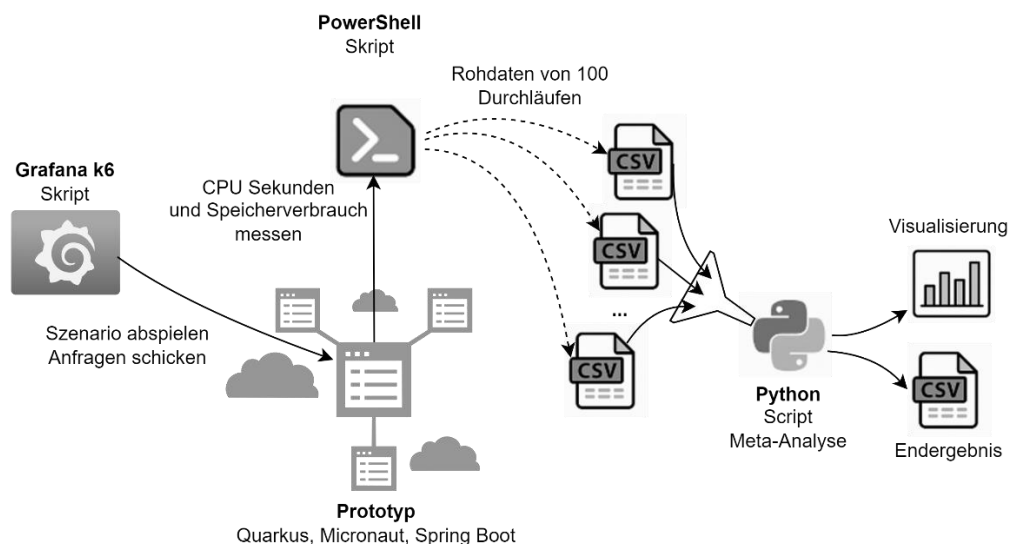


Abb. 11: Aufbau der Testumgebung

Im Last-Test kam ein Szenario zum Einsatz, das mit Grafana k6 vorbereitet wurde. Dieses simulierte mit einer fixierten Frequenz von Anfragen (20 Anfragen pro Sekunde über 20 Sekunden) typische Nutzerinteraktionen in der Anwendung. Diese umfasst auch die Registrierung von Benutzer:innen, den Login, das Bedienen des Warenkorbs und den Checkout. Damit ist gewährleistet, dass alle Services der Architektur belastet werden.

Die erzeugten Messdaten wurden in CSV-Dateien strukturiert und anschließend mit Python-Skripten weiterverarbeitet. Neben der Aufbereitung der Einzelwerte wurde im Rahmen einer Meta-Analyse aus allen Durchläufen ein gesammeltes Gesamtbild erzeugt. Dabei wurden für jeden Service Kennzahlen wie Mittelwert, Standardabweichung, Minimum und Maximum der Gesamt-CPU-Zeit sowie des durchschnittlichen Arbeitsspeicherverbrauchs berechnet. Die Visualisierung der Ergebnisse erfolgte in Form von Balkendiagrammen.

Insgesamt ermöglicht dieser Ansatz eine reproduzierbare Bewertung der CPU-Effizienz und den RAM-Verbrauch der einzelnen Prototypen. Im Weiteren liefert er eine Entscheidungsgrundlage für die Auswahl der performantesten Plattform. Der Ansatz zeigt,

welche Konfigurationen Vorteile bieten und welche Services gegebenenfalls optimiert werden können.

8.3.2. Startzeit

Ergänzend zur Laufzeitmessung von CPU- und Arbeitsspeicherauslastung wurde im Rahmen der Tests auch die Startzeit der einzelnen Microservices erhoben. Ziel dieser Messung ist es die Startzeiten der unterschiedlichen Ausprägungen vergleichbar zu machen.

Die Zeit wurde dabei manuell ermittelt, da sich der Vorgang aufgrund der Konsolenausgabe der Frameworks unkompliziert durchführen ließ. Für jede Service-Instanz wurde der Startvorgang analog zu den CPU- und RAM-Messungen 100-mal wiederholt. Dabei wurde jeweils die benötigte Zeit bis zur vollständigen Bereitschaft anhand der Meldung in der Konsole abgelesen. Die Werte wurden protokolliert und zur späteren Analyse in Millisekunden umgerechnet. Um die Vergleichbarkeit zu fördern, wurde jeder Service nach jedem Startvorgang vollständig beendet und anschließend erneut gestartet.

Die Einzelwerte wurden ebenfalls in CSV-Dateien abgelegt und mit Python ausgewertet. Damit wurde in einer gesammelten Analyse jeweils der Durchschnitt der Prototypen berechnet. Die Ergebnisse wurden in Form von gruppierten Balkendiagrammen visualisiert, welche den direkten Vergleich zwischen JVM-basierten und nativen Varianten innerhalb der Frameworks ermöglichen.

8.3.3. Verarbeitungszeit

Anschließend wurde die Verarbeitungszeit in den implementierten Microservices erhoben. Diese beschreibt die Zeitspanne die ein fachlicher Prozess, in diesem Fall der Kauf eines Produkts, vom Eintreffen der initialen Benutzeranfrage bis zum erfolgreichen Abschluss aller Verarbeitungsschritte benötigt. Diese Metrik bietet eine zusätzliche Basis zur Beurteilung der Leistungsfähigkeit und Reaktionsfähigkeit der Microservice-Architektur.

Die Messung erfolgte dabei durch den Einsatz des bereits diskutierten Tracking Services. Jeder Schritt des Kaufprozesses löst ein Event aus, das mit einer eindeutigen Korrelations-ID versehen und mit einem Zeitstempel in der Datenbank des Tracking Services abgelegt wird. Auf diese Weise kann sowohl die Gesamtdauer des Prozesses als auch die Dauer von einzelnen Prozessschritten bestimmt werden. Für die nachfolgende Analyse wurden die erfassten Rohdaten in CSV-Dateien exportiert und mit Python ausgewertet. Die Prozessdaten wurden zunächst nach Korrelations-ID gruppiert, nach Zeitstempel sortiert und anschließend wurde die Differenz zwischen dem ersten und letzten Event eines Prozesses berechnet. Zusätzlich wurden aus den Zeitstempeln die Zeitspannen zwischen ausgewählten Events abgeleitet wie zum Beispiel „Checkout Started → Order Created“, „Order Created → Payment Success“, „Payment Success → Licenses Generated“ und „Licenses Generated → Order Notification Sent“.

Hier wurde die Messung ebenfalls in identischer Form für alle implementierten Prototypen durchgeführt, sowohl für die JVM-basierten Implementierungen als auch für die GraalVM-basierten Native-Image Varianten. Für jede Konfiguration wurden dabei ebenfalls 100 Kaufprozesse innerhalb eines 20-sekündigen Zeitraums simuliert. Das entspricht einer gleichmäßigen Rate von 5 Kaufprozessen pro Sekunde. Die Ergebnisse sind sowohl

tabellarisch als auch grafisch aufbereitet. Die Gesamtprozessdauer und die Dauer der einzelnen Teilschritte wurden in Balkendiagrammen visualisiert.

8.4. Ergebnisse

Im Folgenden werden die Ergebnisse dargestellt und zur Beantwortung der Hauptforschungsfrage herangezogen.

8.4.1. CPU-Effizienz und RAM-Verbrauch

Framework	SVC	CPU Durchschn.	CPU Min.	CPU Max.	CPU Stdabw.	Mem. Durchschn.	Mem. Min.	Mem. Max.	Mem Stdabw.
Micronaut JVM	Auth	49.3	47.33	54.44	2.36	287.86	270.65	321.53	14.51
Micronaut Native	Auth	45.57	44.83	46.2	0.43	105.89	99.54	111.45	4.26
Quarkus JVM	Auth	61.51	57.45	69.28	3.44	252.08	227.78	276.34	15.3
Quarkus Native	Auth	56.42	55.16	57.72	0.79	76.19	56.95	96.75	16.32
SpringBoot JVM	Auth	50.09	47.42	54.62	2.21	359.98	334.59	369.4	10.4
SpringBoot Native	Auth	45.72	44.98	46.38	0.46	129.29	117.51	137.5	7.87
Micronaut JVM	Andere	2.7	1.02	6.08	1.7	327.53	297.19	358.76	23.18
Micronaut Native	Andere	1.14	0.84	1.49	0.22	97.41	87.98	109.28	7.78
Quarkus JVM	Andere	3.68	1.19	9.11	2.51	249.31	226.05	267.37	14.56
Quarkus Native	Andere	1.36	1.07	1.73	0.22	84.41	72.62	97.64	9.33
SpringBoot JVM	Andere	2.85	1.33	6.15	1.55	360.55	329.54	375.17	15.64
SpringBoot Native	Andere	1.44	1.2	1.75	0.18	125.66	114.4	134.67	7.23

Tab. 2: Überblick der Messergebnisse von CPU-Zeit in s und RAM-Verbrauch in MB. Durchschnitts-, Minimum-, Maximum-Werte und Standardabweichung in s und MB angegeben

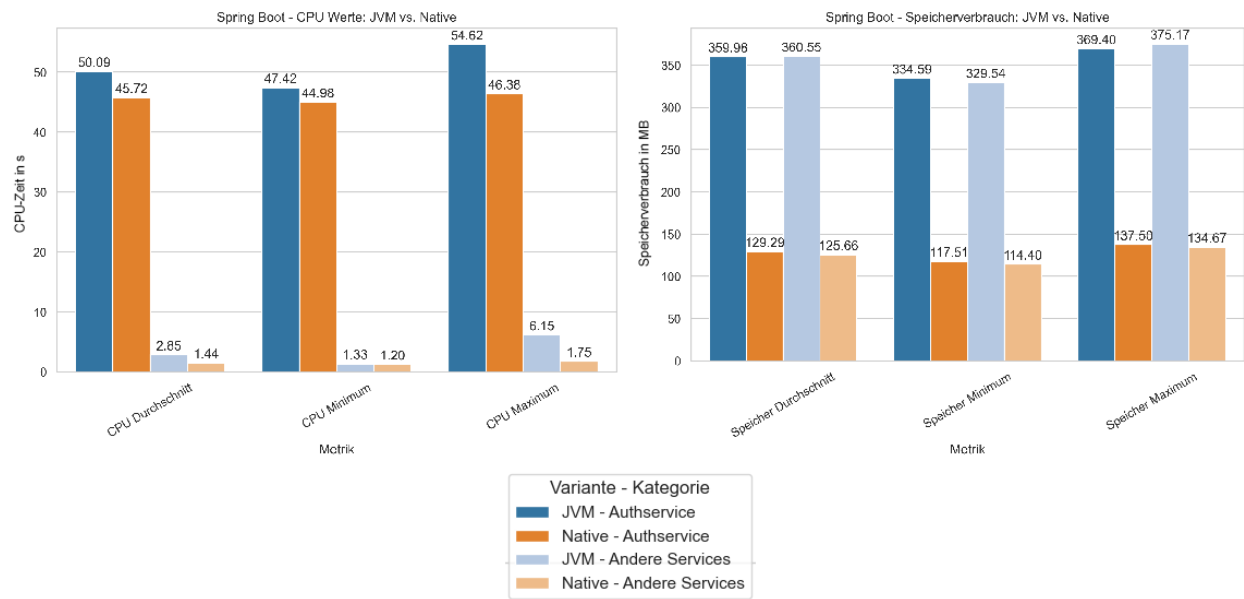


Abb. 12: Spring Boot Ergebnisse – CPU und RAM

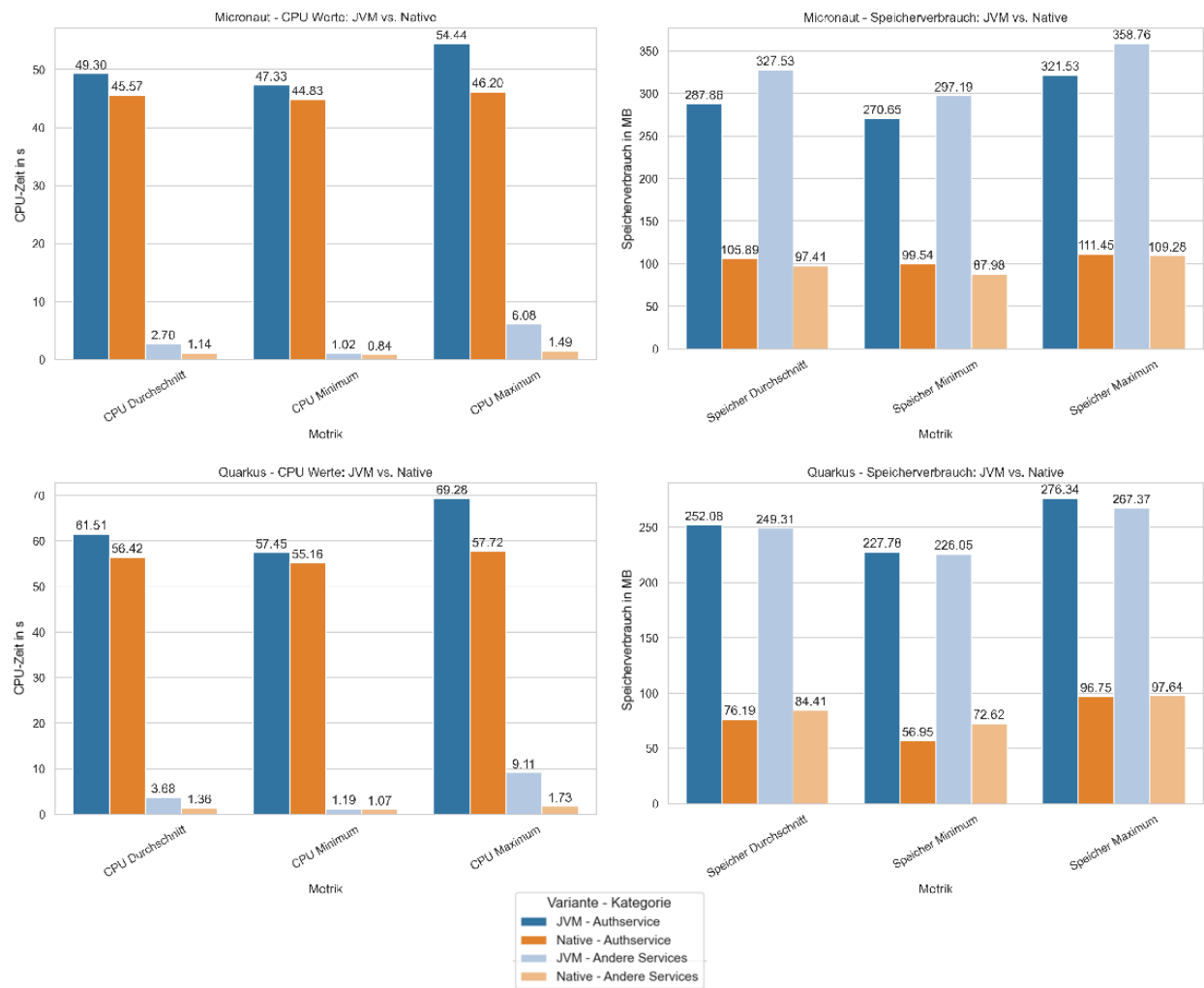


Abb. 13: Micronaut und Quarkus Ergebnisse – CPU und RAM

Die Messung der CPU-Effizienz und des Speicherverbrauchs ermöglichen eine Teilbeantwortung der Hauptforschungsfrage. Die nachfolgende Diskussion bezieht sich auf die in Tabelle 2 und Abbildungen 12 und 13 ersichtlichen Ergebnisse. Insgesamt zeigen alle drei Frameworks in der JVM-Umgebung einen deutlich höheren Ressourcenverbrauch. Dies gilt sowohl für den durchschnittlichen Speicherverbrauch als auch für die CPU-Zeit. Der Authservice, welcher im Testszenario die meiste Rechenarbeit leistete, weist im Durchschnitt CPU-Zeiten von 49 s (Micronaut), 50 s (Spring Boot) und 61 s (Quarkus) auf. Damit erreicht Quarkus den höchsten Rechenzeitbedarf, während Micronaut den niedrigsten Wert liefert.

Beim Speicherverbrauch verhält es sich ähnlich: Micronaut erreicht im Schnitt etwas geringere Werte als Spring Boot und Quarkus. Die RAM-Werte liegen je nach Framework bei etwa 200–400 MB pro Serviceinstanz.

Durch den Einsatz von GraalVM Native-Image sinken Speicherverbrauch und CPU-Zeit messbar. Für den Authservice lassen sich deutliche Einsparungen erkennen. Micronaut Native und Spring Boot Native erreichen hier Werte um die 45 s, während Quarkus Native mit ca. 56 s den letzten Platz belegt. Dies bedeutet im Schnitt eine Einsparung der CPU-Arbeitszeit um rund 7-10 % im Vergleich zu den jeweiligen JVM-Varianten.

Der Vergleich beim Arbeitsspeicher fällt etwas deutlicher aus. Während der Micronaut Authservice in der JVM-Variante durchschnittlich rund 288 MB belegt, reduziert sich dieser Wert durch die native Kompilierung auf etwa 106 MB. Das entspricht einer Ersparnis von ca. 63 %. Bei Spring Boot sinkt der Speicherbedarf des Authservice von ca. 360 MB auf rund 129 MB, was einer Reduktion von 64 % entspricht. Quarkus Native erzielt beim Authservice einen vergleichsweise niedrigen Wert von ca. 76 MB und liegt damit fast 70 % unter dem JVM-Mittelwert von etwa 252 MB. Die anderen Services können hier den Wertebereich von 200 bis 400 MB mit nativen Images teilweise auf unter 100 bis 130 MB reduzieren, was einer Einsparung von bis zu 70 % entspricht.

Auch bei den restlichen Services („Andere“) lassen sich im Schnitt klare Verbesserungen erkennen. Hier liegen die durchschnittlichen CPU-Zeiten jedoch deutlich niedriger als beim Authservice im Bereich von ca. 2-4 s in der JVM-Variante und um ca. 1-1,5 s in der nativen Variante. Dabei zeigte sich beim Arbeitsspeicher ein ähnliches Muster: Die JVM-Versionen bewegten sich im Bereich von ca. 250-360 MB pro Instanz, während die nativen Builds diesen Wert teilweise auf unter 100 MB senken. Die Trennung zwischen Authservice und den restlichen Services wurde zur besseren Übersichtlichkeit vorgenommen, da der Authservice im gegebenen Testszenario signifikant höhere CPU-Zeitwerte aufwies.

Die erreichten Verbesserungen zeigen, dass die nativen Varianten, unabhängig vom gewählten Framework, durch den Einsatz von GraalVM Native-Image einen wesentlichen Beitrag zur Ressourcenoptimierung leisten können. Die JVM-Varianten sind generell ressourcenintensiver, sowohl beim Arbeitsspeicher als auch bei der CPU-Zeit und damit in dieser Microservice-Lösung weniger geeignet. Bei den nativen Varianten wird die CPU-Zeit je nach Framework um bis zu 10 % reduziert, der Arbeitsspeicherbedarf sinkt um 60–70 %. Micronaut Native liefert in Summe die beste Bilanz, da es den CPU-Sekunden-Wert des Authservice am stärksten reduziert und gleichzeitig niedrige RAM-Werte erreicht. Quarkus Native hebt sich vor allem durch seine Speichereffizienz hervor, zeigt aber eine geringere

Leistung beim CPU-Zeitverbrauch. Spring Boot Native liegt in beiden Aspekten in der Mitte, profitiert aber trotzdem von der nativen Kompilierung.

Insgesamt zeigen die Ergebnisse, dass die Nutzung von GraalVM Native Image für alle drei Frameworks einen wichtigen Beitrag zu einer ressourcenschonenden Umsetzung choreografiebasierter E-Shop-Anwendungen leisten kann.

8.4.2. Startzeit

Variante	Durchschn.	Minimum	Maximum	Stdabw.
Micronaut JVM	1495.52	1147	1826	127.54
Micronaut Native	155.35	59	579	116.31
Quarkus JVM	1487.56	1003	1704	159.43
Quarkus Native	53.94	27	93	13.87
SpringBoot JVM	3999.10	2188	6620	618.37
SpringBoot Native	238.03	69	608	118.71

Tab. 3: Ergebnis der durchschnittlichen Startzeit in ms je nach Framework und Variante. Zusätzlich sind Minimum, Maximum und Standardabweichung in ms angegeben.

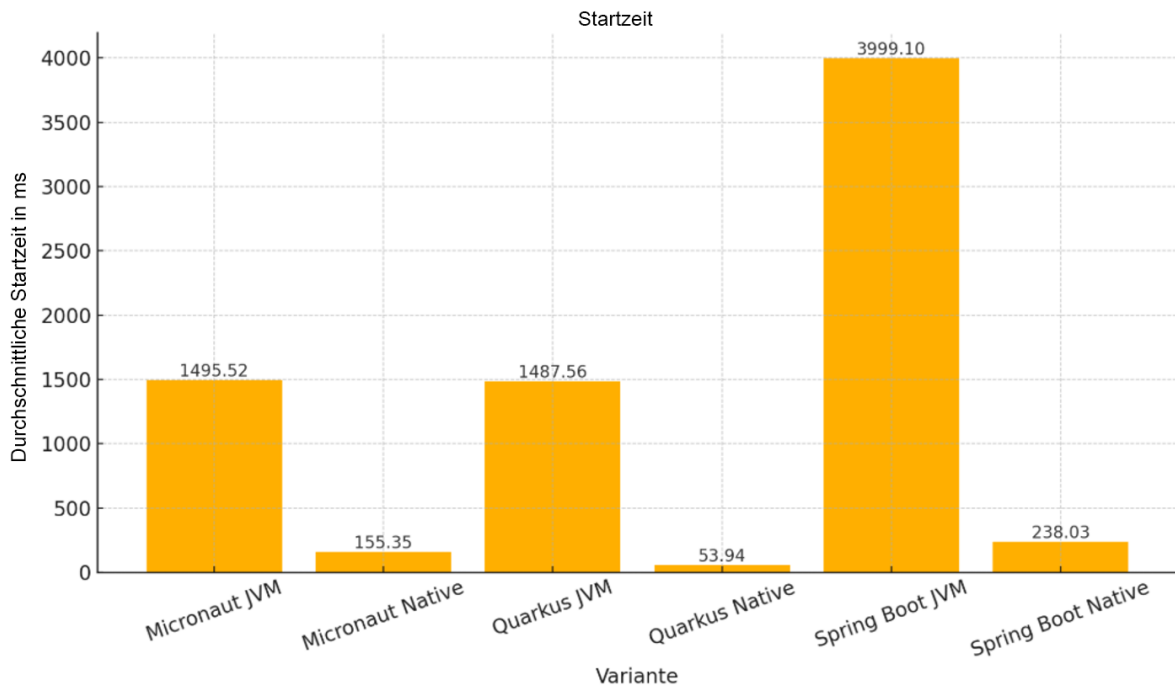


Abb. 14: Durchschnittliche Startzeit in ms – Ergebnis je nach Framework und Variante

Dieser Abschnitt soll die Hauptforschungsfrage bezogen auf Startzeit beantworten. Zunächst zeigen die Ergebnisse in Tabelle 3 und Abbildung 14 folgendes Muster: Die JVM-basierten Varianten aller drei Frameworks benötigen im Durchschnitt längere Startzeiten als ihre nativen Gegenstücke. Besonders auffällig ist hier Spring Boot, das in der JVM-Variante durchschnittlich ca. 4000 ms zum Starten einer Service-Instanz benötigt. Damit liegt Spring Boot weit über Micronaut mit ~ 1495 ms und Quarkus mit ~ 1487 ms im JVM-Betrieb.

Der Übergang zu GraalVM Native-Image bringt in allen drei Fällen Verbesserungen. Bei Micronaut reduziert sich die mittlere Startzeit um ca. 90 %, von knapp 1500 ms auf etwa 155 ms. Bei Quarkus fällt die Startzeit von rund 1487 ms im JVM-Betrieb auf etwa 54 ms, Esad Hrbatović

das ist eine Verbesserung von ca. 96 %. Spring Boot, dessen JVM-Startzeit deutlich über den anderen liegt, erreicht durch native Kompilierung eine Reduktion von rund 4000 ms auf ca. 238 ms, was einer Verbesserung von etwa 94 % entspricht.

Zusammenfassend zeigt sich, dass der Einsatz nativer Images für choreografiebasierte Microservice-Architekturen einen Vorteil für den Betrieb mit sich bringt. Quarkus als Native-Image erweist sich bei dieser Messung als effizienteste Lösung, während Micronaut ebenfalls eine vergleichbar kurze Startzeit erreicht. Spring Boot Native realisiert auch eine deutliche Verbesserung als Native-Image, obwohl es hier insgesamt am schwächsten abschneidet. Ableitend lässt sich festhalten, dass AOT-Kompilierung nicht nur Speicher- und CPU-Effizienz verbessert, sondern auch Startzeiten verkürzt.

8.4.3. Verarbeitungszeit

Variante	Durchschn.	Median	Minimum	Maximum	Stdabw.
micronaut_jvm	19.58	19.0	16.0	39.0	2.90
micronaut_native	12.77	12.0	11.0	31.0	2.26
quarkus_jvm	24.27	22.0	2.0	56.0	6.62
quarkus_native	18.71	17.0	15.0	154.0	7.69
springboot_jvm	22.77	21.0	18.0	61.0	6.54
springboot_native	13.40	13.0	11.0	33.0	2.40

Tab. 4: Ergebnis der Verarbeitungszeiten von Einkaufsprozessen, Durchschnitt, Median, Minimum-, Maximum-Wert und Standardabweichung in ms

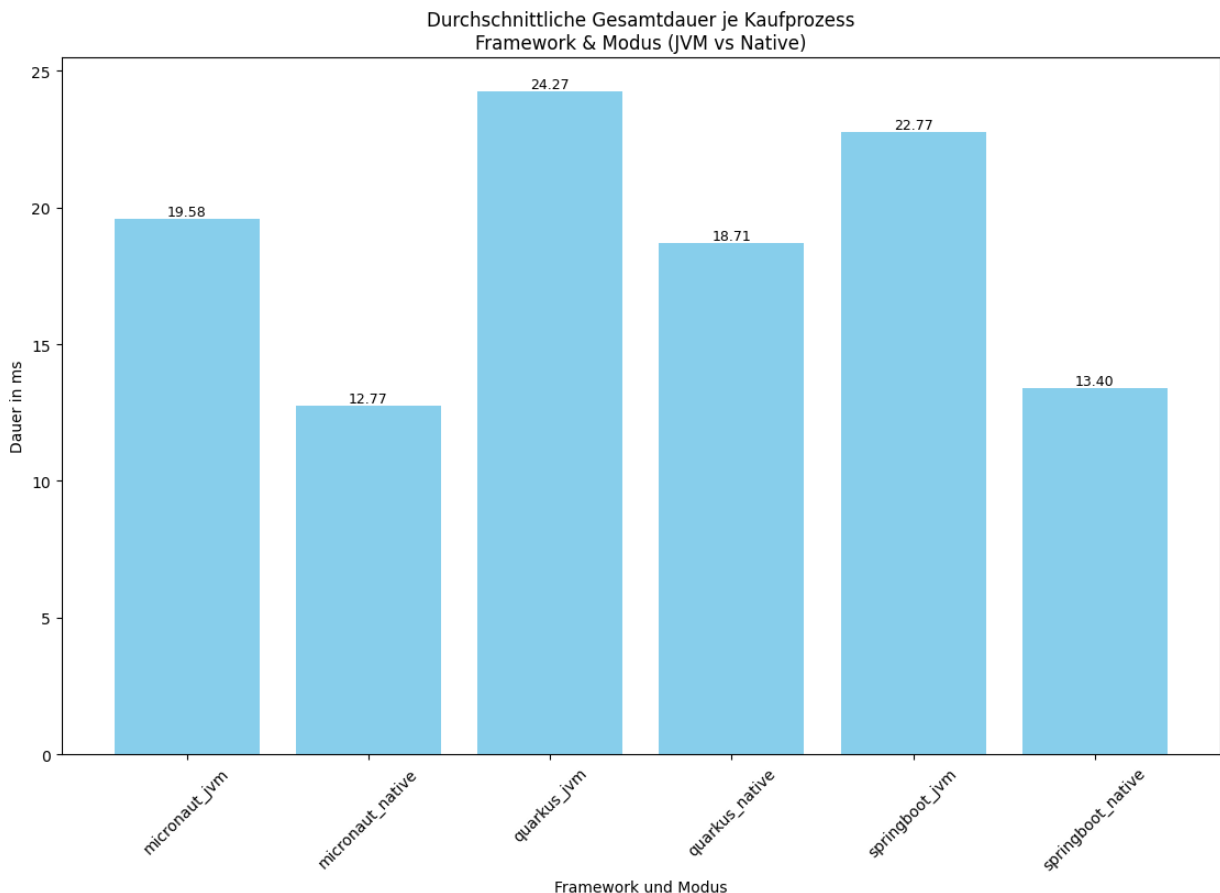


Abb. 15: Durchschnittliche Gesamtdauer von Kaufprozessen in ms - je nach Framework und Variante

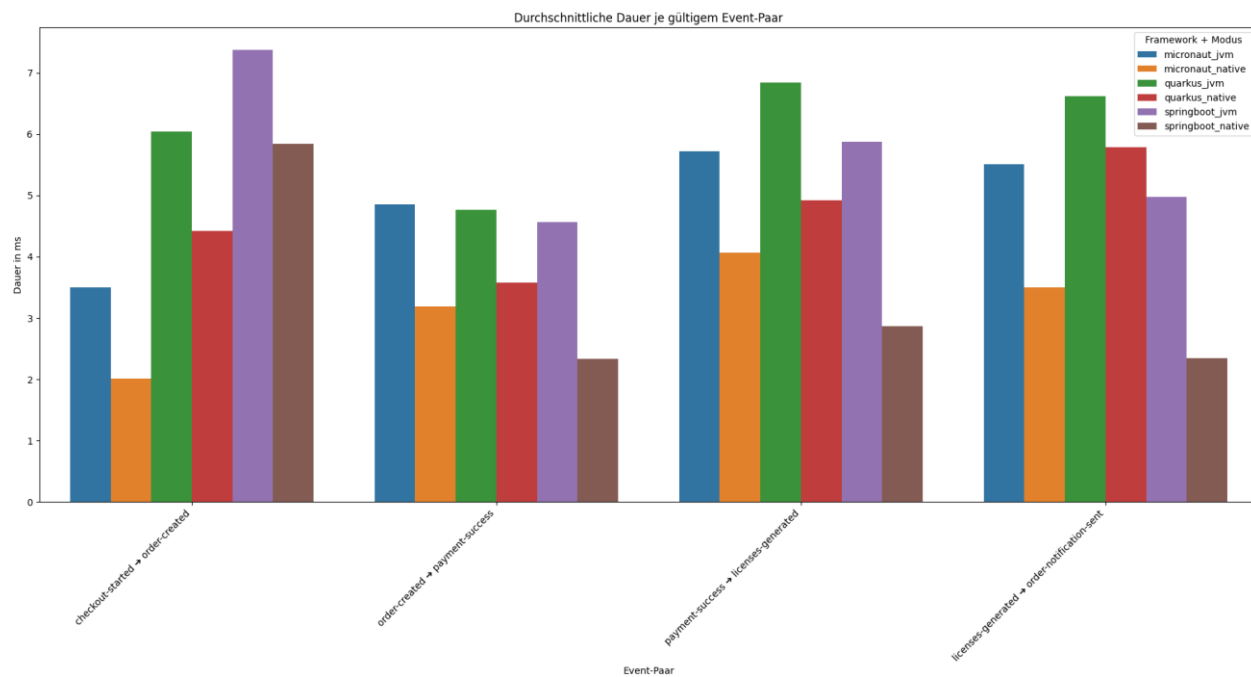


Abb. 16: Ergebnis der durchschnittlichen Dauer in ms von Prozess-Schritten je nach Framework und Variante

Variante	Schritt	Durchschn.	Std. Abw.
micronaut_jvm	checkout-started -> order-created	3.5	0.9
micronaut_jvm	licenses-generated -> order-notification-sent	5.5	1.0
micronaut_jvm	order-created -> payment-success	4.8	0.9
micronaut_jvm	payment-success -> licenses-generated	5.7	1.0
micronaut_native	checkout-started -> order-created	2.0	0.7
micronaut_native	licenses-generated -> order-notification-sent	3.5	1.0
micronaut_native	order-created -> payment-success	3.2	0.4
micronaut_native	payment-success -> licenses-generated	4.1	0.9
quarkus_jvm	checkout-started -> order-created	6.0	2.4
quarkus_jvm	licenses-generated -> order-notification-sent	6.6	2.3
quarkus_jvm	order-created -> payment-success	4.8	0.7
quarkus_jvm	payment-success -> licenses-generated	6.8	2.5
quarkus_native	checkout-started -> order-created	4.4	0.9
quarkus_native	licenses-generated -> order-notification-sent	5.8	1.8
quarkus_native	order-created -> payment-success	3.6	0.8
quarkus_native	payment-success -> licenses-generated	4.9	0.9
springboot_jvm	checkout-started -> order-created	7.4	2.8
springboot_jvm	licenses-generated -> order-notification-sent	5.0	2.1
springboot_jvm	order-created -> payment-success	4.6	1.0
springboot_jvm	payment-success -> licenses-generated	5.9	1.8
springboot_native	checkout-started -> order-created	5.8	1.1
springboot_native	licenses-generated -> order-notification-sent	2.4	0.8
springboot_native	order-created -> payment-success	2.3	0.5
springboot_native	payment-success -> licenses-generated	2.9	0.9

Tab. 5: Ergebnis der durchschnittlichen Zwischenschritt-Dauer in ms samt Standardabweichung

Die Analyse der gemessenen Verarbeitungszeiten in Abbildungen 15 und 16 sowie Tabellen 4 und 5 liefert die Grundlage zur Teilbeantwortung der Frage, wie sich die Verarbeitungszeiten der Frameworks für Bestellvorgänge unterscheiden, sowohl in JVM-Umgebungen als auch in Form von GraalVM Native-Images.

Diese Kennzahl ergibt sich durch die Gesamtdauer eines Bestellvorgangs und durch die einzelnen Schritte, gemessen an den Übergängen zwischen den Teiloperationen. Die Ergebnisse zeigen für die JVM-basierten Varianten folgendes: Micronaut JVM erreicht mit durchschnittlich ~ 19,6 ms die geringste Verarbeitungszeit, gefolgt von Spring Boot JVM mit ~ 22,8 ms und Quarkus JVM liegt mit ~ 24,3 ms an letzter Stelle. In den Messungen sind die Zeiten für Teilschritte bei Spring Boot und Micronaut vergleichbar gering, während diese bei Quarkus meistens darüber liegen.

Durch den Einsatz nativer Images werden bei allen Frameworks Verbesserungen erzielt. Micronaut Native reduziert die Verarbeitungszeit im Schnitt auf ~ 12,8 ms, was einer Verringerung um rund 35 % gegenüber der JVM-Variante entspricht. Quarkus Native verbessert sich auf ~ 18,7 ms und erreicht eine Verbesserung von ca. 23 %. Spring Boot Native erreicht im Schnitt ~ 13,4 ms und zeigt damit eine Reduktion von rund 41 % im Vergleich zur JVM-Variante. Auch bei den Teilschritten sind die nativen Varianten aller

Frameworks durchgängig schneller als die JVM-Varianten. Micronaut Native zeigt mit niedrigen Einzelzeiten und geringer Standardabweichung insgesamt die beste Performance.

Diese Ergebnisse deuten darauf hin, dass Micronaut Native in Bezug auf Verarbeitungszeit, derzeit den effizientesten Betrieb für eine choreografiebasierte Microservice-Architektur bietet. Quarkus Native verbessert sich im Vergleich zu Quarkus JVM, jedoch nicht genug um Micronaut in diesem Aspekt einzuholen. Spring Boot Native ist auf dem letzten Platz, verkürzt aber im nativen Modus die Zeiten gegenüber der JVM-Version trotzdem signifikant.

Insgesamt zeigen die Ergebnisse, dass AOT-Kompilierung mit GraalVM nicht nur Startzeit, Speicher- und CPU-Bedarf reduziert, sondern auch messbar kürzere Verarbeitungszeiten ermöglicht. Für Anwendungen mit hohen Anforderungen an Reaktionsfähigkeit stellen native Varianten daher einen klaren Vorteil dar.

8.4.4. Vergleich

Im folgenden tabellarischen Vergleich werden die Messergebnisse der nativen Varianten von Micronaut, Quarkus und Spring Boot gegenübergestellt. Für jedes der vier Bewertungskriterien wird eine Platzierung vergeben. Die finale Bewertung in Tabelle 7 ergibt sich aus der Summe der Platzierungen der Messkategorien aus Tabelle 6. Platz 1 stellt die beste und Platz 3 die schlechteste Bewertung dar. Die Variante mit der niedrigsten Gesamtsumme der Platzierungen erhält die beste Gesamtplatzierung. Für Micronaut Native ergibt sich die Summe 6 aus den Platzierungen Startzeit (2) + RAM-Verbrauch (2) + CPU-Zeit (1) + Verarbeitungszeit (1).

Kriterium	Micronaut Native	Quarkus Native	Spring Boot Native	Platzierung
Startzeit	~ 155 ms	~ 54 ms	~ 238 ms	Quarkus (1) , Micronaut (2), Spring Boot (3)
RAM-Verbrauch	~ 106 MB	~ 76 MB	~ 129 MB	Quarkus (1) , Micronaut (2), Spring Boot (3)
CPU-Zeit	~ 45.57 s	~ 56.42 s	~ 45.72 s	Micronaut (1) , Spring Boot (2), Quarkus (3)
Verarbeitungszeit	12,8 ms	18,7 ms	13,4 ms	Micronaut (1) , Spring Boot (2), Quarkus (3)

Tab. 6: Leistungskennzahlen der nativen Framework-Varianten im direkten Vergleich

Framework – Prototyp-Variante	Summe Platzierungen	Gesamtplatzierung
Micronaut Native	6	1
Quarkus Native	8	2
Spring Boot Native	10	3

Tab. 7: Ermittlung des Gesamtsiegers auf Basis der Platzierungen

Micronaut Native liefert über alle Messkategorien hinweg die beste Gesamtleistung, vor allem bei CPU-Zeit und Verarbeitungszeit.

9. Diskussion und Ausblick

Im Folgenden wird ein Gesamtfazit gezogen, die vorliegende Arbeit sowie die Ergebnisse rückblickend diskutiert und kritisch hinterfragt. Abschließend werden mögliche zukünftige Forschungsansätze und weiterführende Arbeiten aufgezeigt.

9.1. Conclusio

Auf Grundlage der durchgeführten Analysen und des Literaturvergleichs in Kapiteln 3 und 4 lässt sich Teilfrage 1 wie folgt beantworten: Choreografiebasierte Microservice-Architekturen bieten im Vergleich zu orchestrierten und monolithischen Ansätzen wichtige Vorteile durch verbesserte Skalierbarkeit, Flexibilität und Ausfallsicherheit, vor allem durch ihre dezentrale, asynchrone Kommunikation und lose Kopplung. Sie erfordern jedoch eine Investition bei der Planung von Events, dem Fehlerhandling und der Systemüberwachung, um ihre Komplexität beherrschbar zu halten.

Die Literaturrecherche in Kapitel 5 zur Beantwortung der Teilfrage 2 lässt erkennen, dass choreografiebasierte Microservice-Architekturen eine valide und immer noch zukunftsfähige Lösung darstellen. Wichtige Ansätze wie EDA in Verbindung mit modernen Message Brokern schaffen eine lose Kopplung der Services und ermöglichen eine asynchrone, gut skalierbare Lösung. Im Kontext von E-Shop-Anwendungen kann das hilfreich beim Verarbeiten von Bestellungen, Zahlungen oder Lagerbeständen sein. Durch den Einsatz von idempotenten Consumern, Event Sourcing und dem Database per Service Pattern wird die Datenkonsistenz auch in einem verteilten Kontext sichergestellt. Dies ist ein weiterer Aspekt, welcher in Softwarelösungen beim Online-Handel von wichtiger Bedeutung ist.

CQRS und Strangler Fig bieten Wege, um Datenmanipulation effizient zu gestalten und bestehende monolithische Shop-Systeme schrittweise und mit geringerem Risiko in eine Microservice-Architektur zu migrieren. Ein API-Gateway bündelt die Kommunikation nach außen, während DDD dafür sorgt, dass die Services fachlich sinnvoll geschnitten sind und langfristig wartbar bleiben.

Ergänzend dazu adressieren Patterns wie Circuit Breaker, Token-basierte Authentifizierung und Service Discovery typische Anforderungen wie Ausfallsicherheit, Security und dynamische Skalierbarkeit. Observability-Methoden wie Log-Aggregation, Distributed Tracing und der Einsatz von Sidecars bilden die Grundlage für einen besseren Überblick, Engpässe frühzeitig zu erkennen und Fehler schnell zu beheben.

Zusammengefasst zeigt sich, dass für einen modernen und gut skalierbaren E-Shop mit choreografiebasierten Microservices, ein Zusammenspiel der diskutierten Muster, Technologien und Best Practices empfehlenswert ist. Damit lassen sich die Potenziale von Microservices insgesamt noch besser ausschöpfen.

Im Bezug zu Teilfrage 3, lässt sich Anhand der Kapitel 6 und 7 festhalten, dass sich ein E-Shop-Prototyp für digitale Güter mit choreografiebasierten Microservices in Quarkus, Micronaut und Spring Boot jeweils erfolgreich umsetzen lässt. Dabei ermöglichen alle drei Frameworks eine funktionale Umsetzung des Kaufprozesses über eine asynchrone, Event-getriebene Architektur. Quarkus und Micronaut erwiesen sich dabei als besonders gut geeignet für native Builds mit GraalVM, da sie auf Optimierungen zur Kompilierzeit setzen

und geringere Anpassungen erforderten. Spring Boot bot zwar den größten Funktionsumfang und Flexibilität, benötigte jedoch in nativer Ausführung deutlich höheren Aufwand in der Konfiguration und Fehlerbehandlung. Die Unterschiede zeigten sich vor allem in der Integration von JWT, Kafka und MongoDB sowie im Grad der Unterstützung für AOT-Kompilierung und dem Umgang mit Reflection.

Neben den in Kapitel 8.4 dargestellten und diskutierten Messergebnissen lässt sich zur Beantwortung der Hauptforschungsfrage folgendes Resultat festhalten. In einer choreografiebasierten E-Shop-Anwendung zeigen die Messungen, dass Micronaut, Quarkus und Spring Boot in ihren JVM-basierten Varianten insgesamt höhere Startzeiten, Verarbeitungszeiten, einen größeren Speicherverbrauch und eine höhere CPU-Auslastung aufweisen als ihre GraalVM-nativen Ausführungen. Bezogen auf die Startzeit und den Speicherverbrauch schneidet Quarkus Native am besten ab, während Micronaut Native die insgesamt niedrigste CPU-Zeit und Verarbeitungszeit erzielt. Zusammengefasst bietet Micronaut anhand der direkten Gegenüberstellung in Kapitel 8.4.4. die beste Gesamtbilanz.

9.2. Diskussion

Die durchgeführten Messungen liefern bereits wichtige Indikatoren für einen Vergleich zwischen JVM- und GraalVM-basierten Implementierungen. Bei kritischer Betrachtung stellt sich jedoch die Frage, inwiefern die statistische Aussagekraft der Ergebnisse als belastbar einzustufen ist. Es wurden der Mittelwert, Minimum, Maximum und die Standardabweichung ermittelt, aber eine tiefgreifende statistische Aufbereitung und Überprüfung der Verteilungsform der Daten würde zusätzliche Erkenntnisse liefern. Die Wahl des Mittelwertes als zentrale Kennzahl kann in solchen Szenarien bei stark schwankenden Werten ein verzerrtes Bild erzeugen. Weitergehende Analysen, beispielsweise mit Konfidenzintervallen und angestrebter relativer Präzision würden eine noch höhere Aussagekraft bieten.

Besonders bei der Anzahl der Wiederholungen, z.B. 100 Durchläufe bei Messungen der CPU-Sekunden, RAM-Verbrauch, Startzeiten und Verarbeitungszeit der Bestellprozesse, bleibt die Frage offen ob diese Stichprobengröße ausreichend und statistisch belastbar ist. Ziel der Arbeit war ein praxisnaher Performance-Vergleich um Trends und Unterschiede zwischen den Varianten zu erkennen. Die Ergebnisse reichten aus um Tendenzen zu identifizieren und gleichzeitig den Mess- und Aufwertungsaufwand angemessen zu halten. Das gilt insbesondere beim Vergleich der jeweiligen JVM und nativen Varianten bei denen klare Unterschiede und Performancegewinne zu erkennen waren. Dennoch bleibt zu ergründen, inwiefern die beobachteten Unterschiede zwischen den Frameworks selbst tatsächlich technische Ursachen haben oder ein Resultat von Messunsicherheiten sind.

Die Erfassung der Startzeiten erfolgte manuell über die Konsolenausgabe der Frameworks. Dieses Vorgehen ist am einfachsten umzusetzen, es könnte aber sein, dass die Frameworks die Startzeiten unterschiedlich ermitteln. Ein standardisiertes, Framework-unabhängiges Verfahren könnte hier verlässlichere Ergebnisse liefern.

Schließlich bleibt offen, wie sich die Resultate unter unterschiedlichen Lastbedingungen oder längerem Betrieb verändern würden. Die Tests decken typische Szenarien ab, berücksichtigen aber keine Extrem- oder Fehlerfälle. Weiters wurde eine rein lokale Testumgebung ohne Virtualisierung oder Containerisierung verwendet. Dieser Ansatz

minimiert externe Einflüsse, aber eine Cloud- oder Containerumgebung wäre realitätsnäher und für produktive Microservice-Architekturen eher üblich. Trotz lokaler Ausführung sollte geklärt werden, inwiefern Hintergrundprozesse und andere Faktoren wie Hardwaretemperaturen dennoch Einflüsse auf die Messwerte hatten.

Der entwickelte E-Shop-Prototyp umfasst alle wesentlichen Inhalte eines realen Kaufprozesses und stellte für die Benchmarks eine ausreichende Grundlage dar. Der Funktionsumfang führte jedoch dazu, dass sich der Einfluss einzelner Komponenten wie Services oder bestimmter Patterns schwieriger isolieren lassen. Besonders auffällig ist dies bei den CPU-Werten, wo der AuthService deutlich mehr Rechenaufwand als andere Services aufweist. Deshalb wurde hier eine Trennung zwischen Auth- und „Andere“ Services geschaffen, um einen übersichtlicheren Vergleich zu ermöglichen. Mehrere unterschiedliche, aber dafür kleinere Szenarien, hätten hier gezieltere Aussagen ermöglicht, jedoch wären diese weniger realistisch gewesen. Außerdem wurde auf Ausfallsicherheitsmechanismen verzichtet, was den Projektumfang reduzierte, jedoch fehlen dadurch Informationen darüber wie sich das System unter realen Störungen verhalten würde.

Insgesamt ist der gewählte Ansatz ein Kompromiss. Er ist realitätsnah, ohne den Rahmen der Arbeit zu sprengen und lässt Raum für spätere Untersuchungen unter komplexeren Bedingungen.

Im Vergleich zu früheren Studien z. B. Wycislik et al. 2023 [4], Šipek et al. 2020 [5] bei denen Quarkus häufig der Gesamtsieger bei Startzeit und Ressourceneffizienz war, zeigt diese Arbeit, dass Micronaut in der getesteten E-Shop-Architektur bei CPU-Zeit und Verarbeitungszeit deutliche Vorteile erzielen kann. Abweichungen lassen sich möglicherweise auf die höhere Anzahl von Integrationen, z.B. durch eventgetriebene Kommunikation mit Kafka zurückführen. Solche Faktoren wurden in synthetischen Tests früherer Arbeiten nur eingeschränkt berücksichtigt. Die vorliegende Arbeit zeigt damit auch, dass die Wahl des Frameworks nicht nur basierend auf Kennzahlen getroffen werden sollte, sondern zudem die geplante Architektur, Kommunikationsmuster und die Zielumgebung zu bewerten sind.

Zusammenfassend ermöglicht die gewählte Methodik trotz einiger Einschränkungen eine Beantwortung der Forschungsfragen und bietet eine fundierte Basis für einen praxisnahen Vergleich der untersuchten Frameworks. Die Ergebnisse liefern klare Tendenzen und zeigen, wie sich JVM- und native Varianten unter realistischen Bedingungen verhalten. Weitere Forschungen mit umfangreicher statistischer Auswertung und unterschiedlichen Lastszenarien sind sinnvoll um die Ursachen der beobachteten Unterschiede noch präziser einordnen zu können.

9.3. Future Work

Für weiterführende Arbeiten ergeben sich mehrere Ansatzpunkte. Ein möglicher nächster Schritt besteht darin, die Analyse um betriebsnahe Szenarien zu erweitern. Die vorliegenden Messungen wurden in einer lokalen Umgebung unter sehr kontrollierten Bedingungen durchgeführt. Externe Einflüsse wie Netzwerk-Latenzen waren daher nicht Bestandteil der Testumgebung und konnten in dieser Arbeit nicht mitanalysiert werden. Eine praxisnahe Untersuchung unter realistischen Lastsituationen könnte aufzeigen, ob die identifizierten Unterschiede auch in einem produktiven System bestehen bleiben.

Darüber hinaus bietet sich eine weiterführende Untersuchung der Langzeitstabilität und Wartbarkeit von nativen Images an. Während GraalVM Vorteile bei Startzeit, Speicherverbrauch und CPU-Effizienz zeigt, könnten im Rahmen von Wartungsarbeiten wie generellen Updates und Security-Patches Herausforderungen entstehen. Diese gilt es im laufenden Betrieb zu berücksichtigen. Eine weiterführende Arbeit könnte geeignete Build- und Deployment-Pipelines entwickeln und evaluieren wie sich AOT-Kompilierung mit GraalVM hierbei auswirkt.

Ein weiterer Punkt ist die Analyse des Ökosystems der Frameworks. Hier könnte man untersuchen, welche Drittanbieter Bibliotheken oder Plugins bereits nativ kompatibel sind und wie sich deren Einsatz auf die Gesamtperformance auswirkt. Zusätzlich wären weitere Frameworks wie Helidon oder Chronicle Services ebenso Kandidaten für einen ähnlichen Vergleich.

Im Hinblick auf die Methodik, wäre eine präzise statistische Auswertung eines gleichartigen Vergleichsszenarios wertvoll. Eine Analyse der Verteilungsformen, Einsatz von Konfidenzintervallen und Hypothesentests würden helfen Messunsicherheiten zu behandeln und die Aussagekraft der Messergebnisse zu erhöhen.

Zuletzt könnte untersucht werden, wie sich die Verarbeitungszeiten in komplexeren Orchestrierungs- oder Event-Driven-Ansätzen verändern. Ein direkter Vergleich von Choreografie und Orchestrierung unter Berücksichtigung der nativen Varianten könnte neue Erkenntnisse über Zusammenhänge von Architekturmustern und Framework-Technologien liefern.

Die Arbeit zeigt, dass mit nativen Images signifikante Effizienzgewinne möglich sind. Weitere Forschungsarbeiten sollten dieses Potenzial praxisnah vertiefen und dabei das Verhältnis zwischen technischer Optimierung und tatsächlicher Umsetzbarkeit untersuchen.

Literaturverzeichnis

- [1] N. Singhal, U. Sakthivel, und P. Raj, „Selection Mechanism of Micro-Services Orchestration Vs. Choreography“, *Int. J. Web Semantic Technol.*, Bd. 10, Nr. 1, S. 01–13, Jan. 2019, doi: 10.5121/IJWEST.2019.10101.
- [2] G. Pathak und M. Singh, „A Review of Cloud Microservices Architecture for Modern Applications“, *Jul. 2023*, S. 1–7. doi: 10.1109/WCONF58270.2023.10235199.
- [3] Stephen Cass, „The Top Programming Languages 2024“, *IEEE Spectrum*, Verlag der IEEE Computer Society, 22. Aug. 2024. Zugriffen: 15. Mai 2025, [Online]. Verfügbar unter: <https://spectrum.ieee.org/top-programming-languages-2024>
- [4] Ł. Wyciślik, Ł. Latusik, und A. M. Kamińska, „A Comparative Assessment of JVM Frameworks to Develop Microservices“, *Appl. Sci.* 2023 Vol 13 Page 1343, Bd. 13, Nr. 3, S. 1343, Jan. 2023, doi: 10.3390/APP13031343.
- [5] M. Šipek, D. Muharemagić, B. Mihaljević, und A. Radovan, „Enhancing Performance of Cloud-based Software Applications with GraalVM and Quarkus“, in *2020 43rd International Convention on Information, Communication and Electronic Technology (MIPRO)*, Sep. 2020, S. 1746–1751. doi: 10.23919/MIPRO48935.2020.9245290.
- [6] E. Y. S. Sihombing und M. Zarlis, „EVALUATING THE IMPACT OF GRAALVM AND JVM ON MOBILE BANKING MICROSERVICES PERFORMANCE“, *East.-Eur. J. Enterp. Technol.*, Bd. 1, Nr. 13(133), S. 46–58, 2025, doi: 10.15587/1729-4061.2025.315493.
- [7] S. Newman, „Monolith to Microservices“. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [8] C. Davis, „Cloud Native Patterns: Designing Change-Tolerant Software“. Boston, MA, USA: Addison-Wesley, 2019.
- [9] H. B. Hassan, S. A. Barakat, und Q. I. Sarhan, „Survey on serverless computing“, *J. Cloud Comput.* 2021 101, Bd. 10, Nr. 1, S. 1–29, Juli 2021, doi: 10.1186/S13677-021-00253-7.
- [10] S. Wang, „Thin Serverless Functions with GraalVM Native Image“, *Master Thesis, ETH Zurich*, 2021. doi: 10.3929/ethz-b-000480335.
- [11] M. Richards and N. Ford, *Fundamentals of Software Architecture*. Boston, MA, USA: O'Reilly Media, 2021.
- [12] S. Newman, „Building Microservices: Designing Fine-Grained Systems“. Sebastopol, CA, USA: O'Reilly Media, 2015.
- [13] C. Pahl und P. Jamshidi, „Microservices: A systematic mapping study“, *CLOSER 2016 - Proc. 6th Int. Conf. Cloud Comput. Serv. Sci.*, Bd. 1, S. 137–146, 2016, doi: 10.5220/0005785501370146.
- [14] Chris Richardson, „Microservice Architecture pattern“, *Microservices.io* (von Microservices.io; unterstützt durch Kong). Zugriffen: 15. Mai 2025 [Online]. Verfügbar unter: <https://microservices.io/patterns/microservices.html>.
- [15] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, „Microservices: Yesterday, Today, and Tomorrow,“ in *"Present and*

- Ulterior Software Engineering", M. Mazzara and B. Meyer, Eds. Cham, Switzerland: Springer, 2017, pp. 195–216, doi: 10.1007/978-3-319-67425-4_12.
- [16] H. Dinh-Tuan, M. Mora-Martinez, F. Beierle, und S. R. Garzon, „Development Frameworks for Microservice-based Applications: Evaluation and Comparison“, ACM Int. Conf. Proceeding Ser., S. 12–20, März 2022, doi: 10.1145/3393822.3432339.
 - [17] „spring-projects/spring-boot: Spring Boot helps you to create Spring-powered, production-grade applications and services with absolute minimum fuss.“, Zugegriffen: 15. Mai 2025, [Online]. Verfügbar unter: <https://github.com/spring-projects/spring-boot>
 - [18] „quarkusio/quarkus: Quarkus: Supersonic Subatomic Java.“, Zugegriffen: 15. Mai 2025, [Online]. Verfügbar unter: <https://github.com/quarkusio/quarkus>
 - [19] „micronaut-projects/micronaut-core: Micronaut Application Framework“, Zugegriffen: 15. Mai 2025, [Online]. Verfügbar unter: <https://github.com/micronaut-projects/micronaut-core>
 - [20] „H. Schlosser, „Java trends: Top 10 Frameworks in 2020“, devmio Blog, 25. Feb. 2020. Zugegriffen: 20. Juli 2025, [Online]. Verfügbar unter: <https://devm.io/java/java-trends-top-10-frameworks-2020-168867>.
 - [21] P. Plecinski, N. Bokla, T. Klymkovych, M. Melnyk, und W. Zabierowski, „Comparison of Representative Microservices Technologies in Terms of Performance for Use for Projects Based on Sensor Networks“, Sens. 2022 Vol 22 Page 7759, Bd. 22, Nr. 20, S. 7759, Okt. 2022, doi: 10.3390/S22207759.
 - [22] K. Akinepalli -, „Microservices with Spring Boot: Simplifying Distributed Systems“, IJFMR - Int. J. Multidiscip. Res., Bd. 6, Nr. 5, Okt. 2024, doi: 10.36948/IJFMR.2024.V06I05.28930.
 - [23] J. B. Ottinger und A. Lombardi, „Beginning Spring 5“, Begin. Spring 5, 2019, doi: 10.1007/978-1-4842-4486-9.
 - [24] F. Gutierrez, „Pro Spring Boot“, Spring Boot, 2016, doi: 10.1007/978-1-4842-1431-2.
 - [25] „Spring Boot“, Spring Boot. Zugegriffen: 20. Juli 2025. [Online]. Verfügbar unter: <https://spring.io/projects/spring-boot>
 - [26] „Micronaut Core“. Zugegriffen: 20. Juli 2025. [Online]. Verfügbar unter: <https://docs.micronaut.io/latest/guide/index.html#introduction>
 - [27] M. Jeleń und M. Dzieńkowski, „The comparative analysis of Java frameworks: Spring Boot, Micronaut and Quarkus“, J. Comput. Sci. Inst., Bd. 21, S. 287–294, Dez. 2021, doi: 10.35784/JCSI.2724.
 - [28] M. Šipek, B. Mihaljević, und A. Radovan, „Exploring Aspects of Polyglot High-Performance Virtual Machine GraalVM“, Mai 2019, S. 1671–1676. doi: 10.23919/MIPRO.2019.8756917.
 - [29] C. Wimmer u. a., „Initialize once, start fast: application initialization at build time“, Proc ACM Program Lang, Bd. 3, Nr. OOPSLA, Okt. 2019, doi: 10.1145/3360610.
 - [30] T. Vergilio, L. Ha, und A.-L. Kor, „Comparative Performance and Energy Efficiency Analysis of JVM Variants and GraalVM in Java Applications“, Int. J. Environ. Sustain. Green Technol., Bd. 14, S. 1–32, Jan. 2023, doi: 10.4018/IJESGT.331401.

- [31] V. U. Ugwueze, „Cloud Native Application Development: Best Practices and Challenges“, *Int. J. Res. Publ. Rev.*, Bd. 5, Nr. 12, S. 2399–2412, Dez. 2024, doi: 10.55248/GENGPI.5.1224.3533.
- [32] G. A. S. Cassel, V. F. Rodrigues, R. da R. Righi, M. R. Bez, A. C. Nepomuceno, und C. A. da Costa, „Serverless computing for Internet of Things: A systematic literature review“, *Future Gener. Comput. Syst.*, Bd. 128, S. 299–316, März 2022, doi: 10.1016/J.FUTURE.2021.10.020.
- [33] A. P. Cavaleiro und C. Schepke, „Exploring the Serverless First Strategy in Cloud Application Development“, *Proc. - Symp. Comput. Archit. High Perform. Comput.*, S. 89–94, 2023, doi: 10.1109/SBAC-PADW60351.2023.00023.
- [34] M. V. Yurievich, „Startup Latency Analysis in Java Frameworks for Serverless AWS Lambda Deployments.“, *Am. J. Eng. Technol.*, Bd. 7, Nr. 04, S. 16–21, Apr. 2025, doi: 10.37547/TAJET/VOLUME07ISSUE04-03.
- [35] C. Richardson, „Microservices Patterns“. Shelter Island, NY, USA: Manning Publications, 2018
- [36] A. S. Tanenbaum and M. V. Steen, „Distributed Systems: Principles and Paradigms“, 1st ed. Upper Saddle River, NJ, USA: Prentice Hall PTR, 2001.
- [37] F. G. Rocha, M. S. Soares, und G. Rodriguez, „Patterns in Microservices-based Development: A Grey Literature Review“, *Congr. Ibero-Am. Em Eng. Softw. CibSE*, S. 61–76, Apr. 2023, doi: 10.5753/CIBSE.2023.24693.
- [38] S. Speth, S. Sties, und S. Becker, „A Saga Pattern Microservice Reference Architecture for an Elastic SLO Violation Analysis“, 2022 IEEE 19th Int. Conf. Softw. Archit. Companion ICSCA-C 2022, S. 116–119, 2022, doi: 10.1109/ICSCA-C54293.2022.00029.
- [39] A. Megargel, C. M. Poskitt, und V. Shankararaman, „Microservices Orchestration vs. Choreography: A Decision Framework“, *Proc. - 2021 IEEE 25th Int. Enterp. Distrib. Object Comput. Conf. EDOC 2021*, S. 134–141, 2021, doi: 10.1109/EDOC52215.2021.00024.
- [40] F. Baklouti, N. L. Sommer, und Y. Maheo, „Choreography-based vs orchestration-based service composition in opportunistic networks“, *Int. Conf. Wirel. Mob. Comput. Netw. Commun.*, Bd. 2017-October, Nov. 2017, doi: 10.1109/WIMOB.2017.8115771.
- [41] G. Filippone, C. Pompilio, M. Autili, und M. Tivoli, „An architectural style for scalable choreography-based microservice-oriented distributed systems“, *Computing*, Bd. 105, Nr. 9, S. 1933–1956, Sep. 2023, doi: 10.1007/S00607-022-01139-5/FIGURES/7.
- [42] A. Z. H. Kristianto and A. Zahra, „Performance Analysis of Choreography and Orchestration in Microservices Architecture,“ *Journal of Theoretical and Applied Information Technology*, vol. 99, no. 18, pp. 4220–4229, Sep. 2021.
- [43] T. Myllynen, E. Kamau, S. D. Mustapha, G. O. Babatunde, und A. A. Alabi, „Developing a Conceptual Model for Cross-Domain Microservices Using Event-Driven and Domain-Driven Design“, *Int. J. Multidiscip. Res. Growth Eval.*, Bd. 4, Nr. 1, S. 635–638, 2023, doi: 10.54660/IJMRGE.2023.4.1.635-638.

- [44] A. Chavan, „Exploring event-driven architecture in microservices- patterns, pitfalls and best practices“, *Int. J. Sci. Res. Arch.*, Bd. 4, Nr. 1, S. 229–249, Dez. 2021, doi: 10.30574/IJSRA.2021.4.1.0166.
- [45] S. Z. M. Zuki, R. Mohamad, und N. A. Saadon, „Containerized Event-Driven Microservice Architecture“, *Baghdad Sci. J.*, Bd. 21, Nr. 2, S. 584–591, 2024, doi: 10.21123/BSJ.2024.9729.
- [46] A. Rahmatulloh, F. Nugraha, R. Gunawan, und I. Darmawan, „Event-Driven Architecture to Improve Performance and Scalability in Microservices-Based Systems“, *Proc. - Int. Conf. Adv. Data Sci. E-Learn. Inf. Syst. ICADEIS 2022*, 2022, doi: 10.1109/ICADEIS56544.2022.10037390.
- [47] I. Karabey Aksakalli, T. Çelik, A. B. Can, und B. Tekinerdoğan, „Deployment and communication patterns in microservice architectures: A systematic literature review“, *J. Syst. Softw.*, Bd. 180, S. 111014, Okt. 2021, doi: 10.1016/j.jss.2021.111014.
- [48] P. S. Yadav und A. Mantri, „Mitigating Duplicate Message Processing in Microservice Architectures: An Idempotent Consumer Approach“, *J. Artif. Intell. Mach. Learn. Data Sci.*, Bd. 2, Nr. 1, S. 887–891, März 2024, doi: 10.51219/JAIMLD/PURSHOTAM-S-YADAV/214.
- [49] Z. Long, „Improvement and Implementation of a High Performance CQRS Architecture“, *Proc. - 2017 Int. Conf. Robots Intell. Syst. ICRIS 2017*, S. 170–173, Nov. 2017, doi: 10.1109/ICRIS.2017.49.
- [50] A. Messina, R. Rizzo, P. Storniolo, M. Tripiciano, und A. Urso, „The Database-is-the-Service Pattern for Microservice Architectures“, Sep. 2016, S. 223–233. doi: 10.1007/978-3-319-43949-5_18.
- [51] R. Laigner, Y. Zhou, M. A. V. Salles, Y. Liu, und M. Kalinowski, „Data Management in Microservices: State of the Practice, Challenges, and Research Directions“, 22. August 2021, doi: 10.48550/arXiv.2103.00170.
- [52] V. L. Nogueira, F. S. Felizardo, A. M. M. M. Amaral, W. K. G. Assuncao, und T. E. Colanzi, „Insights on Microservice Architecture Through the Eyes of Industry Practitioners“, *Proc. - 2024 IEEE Int. Conf. Softw. Maint. Evol. ICSME 2024*, S. 765–777, Aug. 2024, doi: 10.1109/ICSME58944.2024.00080.
- [53] M. Overeem, M. Spoor, S. Jansen, und S. Brinkkemper, „An empirical characterization of event sourced systems and their schema evolution — Lessons from industry“, *J. Syst. Softw.*, Bd. 178, S. 110970, Aug. 2021, doi: 10.1016/J.JSS.2021.110970.
- [54] F. Alongi, M. M. Bersani, N. Ghielmetti, R. Mirandola, und D. A. Tamburri, „Event-sourced, observable software architectures: An experience report“, *Softw. - Pract. Exp.*, Bd. 52, Nr. 10, S. 2127–2151, Okt. 2022, doi: 10.1002/SPE.3116
- [55] N. Charankar und D. K. Pandiya, „Title: Enhancing Efficiency and Scalability in Microservices Via Event Sourcing“, *Int. J. Eng. Res. Technol.*, Bd. 13, Nr. 4, Mai 2024, doi: 10.17577/IJERTV13IS040252.
- [56] K. D. Jayaraman, „Exploring CQRS Patterns for Improved Data Handling in Web Applications Resagate Global-Academy for International Journals of Multidisciplinary Research“, *J. Today Ideas-Tomorrow Technol.*, Jan. 2025.

- [57] D. Betts, J. Dominguez, G. Melnik und M. Subramanian, *Exploring CQRS and Event Sourcing: A Journey Into High Scalability, Availability and Maintainability with Windows Azure*, Microsoft Developer Guidance, Feb. 14, 2012. ISBN: 978-1621140252.
- [58] J. Kabbedijk, S. Jansen, und S. Brinkkemper, „A case study of the variability consequences of the CQRS pattern in online business software“, ACM Int. Conf. Proceeding Ser., 2012, doi: 10.1145/2602928.2603078.
- [59] P. Rajković, D. Janković und A. Milenković, „Using CQRS Pattern for Improving Performances in Medical Information Systems“, *Proc. of the 6th Balkan Conference in Informatics (BCI 2013)*, Thessaloniki, Griechenland, 19.–21. Sept. 2013, pp. 86–91.
- [60] S. Roh, K. M. Jeong, H. Y. Cho, und E. N. Huh, „An Efficient Microservices Architecture for MLOps“, Int. Conf. Ubiquitous Future Netw. ICUFN, Bd. 2023-July, S. 652–654, 2023, doi: 10.1109/ICUFN57995.2023.10201181.
- [61] C. Y. Li, S. P. Ma, und T. W. Lu, „Microservice Migration Using Strangler Fig Pattern: A Case Study on the Green Button System“, Proc. - 2020 Int. Comput. Symp. ICS 2020, S. 519–524, Dez. 2020, doi: 10.1109/ICS51289.2020.00107.
- [62] O. C. Oyeniran, A. O. Adewusi, A. G. Adeleke, L. A. Akwawa, und C. F. Azubuko, „Microservices architecture in cloud-native applications: Design patterns and scalability“, Comput. Sci. IT Res. J., Bd. 5, Nr. 9, S. 2107–2124, Sep. 2024, doi: 10.51594/CSITRJ.V5I9.1554.
- [63] G. L. D. Villaca, I. F. da Silva, W. K. G. Assunção, R. B. Rocha, und G. C. Fermino, „Strategies for Mitigating Microservice Anti-Patterns in the Pre-Migration of Monolithic Legacy Systems“, S. 268–277, Dez. 2024, doi: 10.5753/ERES.2024.4273.
- [64] G. de A. Vale, F. F. Correia, E. M. Guerra, T. de Oliveira Rosa, J. Fritzsche, and J. Bogner, “Designing Microservice Systems Using Patterns: An Empirical Study on Quality Trade-Offs,” in *Proceedings of the 2022 IEEE 19th International Conference on Software Architecture (ICSA)*, Mar. 2022, pp. 69–79, doi: 10.1109/ICSA53651.2022.00015.
- [65] D. Taibi, V. Lenarduzzi, und C. Pahl, „Architectural patterns for microservices: A systematic mapping study“, CLOSER 2018 - Proc. 8th Int. Conf. Cloud Comput. Serv. Sci., Bd. 2018-January, S. 221–232, 2018, doi: 10.5220/0006798302210232.
- [66] M. G. de Almeida und E. D. Canedo, „Authentication and Authorization in Microservices Architecture: A Systematic Literature Review“, Appl. Sci. 2022 Vol 12 Page 3023, Bd. 12, Nr. 6, S. 3023, März 2022, doi: 10.3390/APP12063023.
- [67] X. He und X. Yang, „Authentication and Authorization of End User in Microservice Architecture“, J. Phys. Conf. Ser., Bd. 910, Nr. 1, Nov. 2017, doi: 10.1088/1742-6596/910/1/012060.
- [68] F. Montesi and J. Weber, “Circuit Breakers, Discovery, and API Gateways in Microservices”, Sep. 2016, doi: 10.48550/arXiv.1609.05830.
- [69] V. Vernon, *Domain-Driven Design Distilled*. Boston, MA, USA: Addison-Wesley, 2016.

- [70] V. Velepucha und P. Flores, „A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges“, IEEE Access, Bd. 11, S. 88339–88358, 2023, doi: 10.1109/ACCESS.2023.3305687.
- [71] H. Vural und M. Koyuncu, „Does Domain-Driven Design Lead to Finding the Optimal Modularity of a Microservice?“, IEEE Access, Bd. 9, S. 32721–32733, 2021, doi: 10.1109/ACCESS.2021.3060895.
- [72] J. Sangabriel-Alarcón, J. O. Ocharán-Hernández, X. Limón, und M. K. Cortés-Verdín, „Domain-Driven Design in Microservices-Based Systems Development: A Systematic Literature Review and Thematic Analysis“, Program. Comput. Softw., Bd. 50, Nr. 8, S. 742–770, Dez. 2024, doi: 10.1134/S0361768824700749/FIGURES/19.
- [73] J. Gilbert, "Cloud Native Development Patterns and Best Practices". Birmingham, UK: Packt Publishing, 2018.
- [74] Falahah, K. Surendro, und W. D. Sunindyo, „Circuit Breaker in Microservices: State of the Art and Future Prospects“, IOP Conf. Ser. Mater. Sci. Eng., Bd. 1077, Nr. 1, S. 012065, Feb. 2021, doi: 10.1088/1757-899X/1077/1/012065.
- [75] G. Aquino, R. Queiroz, G. Merrett, und B. Al-Hashimi, „The Circuit Breaker Pattern Targeted to Future IoT Applications“, Lect. Notes Comput. Sci. Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinforma., Bd. 11895 LNCS, S. 390–396, 2019, doi: 10.1007/978-3-030-33702-5_30.
- [76] N. Madden, "API Security in Action". Shelter Island, NY, USA: Manning Publications Co. LLC, 2020.
- [77] M. B. Jones, J. Bradley und N. Sakimura, „JSON Web Token (JWT)“, RFC 7519, Proposed Standard, IETF, Mai 2015. Zugriffen: 20. Juli. 2025, [Online]. Verfügbar unter: <https://datatracker.ietf.org/doc/html/rfc7519>.
- [78] A. Venčkauskas, D. Kukta, Š. Grigaliūnas, und R. Brūzgienė, „Enhancing Microservices Security with Token-Based Access Control Method“, Sensors, Bd. 23, Nr. 6, März 2023, doi: 10.3390/S23063363
- [79] A. R. Nasab, M. Shahin, S. A. H. Raviz, P. Liang, A. Mashmool, und V. Lenarduzzi, „An empirical study of security practices for microservices systems“, J. Syst. Softw., Bd. 198, S. 111563, Apr. 2023, doi: 10.1016/J.JSS.2022.111563.
- [80] C. Majors, L. Fong-Jones, and G. Miranda, "Observability Engineering: Achieving Production Excellence". Sebastopol, CA, USA: O'Reilly Media, 2022.
- [81] C. Sridharan, "Distributed Systems Observability". Sebastopol, CA, USA: O'Reilly Media, 2018
- [82] J. Cândido, M. Aniche, und A. V. Deursen, „Log-based software monitoring: a systematic mapping study“, PeerJ Comput. Sci., Bd. 7, S. 1–38, 2021, doi: 10.7717/PEERJ-CS.489/.
- [83] J. Levin und T. A. Benson, „ViperProbe: Rethinking Microservice Observability with eBPF“, Proc. - 2020 IEEE 9th Int. Conf. Cloud Netw. CloudNet 2020, Nov. 2020, doi: 10.1109/CLOUDNET51028.2020.9335808.
- [84] B. Li u. a., „Enjoy your observability: an industrial survey of microservice tracing and analysis“, Empir. Softw Engg, Bd. 27, Nr. 1, Jan. 2022, doi: 10.1007/s10664-021-10063-9.

- [85] Swagger.io, „OpenAPI Specification (Version 3.1.1)“, betrieben von SmartBear Software/OpenAPI Initiative (Apache License 2.0). Zugegriffen: 14. August 2025, [Online]. Verfügbar unter: <https://swagger.io/specification/>.
- [86] „Writing REST Services with Quarkus REST (formerly RESTEasy Reactive)“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://quarkus.io/guides/rest>
- [87] „3. POST - Spring Boot vs Micronaut Framework - Building a Rest API“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://guides.micronaut.io/latest/building-a-rest-api-spring-boot-vs-micronaut-post-gradle-java.html>
- [88] „2. Implementing GET - Spring Boot vs Micronaut Framework - Building a Rest API“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://guides.micronaut.io/latest/building-a-rest-api-spring-boot-vs-micronaut-implementing-get-gradle-java.html>
- [89] „Getting Started | Building REST services with Spring“, Getting Started | Building REST services with Spring. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://spring.io/guides/tutorials/rest>
- [90] „Simplified Hibernate ORM with Panache“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://quarkus.io/guides/hibernate-orm-panache>
- [91] „Simplified MongoDB with Panache“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://quarkus.io/guides/mongodb-panache>
- [92] „Access a MongoDB database with Micronaut Data MongoDB“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://guides.micronaut.io/latest/micronaut-data-mongodb-synchronous-gradle-java.html>
- [93] „Micronaut JAX-RS“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://guides.micronaut.io/latest/micronaut-jaxrs-jdbc-gradle-java.html>
- [94] „Getting Started | Accessing Data with MongoDB“, Getting Started | Accessing Data with MongoDB. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://spring.io/guides/gs/accessing-data-mongodb>
- [95] „JPA Query Methods :: Spring Data JPA“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html>
- [96] „Using JWT RBAC“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://quarkus.io/guides/security-jwt>
- [97] „Micronaut JWT Authentication“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://guides.micronaut.io/latest/micronaut-security-jwt-gradle-java.html>
- [98] „OAuth 2.0 Resource Server JWT :: Spring Security“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html>
- [99] „Apache Kafka Reference Guide“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://quarkus.io/guides/kafka>
- [100] „Configuration Reference Guide“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://quarkus.io/guides/config-reference>
- [101] „Kafka and the Micronaut Framework - Event-Driven Applications“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://guides.micronaut.io/latest/micronaut-kafka-maven-kotlin.html>

- [102] „Apache Kafka Support:: Spring Boot“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://docs.spring.io/spring-boot/reference/messaging/kafka.html#messaging.kafka>
- [103] „Tips for writing native applications“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://quarkus.io/guides/writing-native-applications-tips>
- [104] „Micronaut Core“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://docs.micronaut.io/latest/guide/index.html>
- [105] „Creating your first Micronaut Graal application“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://guides.micronaut.io/latest/micronaut-creating-first-graal-app-gradle-java.html>
- [106] „Introducing GraalVM Native Images :: Spring Boot“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://docs.spring.io/spring-boot/reference/packaging/native-image/introducing-graalvm-native-images.html>
- [107] „Quarkus Security with Jakarta Persistence“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://quarkus.io/guides/security-jpa>
- [108] „6. Security Basic Authentication - Spring Boot vs Micronaut Framework - Building a Rest API“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://guides.micronaut.io/latest/building-a-rest-api-spring-boot-vs-micronaut-security-basic-auth-gradle-java.html>
- [109] „Password Storage: Spring Security“. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html>
- [110] „Grafana k6 | Grafana k6 documentation“, Grafana Labs. Zugegriffen: 21. Juli 2025. [Online]. Verfügbar unter: <https://grafana.com/docs/k6/>

Abbildungsverzeichnis

Abb. 1: Schichtenmodell im Monolith [11]	5
Abb. 2: Offene gemeinsame Komponente im Schichtenmodell [11]	6
Abb. 3: Microservice-Architektur Beispiel [14]	7
Abb. 4: Two-Phase-Commit Ablauf	14
Abb. 5: Saga Aufbau - Beispiel anhand eines Restaurants [35]	15
Abb. 6: Microservice Orchestrierung [35]	16
Abb. 7: Event-Choreografie Beispiel [35]	17
Abb. 8: High-Level-Architektur der E-Shop-Lösung	32
Abb. 9: Event-Choreografie des Einkaufsprozesses	33
Abb. 10: Saga-Ablauf im Fehlerfall mit Ausgleichstransaktionen	34
Abb. 11: Aufbau der Testumgebung	49
Abb. 12: Spring Boot Ergebnisse – CPU und RAM	52
Abb. 13: Micronaut und Quarkus Ergebnisse – CPU und RAM	52
Abb. 14: Durchschnittliche Startzeit in ms – Ergebnis je nach Framework und Variante ..	54
Abb. 15: Durchschnittliche Gesamtdauer von Kaufprozessen in ms - je nach Framework und Variante	55
Abb. 16: Ergebnis der durchschnittlichen Dauer in ms von Prozess-Schritten je nach Framework und Variante	56

Tabellenverzeichnis

Tab. 1: Zusammenfassende Darstellung der Literaturrecherche	30
Tab. 2: Überblick der Messergebnisse von CPU-Zeit in s und RAM-Verbrauch in MB. Durchschnitts-, Minimum-, Maximum-Werte und Standardabweichung in s und MB angegeben	51
Tab. 3: Ergebnis der durchschnittlichen Startzeit in ms je nach Framework und Variante. Zusätzlich sind Minimum, Maximum und Standardabweichung in ms angegeben....	54
Tab. 4: Ergebnis der Verarbeitungszeiten von Einkaufsprozessen, Durchschnitt, Median, Minimum-, Maximum-Wert und Standardabweichung in ms	55
Tab. 5: Ergebnis der durchschnittlichen Zwischenschritt-Dauer in ms samt Standardabweichung	57
Tab. 6: Leistungskennzahlen der nativen Framework-Varianten im direkten Vergleich	58
Tab. 7: Ermittlung des Gesamtsiegers auf Basis der Platzierungen	58

Codeausschnittverzeichnis

Code 1: Rest Interface – Quarkus.....	37
Code 2: Rest Interface – Micronaut.....	38
Code 3: Rest Interface – Spring Boot.....	38
Code 4: Datenbankzugriff mit Panache – Quarkus	39
Code 5: Datenbankzugriff mit CRUD Mongorepository – Micronaut.....	39
Code 6: Datenbankzugriff mit Repository – Spring Boot	40
Code 7: Erstellen eines JWT-Tokens in Quarkus.....	40
Code 8: Zugriff auf JWT-Claims in Quarkus.....	40
Code 9: Rollenbasierte Zugriffskontrolle mit Quarkus	41
Code 10: Zugriffskontrolle und JWT-Erstellung – Micronaut.....	41
Code 11: Erstellen eines JWT-Tokens – Spring Boot.....	42
Code 12: JWT-Authentication Filter – Spring Boot.....	42
Code 13: Rollenbasierte Zugriffskontrolle – Spring Boot.....	42
Code 14: Kafka-Integration – Event Emitter in Quarkus	43
Code 15: Kafka-Integration – Message Consumer in Quarkus.....	43
Code 16: Kafka-Konfiguration in Quarkus application.properties	43
Code 17: KafkaClient mit Micronaut – Events senden.....	43
Code 18: KafkaListener mit Micronaut – Events empfangen.....	44
Code 19: KafkaTemplate mit Spring Boot – Events senden.....	44
Code 20: KafkaListener mit Spring Boot – Events empfangen.....	44
Code 21: Spring Boot Kafka-Konfiguration für Deserialisierung	45
Code 22: Reflection-Hinweis für Quarkus mit @RegisterForReflection	45
Code 23: Reflection-Hinweis für Micronaut mit @Serdeable.....	45
Code 24: Reflection-Hinweis für Spring Boot in reflect-config.json.....	46
Code 25: Bcrypt-Hashing mit Quarkus.....	46
Code 26: Bcrypt-Hashing mit Micronaut.....	46
Code 27: Bcrypt-Hashing mit Spring Boot.....	47

Anhang

Anhang 1: Funktionale Anforderungen

ID	Anforderung
1	Benutzer-Authentifizierung und -Autorisierung
1.1	Benutzerregistrierung
1.2	Das System muss neuen Benutzer:innen ermöglichen, sich durch Angabe einer E-Mail-Adresse und eines Passworts zu registrieren.
1.3	Das System muss Benutzereingaben während der Registrierung auf Richtigkeit und Vollständigkeit validieren.
1.4	Das System muss Benutzeranmeldedaten sicher unter Verwendung von Verschlüsselungs- und Hashing-Algorithmen speichern.
1.2	Benutzeranmeldung
1.2.1	Das System muss Benutzer:innen anhand ihrer E-Mail-Adresse und ihres Passworts authentifizieren.
1.2.2	Das System muss bei fehlgeschlagenen Anmeldeversuchen Rückmeldungen bereitstellen.
1.2.3	Das System muss rollenbasierte Zugriffskontrolle unterstützen (z. B. "Admin", "Benutzer").
2	Verwaltung von Benutzerprofilen
2.1	Profilerstellung und -anzeige
2.1.1	Das System muss automatisch ein Benutzerprofil bei der Registrierung erstellen.
2.1.2	Das System muss Benutzer:innen ermöglichen, ihre Profilinformationen einschließlich persönlicher Daten und Kaufhistorie einzusehen.
2.2	Profilaktualisierung
2.2.1	Das System muss Benutzer:innen ermöglichen, ihre persönlichen Informationen wie Name, Kontaktdaten und Präferenzen zu aktualisieren.
2.2.2	Das System muss Aktualisierungen auf Richtigkeit validieren.
3	Produktverwaltung (Admin-Funktionen)
3.1	Produkterstellung
3.1.1	Das System muss Administratoren ermöglichen, neue Produkte zu erstellen, indem sie Titel, Beschreibung, Preis und Kategorie angeben.
3.1.2	Das System muss Produktinformationen auf Vollständigkeit und Richtigkeit validieren.
3.2	Produktaktualisierung
3.2.1	Das System muss Administratoren ermöglichen, bestehende Produktdetails zu aktualisieren.
3.3	Produktlöschung
3.3.1	Das System muss Administratoren ermöglichen, Produkte zu deaktivieren oder zu löschen.

3.3.2	Das System muss sicherstellen, dass deaktivierte Produkte für Endbenutzer:innen nicht sichtbar sind.
3.4	Kategorienverwaltung
3.4.1	Das System muss Administratoren ermöglichen, Produktkategorien zu erstellen, zu aktualisieren und zu löschen.
4	Produktaufistung und -suche
4.1	Produktaufistung
4.1.1	Das System muss Benutzern eine Liste verfügbarer Produkte anzeigen.
4.1.2	Das System muss Pagination, Sortierung und Filterung der Produktlisten unterstützen.
4.1.3	Das System muss sicherstellen, dass nur aktive Produkte angezeigt werden.
4.2	Produktdetails
4.2.1	Das System muss detaillierte Informationen über ein Produkt bereitstellen, einschließlich Beschreibungen, Preis und Lizenzinformationen.
4.3	Produktsuche
4.3.1	Das System muss Benutzer:innen ermöglichen, nach Produkten anhand von Schlüsselwörtern zu suchen.
4.3.2	Das System muss erweiterte Suchfilter unterstützen, wie Kategorie, Preisspanne und Bewertungen.
5	Warenkorbverwaltung
5.1	Hinzufügen zum Warenkorb
5.1.1	Das System muss Benutzer:innen ermöglichen, Produkte zu ihrem Warenkorb hinzuzufügen.
5.2	Warenkorbanzeige
5.2.1	Das System muss Benutzer:innen ermöglichen, alle Artikel in ihrem Warenkorb anzusehen.
5.2.2	Das System muss Produktdetails und den Gesamtpreis des Warenkorbes anzeigen.
5.3	Warenkorbaktualisierung
5.3.1	Das System muss Benutzer:innen ermöglichen, die Menge der Artikel zu ändern oder Artikel aus dem Warenkorb zu entfernen.
5.3.2	Das System muss Mengenänderungen validieren (z. B. Mindest- und Höchstgrenzen).
5.4	Warenkorb leeren
5.4.1	Das System muss Benutzer:innen ermöglichen, alle Artikel in ihrem Warenkorb mit einer einzigen Aktion zu entfernen.
6	Bestellabwicklung
6.1	Einleitung des Checkouts
6.1.1	Das System muss Benutzer:innen ermöglichen, den Checkout-Prozess von ihrem Warenkorb aus zu starten.
6.2	Bestellerstellung
6.2.1	Das System muss eine Bestellung basierend auf den Artikeln im Warenkorb der Benutzer:innen erstellen.
6.3	Bestellbestätigung

6.3.1	Das System muss Bestelldetails bereitstellen, einschließlich einer Zusammenfassung der gekauften Artikel und der Gesamtkosten.
7	Zahlungsabwicklung
7.1	Zahlungsmethoden
7.1.1	Das System muss Benutzer:innen verfügbare Zahlungsoptionen anzeigen.
7.2	Zahlungsbestätigung
7.2.1	Das System muss der Benutzer:innen die erfolgreiche Zahlungsabwicklung bestätigen.
7.3	Umgang mit Zahlungsfehlern
7.3.1	Das System muss Zahlungsfehler elegant handhaben und Benutzer:innen über das Problem informieren.
7.3.2	Das System muss Benutzer:innen ermöglichen, die Zahlung erneut zu versuchen oder eine andere Zahlungsmethode zu wählen.
8	Lizenzgenerierung und -auslieferung
8.1	Lizenzgenerierung
8.1.1	Das System muss für jedes gekaufte Produkt eine eindeutige digitale Lizenz generieren.
8.1.2	Das System muss Lizenzen den entsprechenden Benutzer:innen und der Bestellung zuordnen.
8.2	Lizenzzustellung
8.2.1	Das System muss die digitale Lizenz per E-Mail oder über das Benutzerkonto an die Benutzer:innen liefern.
8.3	Lizenzverwaltung
8.3.1	Das System muss Benutzer:innen ermöglichen, ihre gekauften Lizenzen in ihrem Konto anzusehen und zu verwalten.
8.3.1	Das System muss Lizenzdetails anzeigen, wie Aktivierungscodes und Ablaufdaten.
9	Benachrichtigungen
9.1	E-Mail-Benachrichtigungen
9.1.1	Das System muss E-Mail-Benachrichtigungen für wichtige Ereignisse senden, einschließlich: Registrierungsbestätigung, Bestellbestätigungen, Zahlungsbelege, Lizenzzustellung.
9.1.2	Das System muss E-Mails mit benutzerspezifischen Informationen personalisieren.
10	Tracking und Analysis
10.1	Ereignisprotokollierung
10.1.1	Das System muss wichtige Ereignisse für Überwachungs- und Analysezwecke protokollieren.

Anhang 2: Kennzeichnung der Nutzung von KI-Software

KI-basiertes Hilfsmittel	Einsatzform	Betroffene Teile der Arbeit	Bemerkungen
DeepLTranslator https://www.deepl.com/	Übersetzung von Textpassagen	Englische Kurzfassung – Abstract	Anschließende Kontrolle und Nacharbeitung wurde durchgeführt.
ChatGPT (OpenAI) https://chatgpt.com/	Hilfe bei Erstellung von Python Skripten zum Aufarbeiten und Visualisieren von Messdaten	Kapitel 8.4	
Consensus https://consensus.app/	Literaturrecherche – Hilfe bei Suche von relevanter Literatur	Kapitel 3-5	Verwendet wurden die Suchfunktion und die Zusammenfassungen um Relevanz der Literatur schneller einzuschätzen.
ChatGPT (OpenAI) https://chatgpt.com/	Gelegentliche Unterstützung bei sprachlicher Ausdrucksweise und Formulierung komplexerer Gedankengänge, Korrektur von Grammatikfehlern	Vereinzelt in Kapiteln 2-9	Keine inhaltliche Textgenerierung, ausschließlich genutzt um einzelne Formulierungen sprachlich präziser auszudrücken. Größtenteils in Eigenverantwortung überarbeitet.
Claude (Anthropic) https://claude.ai/	Unterstützung bei Problemen während der Entwicklung	Kapitel 7	Analyse von Fehlerlogs, Korrekturen im Code und in den Konfigurationen, Hilfestellung bei Lösen von Problemen insbesondere bei Verwendung von GraalVM